

Управление процессами.....	4
Процессы UNIX.....	6
Оптимизация fork.....	7
В UNIX System V.....	7
В UNIX BSD.....	8
К лабораторной по предкам, потомкам, зомби и сиротам.....	8
Операционная система.....	12
Алгоритмы планирования.....	12
Планирование в системах пакетной обработки.....	13
Планирование в системах с разделением времени.....	14
Потоки.....	15
Модели потоков (базовая модель).....	16
40) Однопоточная модель процесса.....	16
41) Многопоточная модель процесса.....	17
Типы (уровни) потоков.....	17
42) Потоки уровня пользователя.....	17
43) Потоки уровня ядра.....	18
Типы параллелизма.....	20
Взаимодействие параллельных процессов. Уровни наблюдения.....	23
59) Программный способ.....	26
Алгоритм Деккера (Петерсона).....	27
Алгоритм Лампорта (Булочная Лампорта).....	28
60) Аппаратный способ.....	29
Семафоры.....	32
Классические семафоры Дейкстры.....	32
Задача производства потребления.....	33
Код “производство-потребление” с сегментом разделяемой памяти.....	36
Обедающие философы.....	41
Семафоры UNIX.....	42
Мониторы.....	45
Простой монитор.....	45
Монитор “Кольцевой буфер”.....	46
Читатели писатели (монитор Хоара).....	48
Код Linux.....	50
Код Windows.....	54
Синхронизация и взаимоисключение.....	59
Способ взаимодействия параллельных процессов: рандеву.....	60
Распределенные системы.....	61
Синхронизация по часам.....	61
Алгоритм временных отметок Лампорта.....	63
Особенности передачи сообщений в распределенных системах.....	65
Алгоритмы взаимодействия процессов в распределенных системах и подходы.....	66
1) Централизованный алгоритм.....	66

Алгоритм записи.....	67
2) Распределенный алгоритм.....	68
3) Token ring.....	68
Транзакции.....	69
Свойства транзакций (ACID):.....	70
2 механизма транзакций.....	71
Протокол двухфазной идентификации транзакций	
В распределенных системах фиксации транзакций могут потребовать	
взаимодействия нескольких процессов на разных системах (узлах). При этом у	
каждого узла свои бд, файлы => очень часто используется специальный	
протокол.....	71
Тупики.....	73
4 условия, необходимые и достаточные для возникновения тупика.....	74
Подходы к борьбе с тупиками.....	75
1 подход: Опережающее требование.....	76
Улучшение подхода 1.....	76
2 подход: Иерархическое распределение.....	76
3 подход.....	77
Классический алгоритм (Дейкстра) – алгоритм банкира.....	77
Алгоритм Хаберманна.....	80
Метод редукции графа.....	84
Алгоритм Канди-Мисра-Хаасс (на основе пробного сообщения).....	87
Управление оперативной памятью.....	88
Связное распределение.....	89
Первое поколение.....	89
Статическое распределение.....	89
Динамическое распределение.....	91
Как будто все, что было до, подразумевает само по себе наличие	
логического ап.....	93
Несвязное распределение.....	94
Виртуальная память.....	94
Страничное распределение.....	95
3 способа преобразования.....	97
Прямое преобразование.....	97
Ассоциативное отображение.....	98
Ассоциативное прямое отображение.....	101
Схема страничного управления памятью с гиперстраницами.....	102
Замещение страниц.....	103
1. Выталкивание случайной страницы.....	103
2. FIFO.....	104
3. Least recently used (LRU).....	105
4. Least Frequently Used (LFU).....	106
5. Not Used Recently (NUR)	
Выталкивание не использовавшейся в последнее время страницы.....	107
Теория рабочего множества.....	108

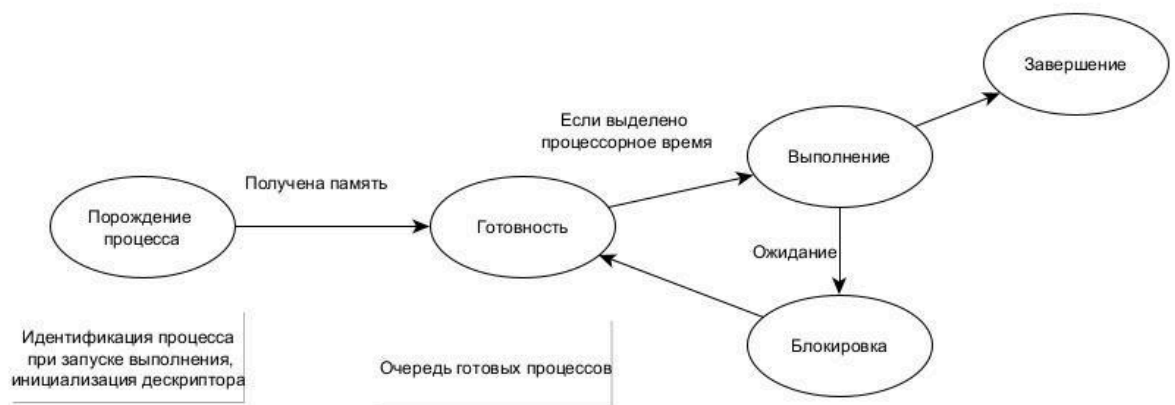
График зависимости длительности периода между прерываниями (кривая линии жизни).....	112
Влияние размера страниц при одном и том же размере физической памяти... 113	
Зависимость частоты обращения к странице от количества команд на странице, выполненных до передачи управления на другую страницу.....	114
Управление сегментами по запросу.....	115
3 способа организации таблиц сегментов.....	117
Проблемы с управлением сегментами по запросу.....	119
Управление сегментами, разделенными на страницы, по запросу.....	120
Вопрос динамического связывания и вопрос отложенного связывания.....	122
PAE.....	124
Структура ядра ОС.....	128
Иерархическая машина Медника-Донована.....	128
Микроядерная архитектура.....	131
Начальный бюджет, период потока.....	134
MSI.....	134
Точные и неточные прерывания.....	136
Режимы работы устройств.....	139
Концептуальная трехшинная архитектура.....	140
Прерывания.....	143
Функции INT8h.....	146
Программный канал в Unix.....	147
Программный канал.....	147
Вызов ps.....	148
ps -ajx.....	148
Process flags:.....	149
Process State Codes.....	149
7 типов файлов.....	149
Hardlink, softlink.....	150
Приоритеты.....	151
IPC (inter process communication).....	151
Сигналы.....	151
Семафоры UNIX.....	154
Сегменты разделяемой памяти.....	156
Очереди сообщений.....	158
Демон.....	160
Правила программирования демона.....	160
Код демона.....	163
Потоки.....	169
Функции.....	170
Функции системного таймера.....	171
RPC.....	172
RPCgen.....	174
bakery.x.....	176

bakery_client.c.....	177
bakery_server.c.....	181

- 1) **Файловая система** — набор правил , определяющий способ организации, хранения и именования данных на носителях
- 2) **Вторичная память** — энергонезависимая память, предназначенная для долговременного хранения информации
- 3) **Файл** — последовательность байт
(структура файла и его содержимое определяется программой, которая его обрабатывает. ОС никак не интерпретирует содержимое файла)
- 4) **Файл** — **именованная** совокупность данных, может быть бессмысленных
- 5) **Каталог** — файл, содержащий информацию о файлах, созданных в каталоге.
(но его содержимое обрабатывает ОС, а не программа пользователя)
- 6) **Команды и данные** хранятся в одной памяти, последовательно
- 7) **Процесс** — программа в стадии выполнения (основная абстракция ОС)
- 8) **Квант** — процессорное время, выделяемое конкретному заданию
- 9) **Квантование времени** обеспечивает гарантированное время ответа ОС
- 10) **Мультипрограммная (мультизадачная) система** — система, в памяти которой находится несколько программ
- 11) **Многопроцессная система** — выполняется много программ

Управление процессами

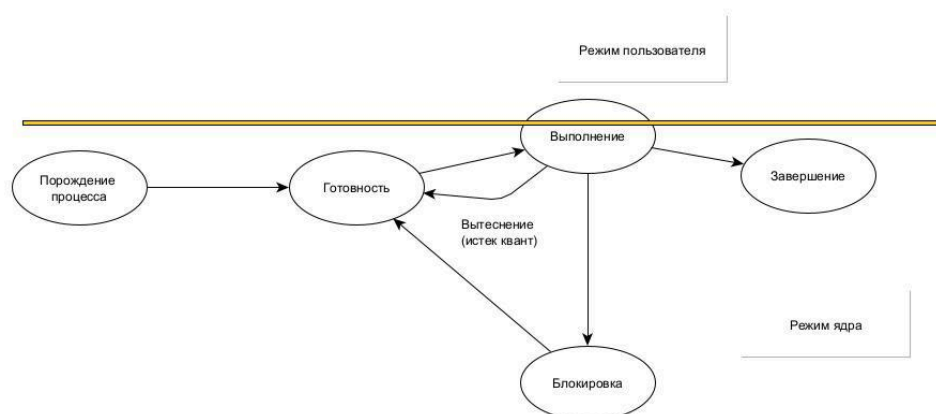
- 12) Самая общая диаграмма состояния процесса



- 13) В современных системах средство описания процесса — структура
- 14) Дескриптор процесса — предопределенная в ОС структура, предназначенная для описания процесса
- 15) Планирование — постановка процессов в очередь
- 16) Диспетчеризация — процесс выделения процессу процессорного времени
- 17) Типы планирования: беспriorитетное (равноправное), приоритетное (с вытеснением, без вытеснения)
Процесс с более высоким приоритетом вытесняется процессом с более низким.

Если приоритетов нет, то процесс выполняется до конца.

- 18) В UNIX идентификатор процесса — число. Процессу выделяется строка в таблице процессов (ее как бы нет, но есть несколько связанных списков, которые позволяют организовать динамическое изменение)
- 19) Планировщик — специальный модуль для организации очереди готовых процессов в системе. Задача: постановка процессов в очередь в соответствии с выбранной системой планирования.
- 20) Процесс — часть времени выполняется в режиме пользователя (в режиме задачи, выполняет собственный код), часть — в режиме ядра (выполняет реентерабельный код ОС)
- 21)

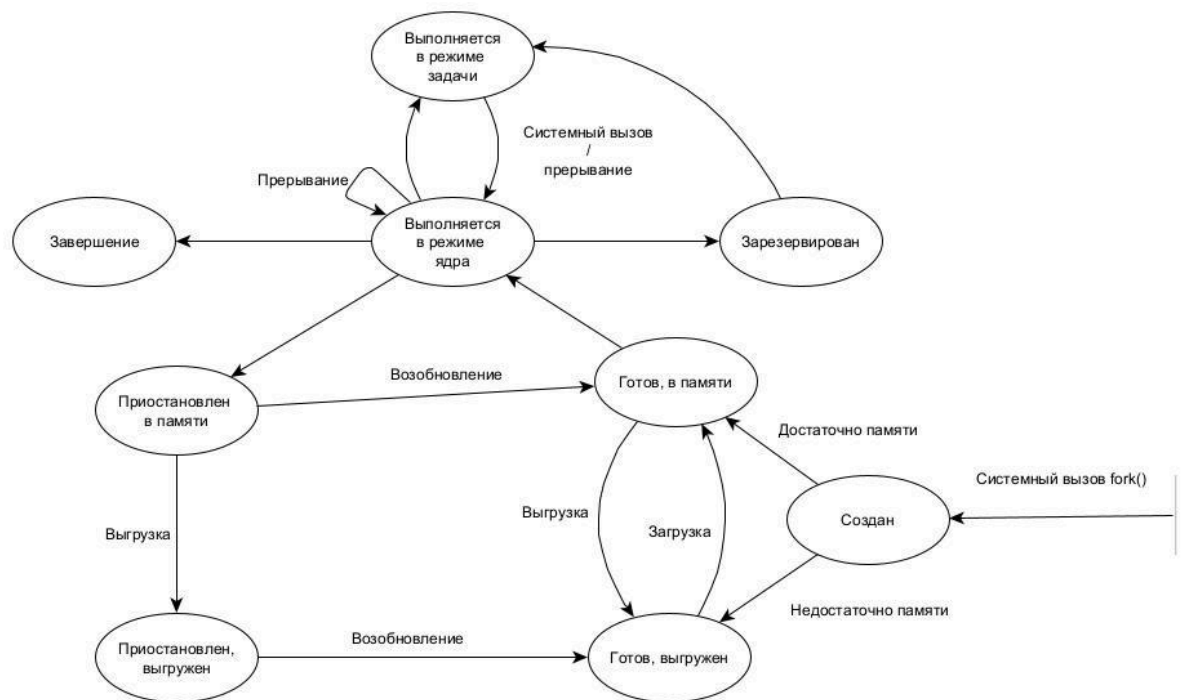


Создать процесс может только ядро. Выделить ресурсы, поставить в очередь может только ОС. Заблокировать/разблокировать может только ОС.

- 22) Процесс — единица декомпозиции системы

Процессы UNIX

- Диаграмма состояния процессов в UNIX



- Каждый процесс в UNIX проходит через состояние “зомби”
- Процесс часть времени выполняется
 - в режиме задачи (режиме пользователя)
 - выполняет собственный код
 - в режиме ядра
 - выполняет реентерабельный(системный) код
- В UNIX любой процесс может породить любое количество процессов системным вызовом `fork`
- В результате системного вызова `fork()` система переходит в режим ядра и выполняется вызванная в результате системного вызова функция ядра:
 - в Linux – `clone()`
 - в FreeBSD – `fork()`
- Системный вызов — процесс вызывает систему, чтобы она его обслужила
- В режиме ядра могут выполняться только функции ядра

- В результате системного вызова `fork` создается новый процесс – процесс-потомок, который является копией процесса-предка в том смысле, что процесс-потомок наследует:
 - код предка
 - сигнальную маску
 - маску режимов создания файлов
 - дескрипторы открытых файлов
- В старых UNIX для процесса-потомка создается собственное адресное пространство – карты трансляции адресов
 - в это адресное пространство копировался код предка
 - В результате в системе могло быть несколько копий одной программы (что неэффективно)
- Пояснение: уже тогда системы имели виртуальное адресное пространство
- В современных системах чтобы выделить виртуальное адресное пространство, нужно создать таблицы страниц (картами трансляций адресов являются таблицы страниц)

Оптимизация `fork`

В UNIX System V

- Для процесса-потомка создается собственное адресное пространство (собственные таблицы страниц), которые ссылаются на страницы адресного пространства предка
 - дескрипторы таблиц страницы у потомка содержали адреса, в которых указывались дескрипторы таблиц страниц адресного пространства предка
- Для страниц адресного пространства предка права доступа менялись с read-write на only-read и устанавливался флаг copy-on-write
- Это имеет смысл для сегмента данных и сегмента стека, поскольку код изменить нельзя в процессе выполнения программы
#комментарий на этом моменте от Натальи Юрьевны
“Ребят, вы какие-то безумные”
- Если или предок, или потомок попытается обратиться (изменить) к какой-то странице, возникнет исключение по правам доступа

- Начав обрабатывать исключение, система найдет флаг `copy-on-write` и создаст копию страницы в адресном пространстве того процесса, который попытался ее изменить
- Таким образом создается копия отдельных нужных страниц, а не всего адресного пространства.
- Такая ситуация в системе существует, пока потомок не вызовет системный вызов `exit()` или системный вызов `exec()`
- Система не анализирует, кто вызвал `exit()` или `exec()` (так как любой потомок может стать предком)
(так как чтобы все это определить, нужно провести анализ, а это очень дорого по времени)
(Коды программы ОС не интерпретирует, не анализирует. Также как файловые системы не рассматривают содержимое файлов. Файла для файловой системы – набор байт)
- Коллективное использования страниц:
 - Два процесса пишут разное, в результате может быть потеря данных
 - Решение: флаг `copy-on-write`

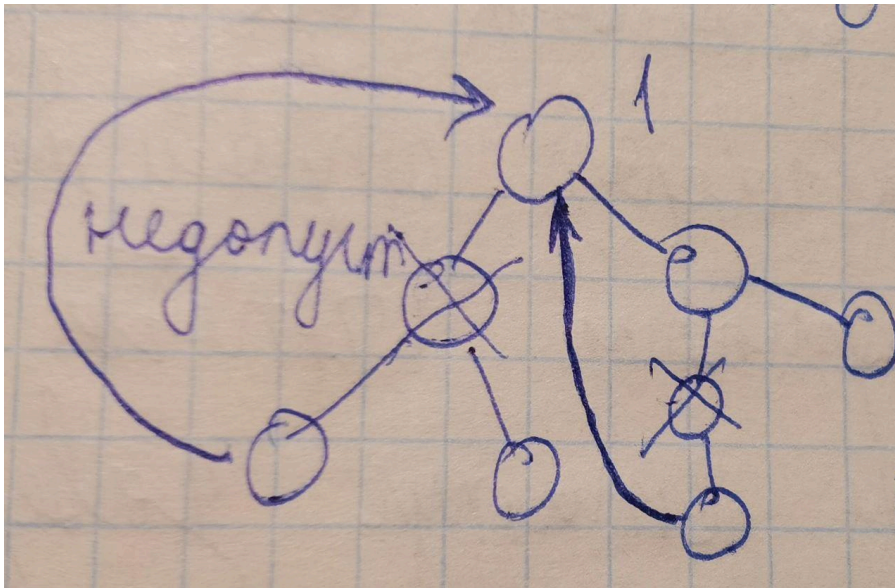
В UNIX BSD

- Реализован системный вызов `vfork()`
- Для потомка не создаются собственные таблицы страниц
- Потомок начинает выполняться в адресном пространстве предка, а предок блокируется до тех пор, пока потомок не вызовет `exit` или `exec`.
- В современных системах реализована страничная система
 - может быть замена страницы на страницу
 - страница = 4 КБ

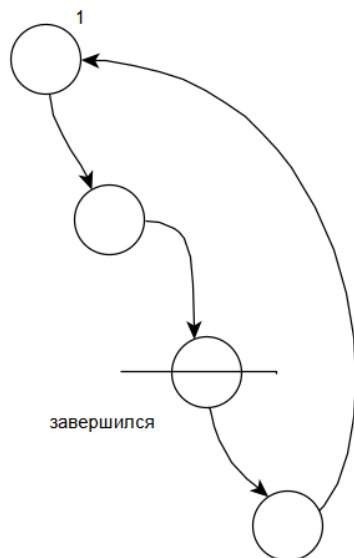
К лабораторной по предкам, потомкам, зомби и сиротам

- `exit()` — системный вызов завершения процесса
 - в отличие от `return` – возврат управления
- `exec()` — системный вызов, в результате которого процесс переходит на выполнение программы, указанной как параметр `exec` (то есть переходит на другое адресное пространство)

- то есть адресное пространство предка не изменяется => можно возвращать обычные права => флаг Copy on write сбрасывается.
- поэтому пишут, что `exec()` создает низкоуровневый процесс
- процесс на самом деле не создается, но создается новое адресное пространство программы, переданной в качестве параметра `exec`
- чтобы перевести процесс на новое адресное пространство, в его дескрипторе адрес старых таблиц заменяется на адрес новых таблиц страниц
- (Для ОС существует полное имя файла(путь+имя). Система разбирает путь, что определить, существует файл или нет. Проверки выполняются в режиме ядра. Если все проверки выполнены успешно, то начинаются действия по созданию нового адресного пространства(в случае вызова `fork()`).
- Таблицы страниц не выделяются, а создаются
- "Статус, что ли, чтобы ее устанавливать"
- `pid = fork()`:
 - `pid = 0` — для потомка
 - процессу родителю возвращается `id` потомка
 - `-1` если произошло аварийное завершение (по соглашению UNIX)
- `getpid()` – возвращает идентификатор процесса
- `getppid()` — возвращает идентификатор процесса-предка
- Понятия потомок и предок обусловлены, так как Unix поддерживает иерархию отношений предок-потомок.
- Иерархия поддерживается структурой, которая описывает процесс, то есть дескриптором процесса, и полями этой структуры, которые содержат указатели на структуры предка и потомков
- В системе всегда есть процесс с идентификатором:
 - `0` — процесс, запустивший систему
 - `1` — процесс, открывший терминал
 - терминальный процесс — является предком всех процессов, запущенных в этом терминале



- Unix не может допустить разрушения иерархии.
- При завершении любого процесса система проверяет, не остались ли у него незавершившиеся потомки.
- Процесс, у которого завершился предок, называется процесс-сирота и его усыновляет терминальный процесс.
- Потомки получают адрес нового предка, а усыновивший – адрес новых потомков. В результате иерархия сохраняется.
- Система не принимает сложных решений:



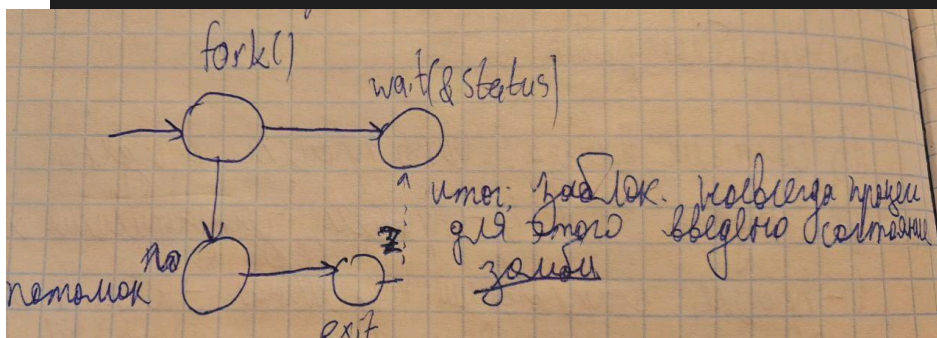
- Чтобы процессы-предки не завершались раньше процессов-потомков, в UNIX имеется системный вызов **wait()**

- **wait(&status)** — блокирует процесс до завершения процесса-потомка.
Система не контролирует, вызывал ли процесс wait.
- **waitpid(childpid, &status, WUNTRACED)**
- Смысл wait — получить статус завершения, проанализировать его и в зависимости от него ветвиться

```

if (WIFEXITED(status))
{
    printf("exited, status=%d\n", WEXITSTATUS(status));
}
else if (WIFSIGNALED(status))
{
    printf("killed by signal %d\n", WTERMSIG(status));
}
else if (WIFSTOPPED(status))
{
    printf("stopped by signal %d\n", WSTOPSIG(status));
}

```



- Итог: процесс не может получить статус завершения и заблокируется навсегда, что плохо, так как занимает ресурсы
=> в Unix введено состояние зомби
- Зомби — процесс, у которого отобраны все ресурсы кроме последнего (дескриптора, чтобы предок мог получить статус завершения потомка и продолжить выполнение)

Операционная система

- 23) ОС — комплект программ, которые совместно управляют ресурсами вычислительной системы и процессами, которые используют эти ресурсы при вычислениях
- 24) Основная задача ОС — выделение процессам ресурсов
- 25) Ресурс — любой компонент вычислительной системы и предоставляемые ей возможности:
- процессорное время
 - объем оперативной памяти – количество ОЗУ, которое необходимо процессу для выполнения
 - устройства ввода-вывода
 - каналы, таймеры, объекты ядра (семафоры, разделяемая память, очередь сообщений)
 - Реентерабельные коды ОС и системные таблицы
- 26) Реентерабельный код — код чистой процедуры или процедуры, которая может обращаться сама к себе (разные процессы могут быть в разных точках этой процедуры — достигается исключением из кода данных(глобальных переменных). Это достигается за счет системных таблиц, в которых хранятся данные, с которыми работает ОС. Доступ к системным таблицам контролируется)

Алгоритмы планирования

- 27) Типы планирования: бесприоритетное (равноправное), приоритетное (с вытеснением, без вытеснения)
Если ОС без приоритетов, то процессы выполняются в порядке очереди FIFO.
Если с приоритетами, то высокоприоритетный процесс может вытеснить менее приоритетный процесс.
- 28) Приоритеты:
- а) абсолютные или относительные
 - б) статические (назначается процессу до выполнения) или динамические (меняется в процессе выполнения)
- 29) В системах разделения времени по истечении кванта переключение процессов
- а) с вытеснением / без вытеснения

- b) с переключением / без переключения
- 30) В системах пакетной обработки время не квантуется, но могут быть приоритеты и вытеснение (у нас системы разделения времени)

Планирование в системах пакетной обработки

a) FIFO

(FCFS – *супостаты перефразируют* ©)

Планирование без приоритета, без вытеснения.

Но, конечно, возможна блокировка при запросе ресурса, процесс будет поставлен в конец очереди.

b) Shortest Job First (наикратчайшее задание первым – более высокий приоритет коротким задачам)

IBM360->показатель эффективности->количество задач в единицу времени:

- i) Больше выполнение коротких задач в единицу времени
- ii) Бесконечное откладывание (приоритет отдается коротким задачам, поэтому задачи, требующие большого числа вычислений, постоянно откладываются)

“Была обнаружена задача, которая не была выполнена за 40

лет”

Бесконечное откладывание — один процесс в течение длительного времени не может получить нужный ему ресурс.

c) Shortest Remaining Time (наименьшее оставшееся время)

- i) Вытеснение процесса с выполнения, если поступил процесс с меньшим **оценочным** временем выполнения
- ii) Накладные расходы на слежку за временем выполнения текущего процесса, сравнение его с оценочным временем поступившего процесса.
- iii) Бесконечное откладывание(еще большее)

d) Highest Response Ratio Next

Используется пересчет приоритетов для исключения бесконечного откладывания

- i) $p = (t_w + t_s) / t_s$ (p - приоритет, t_w - время ожидания процесса в очереди, t_s - запрошенное время обслуживания)

Чем дольше процесс находится в очереди готовых процессов, тем выше у него приоритет => процесс перемещается в очереди и выполняется.

31) Динамический приоритет только у пользовательских процессов

Планирование в системах с разделением времени

- a) RR — Round Robin — очередь с переключением
 - i) Процесс выполняется в выделенный ему квант. Если не успел — в конец очереди.
Если реализовано приоритетное планирование с вытеснением, то процесс с более высоким приоритетом вытесняет с выполнения процесс с более низким приоритетом.
 - ii) Проблема — статический квант времени
- b) Адаптивное планирование, или многоуровневые очереди
 - i) Несколько очередей, в которых находятся процессы с одинаковыми приоритетами
 - ii) В самой приоритетной — вновь созданные и завершившие ожидание ввода-вывода (т. е. получившие ресурс). Процессам из этой очереди выделяется квант, размер которого выбирается так, чтобы больше(?) процессов успели завершиться или выполнить I/O.
 - iii) Если процесс за квант не завершился или не выполнил запрос I/O — перемещается в очередь с меньшим приоритетом. И так далее, до самой низкой очереди.
Холостой процесс (ОС не может ничего не делать).
 - iv) В последней очереди — вычислительные процессы
- c) Min Max RR
- d) Average Max RR
- e) Shortest Remaining Burst RR

Алгоритмы Min Max RR, Average Max RR, Shortest Remaining Burst RR решают вопрос повышения эффективности планирования по алгоритму RR путем назначения процессам динамического кванта времени.

Windows декларируется как RR

Потоки

32) Потоки как стремление к сокращению расходов, связанных с полным переключением контекста

33) Полный контекст состоит из:

- аппаратный контекст — содержимое регистров процесса
- информация о ресурсах, которыми владеет процесс (например, указатели на структуры, связанные с управлением памятью)

Переключение полного контекста происходит при переключении процессора на выполнение другого процесса.

Syscall – переключение только аппаратного контекста.

Чтобы восстановить выполнение процесса, нужно сохранять контекст.

Состояние процесса изменяется постоянно, то есть переключение полного контекста – постоянное действие, а не единичное.

34) Поток — непрерывная часть кода программы, которая может выполняться параллельно с другими частями кода

35) Поток создается в процессе, при этом мы сами создаем части программы, которые распараллеливаются

36) Процесс с точки зрения темы “процессы и потоки” имеет две характерные черты:

a) владение ресурсами

i) **Владельцем ресурсов в системе является процесс.**

Ему принадлежит адресное пространство, которое в наших системах является виртуальным и защищенным. При выполнении процессу выделяется физическая память (страницы физической памяти), но т.к. процесс выполняется как параллельные потоки, то выдается потокам

ii) Ресурсы:

(1) Файлы

(2) Устройства ввода-вывода

(3) Любые разделяемые ресурсы

b) Планирование (постановка процесса в очередь) и выполнение (непосредственное выполнение процесса, когда он получает процессорное время в результате диспетчеризации)

37) В результате такого подхода, поток становится единицей диспетчеризации (именно им выделяется процессорное время)

- 38) Говорят, что потоку принадлежит счетчик команд и аппаратный контекст
- 39) При переключении потоков переключается только аппаратный контекст

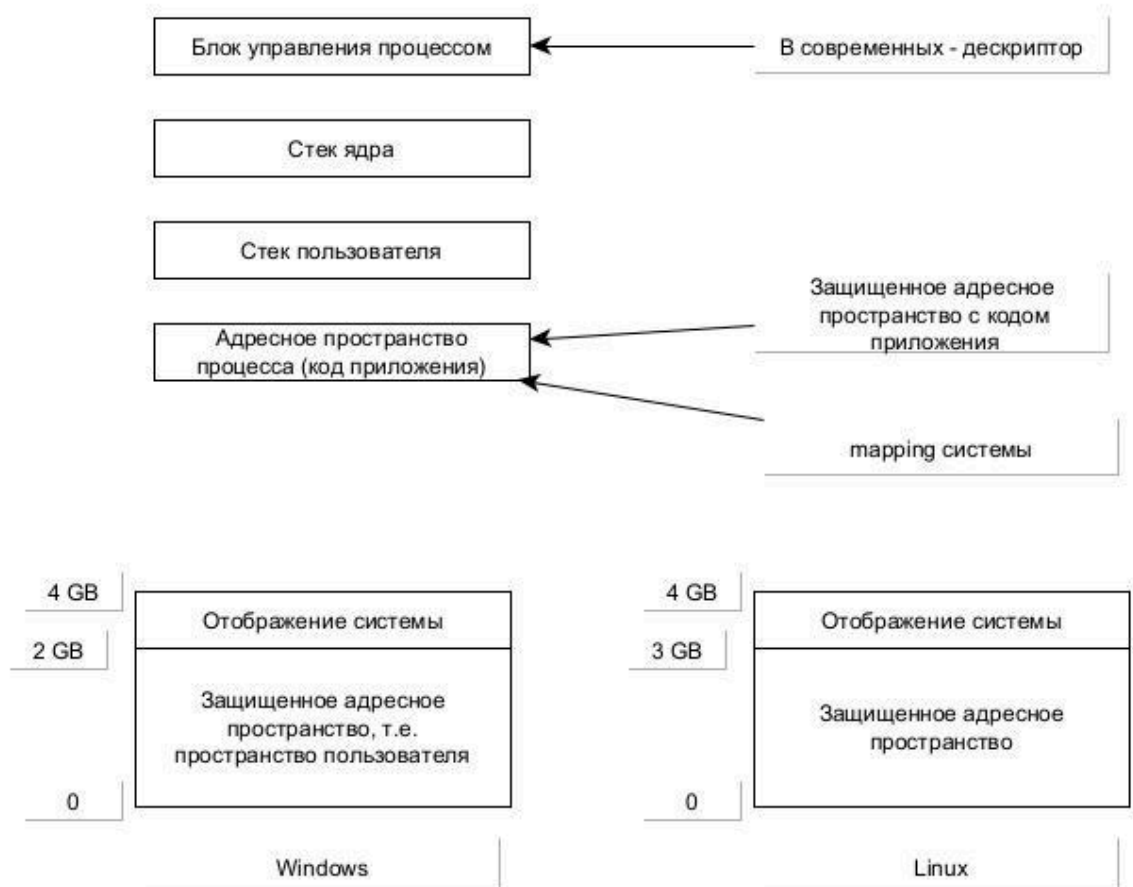
Модели потоков (базовая модель)

40) Однопоточная модель процесса

- а) Система не может выполнять дополнительный анализ, чтобы дифференцировать процесс с потоками, иначе будут потери времени.

=> если в системе есть потоки и в процессе не создается других потоков, то для этой системы считается, что процесс содержит **основной(главный) поток**.

(Если в процессе не созданы специальные потоки для системы, то процесс имеет основной поток)

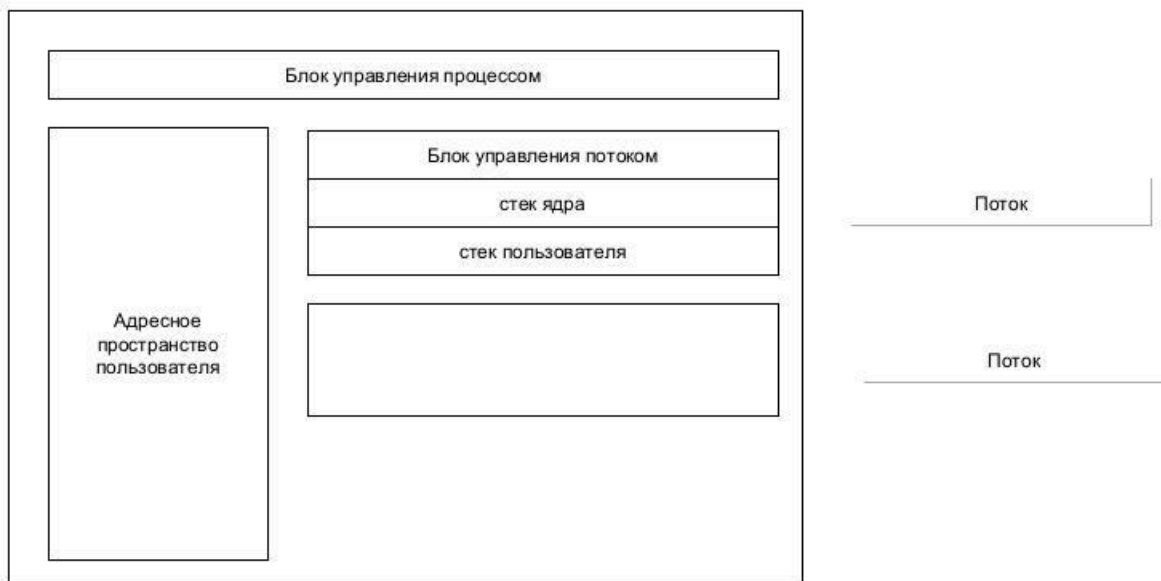


- b)
- c) Адресные пространства виртуальные.

Не может быть одного стека, который использовался бы и в режиме ядра и в режиме пользователя.

Так как пользователь может создать несколько стеков для своих нужд, то любой процесс имеет минимум 2 стека.

41) Многопоточная модель процесса



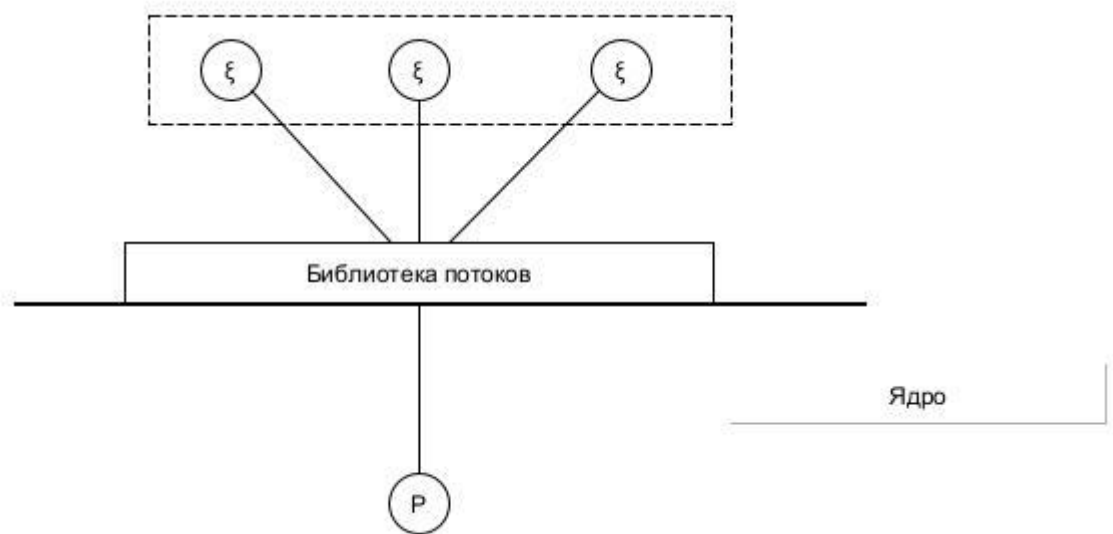
- a) Выполняются потоки, но владельцем ресурсов является процесс
- b) В результате все потоки процесса разделяют его ресурсы и в первую очередь его адресное пространство
- c) Потоки своего адресного пространства не имеют
- d) Два основных типа потоков
 - i) уровня пользователя
 - ii) уровня ядра

Типы (уровни) потоков

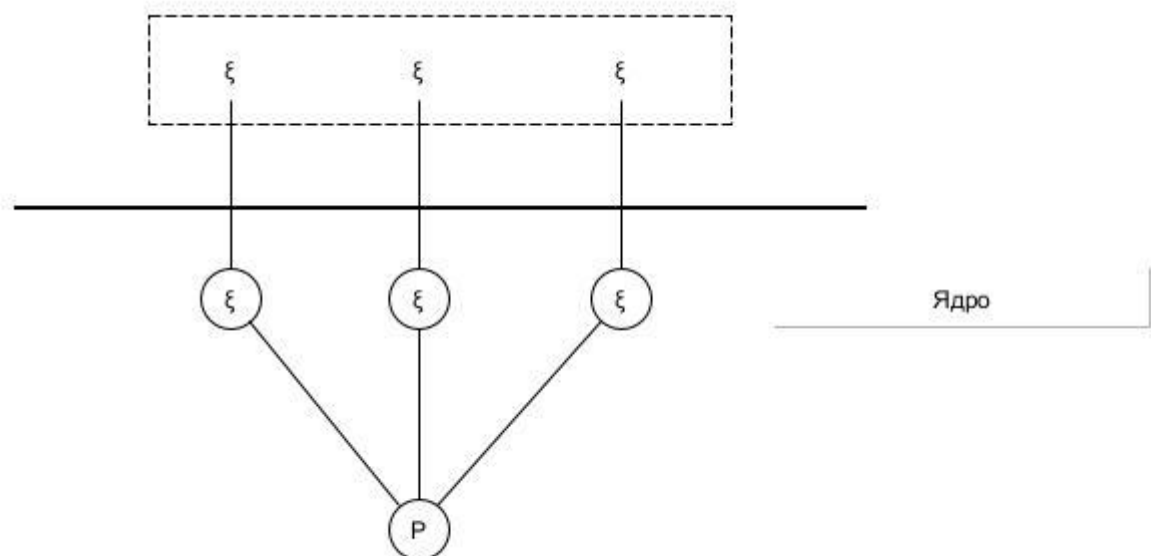
42) Потоки уровня пользователя

- a) **О них ядру ничего неизвестно**
- b) Для управления необходима специальная библиотека потоков, которая осуществляет
 - i) планирование выполнения потоков

- ii) выделение процессорного времени, диспетчеризация
 - iii) обеспечивает обмен сообщениями между потоками
 - iv) переключение контекста(Сохранение контекста?)
(выполнен, блокирован, готов к выполнению)
- с) Проблема, когда запрошен ввод-вывод: напрямую программе обращаться к I/O нельзя (система будет не защищена), нужен системный вызов. Любое обращение к I/O - syscall.
- Обращение к внешнему устройству потоком приводит к блокировке всего процесса. Все потоки ждут завершения I/O, т. к. ядро ничего не знает о наших потоках.



43) Потоки уровня ядра



- а) Создаются системными вызовами

- b) Система знает об этих потоках, у нее есть структура для управления ими
 - c) Процесс должен иметь ссылки на свои потоки
 - d) В Windows, Linux, Mac — потоки уровня ядра, в Solaris — и те, и те
 - e) Потоки разделяют код, данные, файлы и объекты ядра для взаимодействия.
 - f) В модели с потоками уровня пользователя/уровня ядра процесс рассматривается как процесс с основным потоком.
- 44) Мотивация: потоки полезны, когда процесс выполняет несколько задач независимо друг от друга. Особенно выигрышно, когда одна задача может блокироваться, а другие могут продолжать выполняться без блокировки (пример: текстовый редактор)
- 45) Достоинства мультипоточности:
- a) отзывчивость
 - i) один поток может обеспечить быструю реакцию приложения, когда остальные потоки блокированы или заняты медленными вычислениями
 - b) разделение ресурсов
 - i) потоки разделяют общий код, данные и другие ресурсы, что позволяет некоторым задачам выполняться параллельно в едином адресном пространстве
 - c) экономичность
 - i) необходимо переключить только аппаратный контекст, так как потоки выполняются в едином адресном пространстве и разделяют общие ресурсы процесса
 - d) масштабируемость
 - i) однопоточные могут выполняться на 1 процессоре
 - ii) реальная параллельность в многопроцессорных системах
- 46) Проблемы мультипоточности:
- a) определение задач (выделение задачи)
 - i) требует изучения приложения для поиска действий, которые могут выполняться параллельно
 - b) сбалансированность

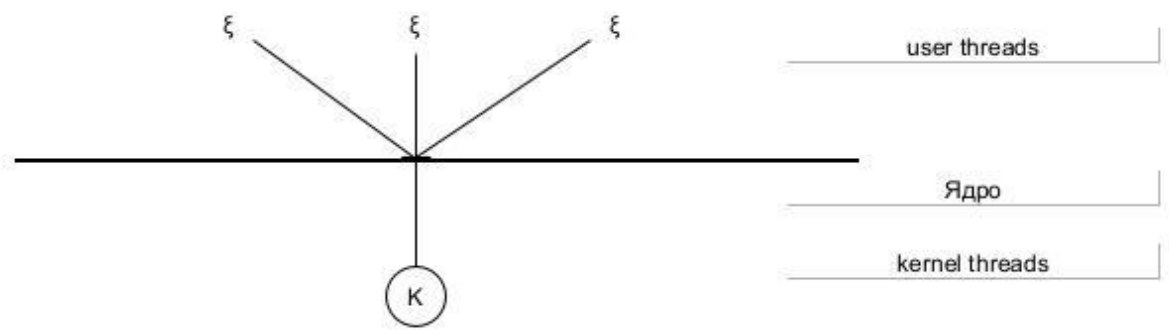
- i) необходимо делить программу на задачи так, чтобы каждый поток выполнял равный объем действий. Не стоит тратить поток на тривиальную задачу.
- с) разделение данных
 - i) параллельные потоки могут обращаться к одним и тем же данным
 - ii) без средств взаимоисключения будет потеря данных или неправильные значения
- d) зависимость данных
 - i) если одна задача зависит от результатов выполнения другой задачи, то выполнение этих задач должно быть синхронизированным
- e) тестирование и отладка
 - i) отладка многопоточных программ намного сложнее, чем однопоточных

Типы параллелизма

- 47) Есть 2 пути:
 - a) распараллеливание по данным
 - i) данные разделяются между потоками
 - ii) возможна реальная параллельность на многоядерном процессоре
 - iii) все потоки выполняют одну и ту же задачу для подмножеств множества данных
 - b) распараллеливание задач
 - i) разделение программы на параллельные задачи (текстовый редактор)

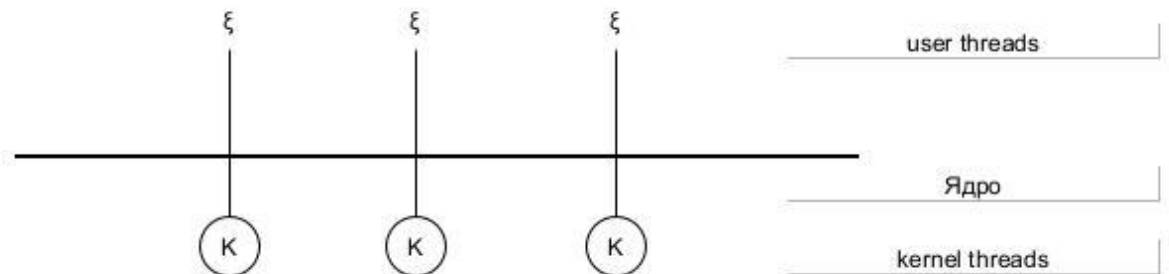
Обычно эти типы сочетаются.

- 48) Модель “многие к одному”



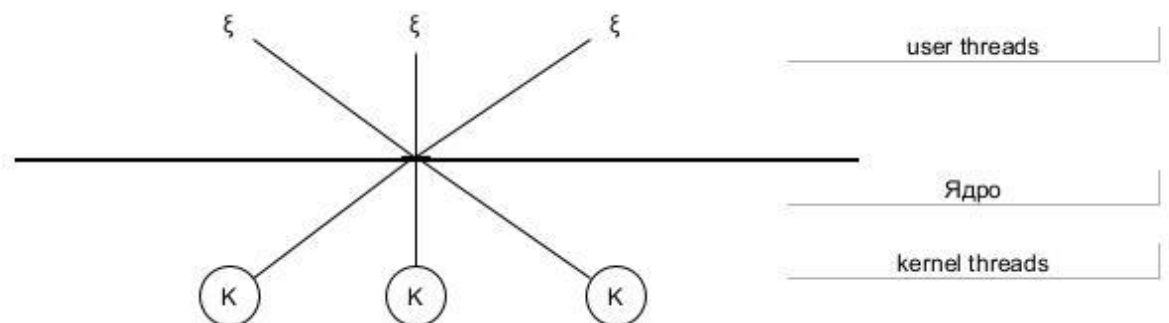
- a) есть потоки пользователя и библиотека для них
- b) для ядра процесс выполняется как процесс с главным потоком
- c) Проблема: блокировка при I/O

49) Модель “один к одному”



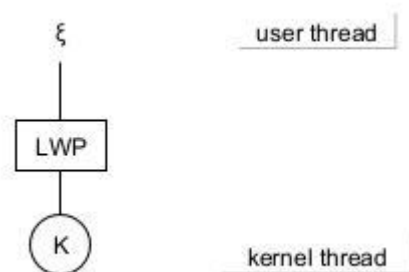
- a) создаются отдельные потоки ядра для обработки каждого потока уровня пользователя
- b) Большие накладные расходы по управлению потоками
- c) Такая модель используется Windows и Linux.
- d) В большинстве OS число потоков, которые могут быть созданы, ограничено. Это связано с ресурсами и возможностями ОС.

50) Модель “многие ко многим”



- a) Создание потоков – затратное действие. Это структура, поля которой должны быть инициализированы.
- b) В системе заранее создаются потоки ядра, которые могут быть выделены для потоков уровня пользователя
- c) То есть также нет ограничений на количество потоков, созданных в приложении, также выд. отд. процессора – мультипроц. (?? не смог до конца расшифровать)
- d) Таким образом в ядре существуют уже инициализированные структуры потоков**

- 51) Двухуровневая модель (многие ко многим + один к одному — гибрид)
- 52) Библиотеки потоков предоставляют API для создания и управления потоками. Могут быть реализованы в пространстве пользователя (не видны ядру) или в пространстве ядра (предоставляются системные вызовы)
 - a) pthreads (POSIX)
 - b) threads (Win32)
 - c) Java threads (потоки java в основном выполняются в JVM)
- 53) LWP – Lightweight Processes
 - a) В Linux потоки как бы создаются через LWP



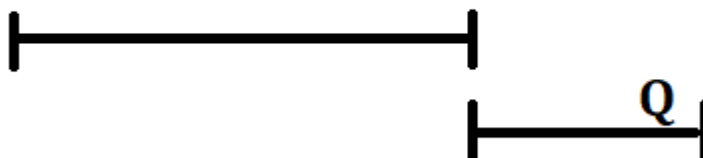
- b) pthread library использует функцию ядра clone() (та же функция, что создает процесс)
- c) В ядре структура потока инициализируется заранее
- d) При вызове pthread_create() используется уже заранее инициализированная структура
- e) Количество доступных потоков ядра может изменяться динамически

Взаимодействие параллельных процессов. Уровни наблюдения

54) Последовательное выполнение

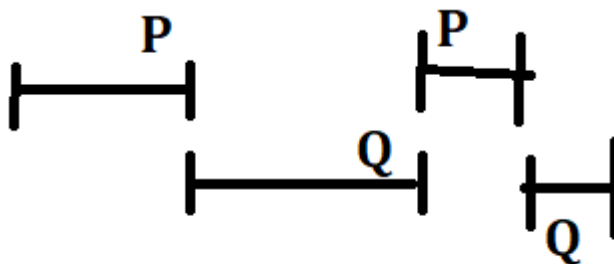
- а) Процессы не могут взаимодействовать

P



55) Квазипараллельное выполнение

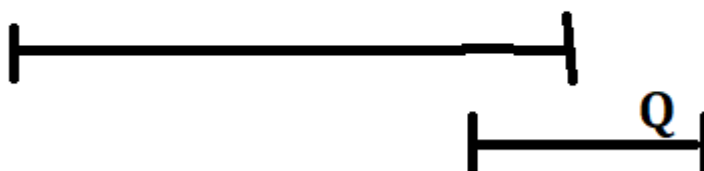
- а) Однопроцессорная система, мультизадачная с разделением времени
б) С точки зрения пользователя параллельное
с) С точки зрения ядра последовательное



56) Многопроцессорная система

- а) Реальная параллельность

P



57) Проблемы квазипараллельного и параллельного выполнения

- а) Пример: два банка должны перечислить на один и тот же счет

P1	P2
$S := S + D1$	$S := S + D2$
	mov
	add
	потерял квант
получил квант	
mov	
add	
загрузил в память	
	получил квант, загрузил в память
$S = S + D1$	$S = S + D2$

Итог: потеряли деньги



- b) Рассмотрим реальную параллельность, два процессора, потоки разделяют мувар
- c) Ядра – процессоры с полным набором регистров

P1	P2
mov eax, myvar	mov eax, myvar
inc eax	inc eax
mov myvar, eax	mov myvar, eax

P1 на процессоре 1	eax	myvar	eax	P2 на процессоре 2
--------------------	-----	-------	-----	--------------------

		0		
mov eax, myvar	0	0		
	0	0	0	mov eax, myvar
	0	0	1	inc eax
	0	1	1	mov myvar, eax
inc eax	1	1	1	
mov myvar, eax	1	1	1	

Итог: потеря данных

d) Пример для одного процессора

P1	eax	myvar	eax	P2
		0		
Выполняется P1				
mov eax, myvar	0	0		
P1 потерял квант, выполняется P2				
сохранен контекст	0	0	0	mov eax, myvar
		0	1	inc eax
		1	1	mov myvar, eax
P2 потерял квант, выполняется P1				
Восстанав ливается аппаратны й контекст	0	1	1	Сохранен контекст

inc eax	1	1		
mov myvar, eax	1	1		

Итог: потеряли 1, проблемы одни и те же

- e) Такие переменные как S, myvar называются разделяемыми
- f) race condition – гонки
- g) Должен обеспечиваться монополярный доступ (средствами взаимного исключения)

58) Основные методы взаимного исключения:

- a) программный
- b) аппаратный
- c) с помощью семафоров
- d) с помощью мониторов

59) **Программный способ**

Не используются никакие средства операционной системы.

- a) Взаимное исключение обеспечивается в ходе программы (флаги)
- b) prog_example_1

f1, f2: logical		
p1: while (1) while(fp2); fp1 = 1; CR1; fp1 = 0; PR1; end p1;	p2: while (1) while(fp1); fp2 = 1; CR2; fp2 = 0; PR2; end p2;	begin fp1 = 0; fp2 = 0; parbegin p1; p2; parend end

Флаг не помог

c) prog_example_2

f1, f2: logical		
p1: while (1) fp1 = 1; while(fp2); CR1; fp1 = 0; PR1; end p1;	p2: while (1) fp2 = 1; while(fp1); CR2; fp2 = 0; PR2; end p2;	begin fp1 = 0; fp2 = 0; parbegin p1; p2; parend end

- p2 перехватил квант установил флаг, потерял квант
- p1 установил флаг, потерял квант
- p2 циклится в цикле проверки fp1 весь квант
- p1 циклится в цикле проверки fp2 весь квант
- Ни один из процессов не может выйти из проверки (deadlock), при этом тратят квант на проверку
- Активное ожидание на процессоре

Алгоритм Деккера (Петерсона)

- d) Только для 2 параллельных процессов
- e) Ввел дополнительную переменную, которую назвал “чья очередь” (у нас просто “очередь”)

f1, f2: logical; que: int;		
p1: while (1); fp1 = 1; while(fp2) { if (que == 2) { fp1 = 0; while (que==2); fp1 = 1 } } CR1; fp1 = 0; que = 2; PR1; end p1;	p2: while (1); fp2 = 1; while (fp1) { if (que == 1) { fp2 = 0; while (que==1); fp2 = 1; } } CR2; fp2 = 0; que = 1; PR2; end p2;	begin fp1 = 0; fp2 = 0; que = 1; parbegin p1; p2; parend end

- f) В тупик не входят, так как сбрасывают свой флаг по анализу флага другого процесса
- g) Таким образом исключается бесконечное откладывание
- Деккер предложил решение для 2-х процессов, но на N это решение не масштабируется.

Алгоритм Лампорта (Булочная Лампорта)

- h) Каждый входящий в булочную клиент получает листок с номером. Каждый следующий клиент получает листок с большим номером. Клиенты обслуживаются строго по номерам. Когда два клиента заходят одновременно, они получают одинаковые номера, но первым будет обслужен тот, у кого меньше номер паспорта
- i) Решение проблемы для входа в критическую секцию для n процессов
- j) Под обслуживанием понимается вход в критическую секцию в соответствии с ранее полученным номером

```
1. var choosing: shared array[0..n-1] of boolean;
2. number: shared array[0..n-1] of integer;
3. repeat
4.   choosing[i] := true;
5.   number[i] := max(number[0], ..., number[n-1]) + 1;
6.   choosing[i] := false;
7.   for j := 0 to n-1 do begin
8.     while choosing[j] do (*nothing*);
9.     while number[j] <> 0 and
10.      (number[j], j) < (number[i], i) do //
        лексикографическое отношение
11.      (*nothing*)
12.   end;
13.   (*critical section*)
14.   number[i] := 0;
15.   (*remainder section*)
16. until false;
```

- 1-2: объявление переменных
 - choosing[i] истинно, если $p[i]$ выбирает номер, который он будет использовать для входа в критическую секцию, т.е. он должен получить number[i].
 - если процесс не пытается войти в критический участок, то $number[i] = 0$
- 4-6: показано, что процесс получает следующий по порядку номер

- для получения установлена охрана $\text{choosing}[i] = \text{true}$ и $\text{choosing}[i] = \text{false}$
 - 7-12: ожидание своей очереди для входа в критическую секцию
 - 8: процесс ждет, если выполняется выбор
 - 9: если $\text{number}[j] \neq 0$, он получил номер и ожидает своей очереди
 - 10: пример $(a, b) < (c, d)$, лексикографический порядок.
 - Истинно, если $a < c$ или если $(a = c \text{ и } b < d)$
- k) Обслуживание по очереди не требует других средств взаимного исключения
- l) Процессы входят в критическую секцию по номеру в очереди
- m) Нет никаких системных вызовов, то есть при выполнении такой программы процесс не перейдет в режим ядра
- n) Но присутствует активное ожидание на процессоре из-за проверки флага или возможности войти в критическую секцию. На это расходуется квант.

60) Аппаратный способ

- a) Взаимоисключения реализуются с помощью машинной команды `test_and_set(a, b)`
- i) появилась в IBM 370
 - ii) является неделимой
 - iii) реализует проверку и установку ячейки памяти, которая называется байтом блокировки
 - (1) 0 – ресурс доступен
 - (2) 1 – ресурс занят
 - iv) читает значение логической переменной `b` и копирует его в `a`, а затем для `b` устанавливается значение ‘истина’
 - v) не значит, что переходит в режим ядра

b)

```

program use_ts;
  flag, c1, c2: logical;
p1: while (1)
  {

```

```

        c1 = 1;
        while (c1 == 1)
            test_and_set(c1, flag);
        CR1;
        flag = 0;
        PR1;
    }

p2: while (1)
    {
        c2 = 1;
        while (c2 == 1)
            test_and_set(c2, flag);
        CR2;
        flag = 0;
        PR2;
    }

main()
{
    flag = 0;
    parbegin
        p1, p2;
    parend;
}

```

- c) Логическая переменная flag имеет значение
- i) “истина”, когда каждый из процессов находится в своем критическом участке,
 - ii) “ложь в противном случае
- d) Пусть p1 хочет войти в критический участок, а p2 уже находится в этом критическом участке:
- i) p1 устанавливает c1 в 1 и входит в цикл проверки переменной flag.
 - ii) Поскольку p2 находится в своем критическом участке, flag = 1
 - iii) Команда test_and_set находит этот flag и устанавливает c1 в 1

- iv) Таким образом p1 находится в своем цикле активного ожидания, пока p2 не выйдет из своего критического участка
- e) Считается, что данный способ исключает бесконечное откладывание (точнее, его вероятность мала)
- f) Когда процесс выходит из CR и сбрасывает flag, то скорее всего другой процесс сможет перехватить и установить flag
- g) У Вахаллии test_and_set называется простой блокировкой
- h) Использование команды test_and_set в цикле проверки значения переменной называется циклической блокировкой (spin lock, simple lock, simple mutex)
- i)

```
void spin_lock(spin_lock_t *c) //c – condition (условие)
{
    while (test_and_set(*c) != 0)
        /*ресурс занят*/
}

void spin_unlock(spin_lock_t *c)
{
    *c = 0;
}
```

- j) Неделимость достигается за счет блокировки локальной шины памяти
- k) В результате длительная задержка может привести к длительному занятию шины одним процессом (негативно влияет на отзывчивость системы)
- l) Решение проблемы:

```
void spin_lock(spin_lock_t *c)
{
    while (test_and_set(*c) != 0)
        while(*c != 0);
}
```

Если переменная занята, то выполняется внутренний цикл без блокировки шины памяти

Семафоры

61) Взаимоисключение при помощи семафоров

Предыдущие два способа связаны с активным ожиданием: тратится квант на проверку флага или очереди. Это непроизводительная трата. Для того, чтобы использоваться процессорное время более эффективно в использование вошел способ Дейкстры (датчанин).

Классические семафоры Дейкстры.

- a) Смысл: уйти от активного ожидания
- b) Предложил: семафоры
- c) Первоначально семафор — бинарный семафор, который может принимать два значения: 0 и 1. (Semaphore(S))
- d) На семафоре определены две операции:
 - i) $P(S)$ — захватить
 - ii) $V(S)$ — освободить
- e) Если значение семафора больше 0, то его можно декрементировать.
 $P(S)$: если $S > 0$, то $S = S - 1$
- f) Если $S == 0$, то другой процесс захватить этот семафор уже не может
- g) $V(S)$: если $S == 0$, то $S = S + 1$
Освободить, и дать другому захватить
- h) Если процесс не может выполнить $P(S)$, то он блокируется в ожидании освобождения семафора до тех пор, пока другой процесс не выполнит $V(S)$
- i) К семафору организуется очередь процессов. Будет разблокирован первый процесс из очереди.
- j) Разблокировать/заблокировать процесс может только ядро.
 $\Rightarrow V(S), P(S)$ — системные вызовы
- k) $P(S), V(S)$ — неделимые операции
- l) Если семафор принимает целые положительные и 0 значения, то такой семафор называется **считающим**.

m) S: binary semaphore. Начальное значение: S=1

P1:	P2:
P(S)	P(S)
CR1	CR2
V(S)	V(S)
PR1	PR2
.....	

- n) Пока 1 процесс находится в критическом участке, другой процесс не может выполнить P(S) и V(S)
- o) P(S), V(S) — системные вызовы, в результате этого исключается активное ожидание. НО выполняется переход в режим ядра => переключение аппаратного контекста.
- p) Если процесс не может выполнить P(S), V(S), то он теряет квант и блокируется – происходит полное переключение контекста.

Задача производства потребления

- q) Решение предложено Дейкстрой
- r) Имеется два типа процессов и буфер.
- s) Процессы производители могут только производить единицу данных и класть ее в буфер
- t) Другие процессы, которые называются процессами потребителями, могут только брать данные из буфера.



- u) SE — количество пустых ячеек буфера
SF — количество заполненных ячеек буфера
SB — бинарный семафор
SE и SF — считающие семафоры.
se, sb, sf: semaphore

v) producer:

```
while(1)
{
    P(SE)
    P(SB)
    N = N + 1; // добавить в буфер
    V(SB)
    V(SF)
}
```

w) consumer:

```
while(1)
{
    P(SF)
    P(SB)
    N = N - 1; // считать из буфера
    V(SB)
    V(SE)
}
```

- х) начальные значения: $SE = N$, $SF = 0$, $SB = 1$
- у) Producer должен положить единицу данных в буфер. Он может это сделать только если там есть пустые ячейки. Делает $SE = SE - 1$. Если $SE < 0$, то нет мест и производитель блокируется.
- z) Если получается положить 1, то продюсер освобождает бинарный семафор(который защищает разделяемую переменную), и увеличивает количество заполненных ячеек.
- aa) Консьюмер сможет что-то прочитать, только если есть хотя бы 1 заполненная ячейка, для этого вызывается $P(SF)$. Если заполненных ячеек нет, то $P(SF)=0$ и процесс блокируется, пока продюсер не заполнит буфер.
- bb) Если консьюмеру удалось что-то взять из буфера, то он освобождает бинарный семафор и увеличивает число пустых

ячеек, тем самым разблокируя продюсера.

сс) Ситуации блокировок являются вероятностными, поскольку все процессы в системе являются асинхронными, то есть выполняются с собственной скоростью => вполне вероятны две ситуации:

- i) если запаздывает потребитель, то производитель заполнит все ячейки и будет ждать освобождения
- ii) если запаздывает производитель, то потребитель может все считать и будет блокирован на SF

dd) В современных системах реализованы множественные семафоры, которые семантически представляются **массивами считающими семафоров**.

Связано это с тем, что у семафоров есть одна важная особенность.

62) Семафор VS Mutex

- a) Mutex относится к средствам lock и unlock
- b) У Mutex есть хозяин, освободить Mutex может только тот процесс, который его захватил.
- c) Семафоры всегда рассматриваются отдельно
- d) Это прежде всего связано с тем, что у семафора нет хозяина — семафор может освободить любой процесс (которые не заблокирован на семафоре)

p1:	p2:
P(S)	V(S)
....	

=> семафоры могут рассматриваться как средства передачи информации.

- e) Так как любой может освободить или захватить семафор, и так как процесс изменяет целый ряд семафоров — это чревато тупиковыми ситуациями.

p1:	p2:
P(S1)	P(S2)

P(S2)	P(S1)
....	
V(S2)	V(S1)
V(S1)	V(S2)
....	

Первый захватил S1, второй захватил S2

Первый пытается захватить S2, и блокируется на S2

Второй пытается захватить S1, и блокируется на S1.

Оба ждут освобождения семафора, и не могут это сделать.

Они попали в тупик.

=> в современных ОС определены наборы семафоров.

63) Свойство набора семафоров

Одной неделимой операцией можно изменить значения всех или части набора семафоров.

Код “производство-потребление” с сегментом разделяемой памяти

Примечание: нужно использовать не структуру буфера, а арифметику указателей, но для производства потребления здесь важнее работа с массивами указателей start_produce, stop_produce, start_consume, stop_consume и semop

```
typedef struct
{
    char data[26];
    int rpos;
    int wpos;
    char element;
} buffer_t;
struct sembuf start_produce[2] = {{1, -1, 0}, {0, -1, 0}};
struct sembuf stop_produce[2] = {{0, 1, 0}, {2, 1, 0}};
struct sembuf start_consume[2] = {{2, -1, 0}, {0, -1, 0}};
struct sembuf stop_consume[2] = {{0, 1, 0}, {1, 1, 0}};
void producer(buffer_t *buffer, int semid, int producer_id)
{
    srand(time(NULL) + producer_id);
```

```

int sleep_time;
char elem;
while (buffer->wpos < 24)
{
    sleep_time = rand() % 2 + 1;
    sleep(sleep_time);

    if (semop(semid, start_produce, 2) == -1)
    {
        perror("Can't semop'\n");
        exit(1);
    }
    elem = buffer->element++;
    buffer->data[buffer->wpos++] = elem;
    printf("\e[1;32mПроизводитель %d записал: %4c\e[0m\n", producer_id,
elem);
    if (semop(semid, stop_produce, 2) == -1)
    {
        perror("Can't semop'\n");
        exit(1);
    }
}
}

void consumer(buffer_t *buffer, int semid, int consumer_id)
{
    srand(time(NULL) + consumer_id);
    int sleep_time;
    char elem;
    while (buffer->rpos < 24)
    {
        sleep_time = rand() % 2 + 1;
        sleep(sleep_time);
        if (semop(semid, start_consume, 2) == -1)
        {
            perror("Can't semop'\n");
            exit(1);
        }
        elem = buffer->data[buffer->rpos++];
        printf("\e[1;33mПотребитель %d прочитал: %4c\e[0m\n", consumer_id,
elem);

        if (semop(semid, stop_consume, 2) == -1)

```

```

        {
            perror("Can't semop\n");
            exit(1);
        }
    }
}

int main()
{
    pid_t childpid[6];
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    int shmid = shmget(IPC_PRIVATE, sizeof(buffer_t), IPC_CREAT | perms);
    if (shmid == -1)
    {
        perror("Can't shmget\n");
        exit(1);
    }
    buffer_t *buffer = (buffer_t*) (shmat(shmid, 0, 0));
    if (buffer == (buffer_t*) -1)
    {
        perror("Can't shmat\n");
        exit(1);
    }
    for (int i = 0; i < 26; i++)
    {
        buffer->data[i] = 0;
    }
    buffer->rpos = 0;
    buffer->wpos = 0;
    buffer->element = 'a';
    int semid = semget(IPC_PRIVATE, 3, IPC_CREAT | perms);
    if (semid == -1)
    {
        perror("Can't semget\n");
        exit(1);
    }
    if (semctl(semid, 0, SETVAL, 1) == -1)
    {
        perror("Can't semctl SB\n");
        exit(1);
    }
    if (semctl(semid, 1, SETVAL, 26) == -1)
    {

```

```

        perror("Can't semctl SE\n");
        exit(1);
    }
    if (semctl(semid, 2, SETVAL, 0) == -1)
    {
        perror("Can't semctl SF\n");
        exit(1);
    }
    for (int i = 0; i < 3; i++)
    {
        if ((childpid[i] = fork()) == -1)
        {
            perror("Can't fork producer\n");
            exit(1);
        }
        else if (childpid[i] == 0)
        {
            producer(buffer, semid, i + 1);
            exit(0);
        }
    }
    for (int i = 3; i < 6; i++)
    {
        if ((childpid[i] = fork()) == -1)
        {
            perror("Can't fork consumer\n");
            exit(1);
        }
        else if (childpid[i] == 0)
        {
            consumer(buffer, semid, i - 2);
            exit(0);
        }
    }
    for (int i = 0; i < 6; i++)
    {
        int status;
        pid_t child_pid = waitpid(childpid[i], &status, WUNTRACED |
WCONTINUED);
        if (child_pid == -1)
        {
            perror("waitpid");

```

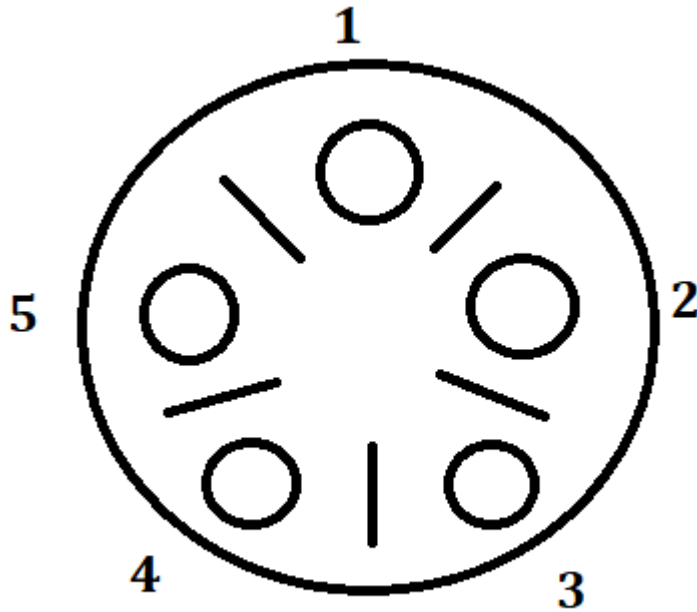
```

        exit(EXIT_FAILURE);
    }
    printf("Child has finished: PID = %d\n", child_pid);
    if (WIFEXITED(status))
    {
        printf("exited, status=%d\n", WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status))
    {
        printf("killed by signal %d\n", WTERMSIG(status));
    }
    else if (WIFSTOPPED(status))
    {
        printf("stopped by signal %d\n", WSTOPSIG(status));
    }
}
if (shmdt((void *)buffer) == -1)
{
    perror("Can't shmdt\n");
    exit(1);
}
if (semctl(semid, 0, IPC_RMID, NULL) == -1)
{
    perror("Can't delete semaphore\n");
    exit(1);
}
if (shmctl(shmid, IPC_RMID, NULL))
{
    perror("Can't mark a segment as deleted.\n");
    exit(1);
}
}

```

Обедающие философы

За круглым столом сидят 5 философов. Перед каждым философом стоит тарелка. Между тарелками лежит по 1 палочке(для еды нужно 2).



Существует три способа взаимодействия философов:

- 1) Каждый философ пытаются взять сразу оба прибора (левый и правый) и начать есть. После, возвращает оба прибора
- 2) Каждый из философов берет правый прибор и пытается взять левый, удерживая правый
- 3) Если не может взять левый, возвращает правый

Проблемы:

- 1) Приводит к бесконечному откладыванию
 - a) 1 философ умрет голодной смертью
 - b) 2 и 5 берут оба прибора (1, 3, 4 ждут)
 - c) 5 положил, 4 может взять (1, 3, 5 ждут)
 - d) 2 положил, 3 может взять (1, 2, 4 ждут)
 - e) 1 постоянно ждет
- 2) Все взяли правый прибор, пытаются взять левый — классический пример блокировки deadlock
- 3) Все взяли правый, не могут взять левый => возвращают правый и повторяют эти действия.
 - a) В системе процессы захватывают и освобождают одни и те же ресурсы => полезная работа не выполняется

```

var
  forks: array[1..5] of semaphore;
  i: integer;
begin
  repeat
    forks[i] := 1;
    i := i - 1
  until i = 0;
parbegin
  1: begin
    left := 1; right := 2;
    ...
  end;
  5: begin
    left := 5; right := 1;
    repeat
      //размышляет
      P(forks[left], forks[right]); // захватывает
      // ест
      V(forks[left], forks[right]); // освобождение
    forever
  end; // 5
parend
end.

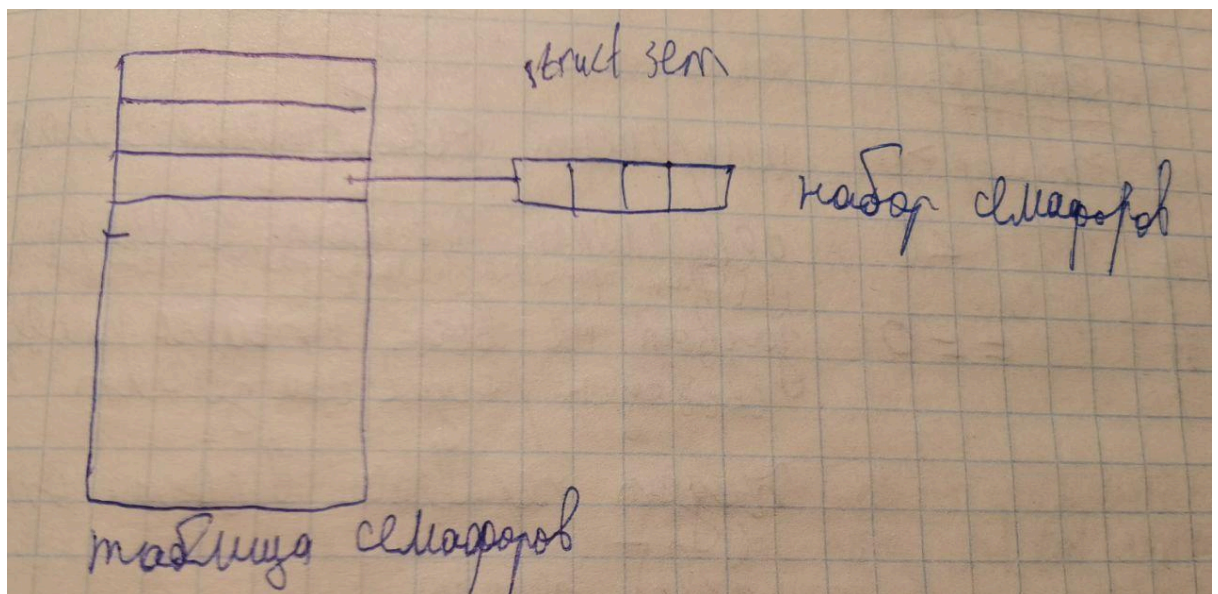
```

Семафоры UNIX

- Семафоры UNIX вносят дополнения в семафоры Дейкстры.
- В современных системах поддерживаются наборы семафоров
- Наборы семафоров представлены массивами семафоров
- В UNIX LINUX семафоры поддерживаются системой таблицей(находится в адресном пространстве системы) семафоров. Каждый элемент таблицы описывает один набор семафоров (то есть это дескриптор)
- Дескриптор набора семафоров хранит:
 - имя (целое число) – присваивается процессом, который создал набор семафоров. Другие процессы по этому имени могут получить доступ к нему.
 - uid
 - идентификатор создателя наборов семафоров и идентификатор его группы.

Процесс, uid которого совпадает с uid созд. может удалить набор и изменить его существующие права(?)

- права доступа
- количество семафоров в наборе
- время изменения 1 или нескольких семафоров последним процессом
- время последнего изменения управляющих параметров набора каким-либо процессом
- указатель на массив семафоров
=> в наборе семафоров семафоры обозначаются индексами начиная с нулевого



- На семафорах определены системные вызовы:
 - `semget()` — создание набора семафоров
 - `semctl()` — изменение управляющих параметров набора семафоров
 - `semop()` — операции на наборе семафоров или на семафоре
- Для определения операций на семафорах определена структура

```
struct sembuf
{
    ushort sem_num; //index
    short sem_op; // операция
    short sem_fl; // флаги
}
```


- В отличие от семафоров Дейкстры, на семафорах UNIX определено 3 операции
- sem_op:
 - > 0 — инкремент, освобождение семафора
 - < 0 — декремент, захват семафора
 - блокирование процесса, если декремент невозможен
 - $== 0$ — захват не выполняется, процесс проверяет, свободен ресурс или занят
 - будет переведен в состояние ожидания освобождения ресурса. После его освобождения декремент не выполн.
- Флаги sem_flg:
 - IPC_NOWAIT — информирует систему о нежелании процесса переходить в состояние ожидания (освобождения семафора)
 - Наличие данного флага объясняется желанием избежать блокирования всех процессов, находящихся в очереди процессов, в случае, если захвативший ресурс процесс завершится аварийно или получит сигнал kill
 - Так как kill невозможно перехватить, процесс не может освободить семафор
 - Блокированный процесс не получает процессорного времени
 - SEM_UNDO — данный флаг указывает ядру, что необходимо отслеживать изменение значения семафора в результате вызова semop.
 - Указывает ядру, что при завершении процесса, вызвавшего semop, необходимо ликвидировать сделанные изменения, чтобы другие процессы не были заблокированы на семафорах навечно
- semget возвращает файловый дескриптор созданного набора семафоров
- Семафоры могут служить как источник информации — процесс освобождает семафор как событие. Но это одно значение — свободен или занят. Но есть и другие средства передачи данных.

Мониторы

- Проблемы, связанные с семафорами и другими средствами ОС, состоят в том, что они не структурны
- Все системные вызовы, связанные с взаимным исключением, можно определить как lock и unlock
- Блокирующие примитивы

lock	wait(c)	wait(c)
unlock	post(c)	signal(c)

- Все они приводят к блокировке процесса (он не находится в состоянии активного ожидания)
- Заблокировать и освободить ресурс может только ядро системы
- Если нарушить соотношение между примитивами, то можно утратить блокировку
- Были предложены мониторы

→ Монитор является средством структурирования средств захвата и освобождения

→ Мониторы представляют структуру, состоящую из полей данных и функций, которые могут изменять эти данные

→ Доступ к данным монитора доступен только с помощью функций

→ Сам монитор становится ресурсом

→ Процесс, находящийся в мониторе — процесс, вызвавший функцию монитора

→ Другие процессы выстраиваются в очередь к монитору

→ Классическая реализация предполагает wait(), signal() и тип conditional

Простой монитор

- Обеспечивает выделение ресурса произвольному числу процессов

```
monitor: resource
var
    busy: logical;
    x: conditional;    // тип условие
```

```

procedure acquire          // приобретать
begin
    if busy then wait(x);
    busy := true;
end;

procedure release          // осуществлять
begin
    busy := false;
    signal(x);
end;
begin
    busy := false;
end.

```

- Если ресурс занят, процесс будет блокирован примитивом wait на условии.
- Система разблокирует его, когда процесс вызовет signal
- Освободив ресурс, сбрасывается логическая переменная и разблокирует (signal(x))
- К переменной типа условие выстраивается очередь

Монитор “Кольцевой буфер”

- Реализует задачу “производство-потребление”
- Процессы-производители записывают данные в кольцевой буфер
- Процессы-потребители выбирают данные из кольцевого буфера

```

monitor: resource
var
    bcircle: array[0..n-1] of type;
    pos: 0..n; // текущая позиция
    j: 0..n-1; // заполняемая позиция
    k: 0..n-1; // освобождаемая позиция
    buffer_full, buffer_empty: conditional;

procedure producer(data: type)
begin
    if pos = n then
        wait(buffer_empty);
    bcircle[j] := data;
    pos := pos + 1;

```

```

        j := (j + 1) mod n;
        signal(buffer_full);
    end;

procedure consumer(data: type)
begin
    if (pos = 0) then
        wait(buffer_full);
    data := bcircle[k];
    pos := pos - 1;
    k := (k + 1) mod n;
    signal(buffer_empty);
end;

begin
    pos := 0;
    j := 0;
    k := 0;
end.

```

- Эквивалентно решению Дейкстры
- Producer заполняет элементы буфера и сигнализирует, что он это сделал (signal(buffer_full))
- Consumer освобождает ячейку буфера и сигнализирует signal(buffer_empty))
- Producer заполняет буфер, если он не полон (иначе он будет блокирован на условии buffer_empty)

Читатели писатели (монитор Хоара)

Особенностью задачи читатели писатели является наличие двух типов процессов:

- процессов читателей, которые могут только читать данные
- процессов писателей, которые могут только изменять данные

В мониторе читатели/писатели 4 функции.

turn – очередь.

```
monitor: resource
var nr: integer // кол-во читателей
var wrt: logical // активный писатель
var c_read, c_write: conditional // can ....

procedure start_read:
  begin
    if wrt or turn(c_write)
      then wait(c_read)
    nr = nr + 1
    signal(c_read)
  end
```

Читатели читают параллельно.

Писатель может работать только в режиме взаимного исключения.

(Писатели должны работать в режиме монопольного доступа)

```
procedure stop_read:
  begin
    nr = nr - 1
    if nr == 0 then
      signal(c_write)
  end

procedure start_write:
  begin:
    if nr > 0 or wrt then
      wait(c_write)
    wrt = true
```

```

        end

    procedure stop_write:
        begin
            wrt = false
            if turn(c_read) then
                signal(c_read)
            else
                signal(c_write)
            end
        end

begin
    nr := 0;
    wrt := false;
end.

```

Для того, чтобы начать читать, читатель вызывает startread.

Он:

1. проверяет, есть ли активный писатель
 2. проверяет очередь ждущих писателей(c_write)
- Если есть или активный писатель или ждущий писатель, то читатель переводится в состояние ожидания на переменную типа условия c_read (т. к. если кто-то будет изменять данные, то их нельзя читать)
 - Иначе увеличивается число активных читателей — nr.
 - Посылается сигнал, что можно читать.
 - То есть читатель посылает сигнал другим читателям: возникает цепь: каждый читатель активизирует следующего.
- ❖ Когда читатель завершает чтение, то вызывает stopread.
 - ❖ В этой функции уменьшается число активных читателей nr.
 - ❖ Если nr = 0, т. е. все прочитали, то посылается сигнал “можно писать”.
- Когда писатель хочет начать писать, то вызывает startwrite и проверяет, есть ли активный писатель или активные читатели (nr > 0).

- Если условие выполняется, то переходит в состояние ожидания на переменную типа условия `s_write`.
- Иначе устанавливает переменную `wrt` в `true`, информируя о том, что он пишет.

- ❖ Когда писатель завершает писать, то вызывается `stopwrite`.
- ❖ В этой функции сбрасывается флаг `wrt`, и проверяется очередь ждущих читателей.
- ❖ Если есть ждущие читатели, то посылается сигнал “можно читать”. Иначе: “можно писать”.

→ Особенность: цепная реакция по активизации читателей.

- В данном виде монитора исключено бесконечное откладывание читателей и писателей за счет проверки соответствующего условия.
- Писатель имеет право писать, если нет активных читателей. Читателям дается возможность прочитать.
- Читатель в функции `start_read` также проверяет, есть ли ждущие писатели. Если есть — не сможет начать читать.
- Таким образом исключается бесконечное ожидание писателей.
- Аналогично `stopwrite` — если есть ждущие читатели, то им дается разрешение читать. Т. о. откладывается бесконечное ожидание читателей.

- ❖ Такая реализация допускает эффект потерянного обновления
- ❖ Писатели получают возможно измененные данные, а читатели читают только последнее изменение
- ❖ С точки зрения чтения только актуальных данных — это не плохо

Код Linux

```
#include <sys/sem.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <stdio.h>
#include <sys/ipc.h>
#include <stdlib.h>
```

```

#include<unistd.h>
#include <time.h>
#define ACTIVE_READERS  0
#define ACTIVE_WRITER   1
#define WRITERS_QUEUE   2
#define READERS_QUEUE   3
#define BIN_WRITER      4
struct sembuf start_read[] = {{READERS_QUEUE, 1, 0},
                               {BIN_WRITER, 0, 0},
                               {WRITERS_QUEUE, 0, 0},
                               {ACTIVE_READERS, 1, 0},
                               {READERS_QUEUE, -1, 0}};
struct sembuf stop_read[] = {{ACTIVE_READERS, -1, 0}};
struct sembuf start_write[] = {{WRITERS_QUEUE, 1, 0},
                                {ACTIVE_READERS, 0, 0},
                                {BIN_WRITER, 0, 0},
                                {ACTIVE_WRITER, -1, 0},
                                {BIN_WRITER, 1, 0},
                                {WRITERS_QUEUE, -1, 0}};
struct sembuf stop_write[] = {{BIN_WRITER, -1, 0},
                               {ACTIVE_WRITER, 1, 0}};

void sig_handler()
{
    if (getpid() != getpgrp())
        exit(0);
}

void reader(int *shm, int semid, int reader_id)
{
    srand(time(NULL));
    int sleep_time;
    while (1)
    {
        sleep_time = rand() % 2 + 1;
        sleep(sleep_time);
        if (semop(semid, start_read, 5) == -1)
        {
            perror("Can't start_read.\n");
            exit(-1);
        }
        int value = *shm;
        printf("Reader %d read: %5d\n", reader_id, value);
    }
}

```



```

        if (semop(semid, stop_read, 1) == -1)
        {
            perror("Can't stop_read.\n\n");
            exit(1);
        }
    }
}

void writer(int *shm, int semid, int writer_id)
{
    srand(time(NULL));
    int sleep_time;
    while (1)
    {
        sleep_time = rand() % 2 + 1;
        sleep(sleep_time);
        if (semop(semid, start_write, 6) == -1)
        {
            perror("Can't start write\n");
            exit(1);
        }
        int value = ++(*shm);
        printf("Writer %d wrote: %5d\n", writer_id, value);
        if (semop(semid, stop_write, 2) == -1)
        {
            perror("Can't stop_write.\n");
            exit(1);
        }
    }
}

int main()
{
    pid_t childpid[7];
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    void *handler = NULL;
    if ((handler = signal(SIGINT, sig_handler)) == SIG_ERR)
    {
        perror("Can't signal.\n");
        exit(1);
    }
    int fd = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | perms);
    if (fd == -1)
    {

```

```
        perror("Can't shmget\n");
        exit(1);
    }
    int *shm = (int*) (shmat(fd, 0, 0));
    if (shm == (int*) -1)
    {
        perror("Can't shmat\n");
        exit(1);
    }
    *shm = 0;
    int semid = semget(IPC_PRIVATE, 5, IPC_CREAT | perms);
    if (semid == -1)
    {
        perror("Can't semget\n");
        exit(1);
    }
    if (semctl(semid, ACTIVE_READERS, SETVAL, 0) == -1)
    {
        perror("Can't semctl active_readers\n");
        exit(1);
    }
    if (semctl(semid, ACTIVE_WRITER, SETVAL, 1) == -1)
    {
        perror("Can't semctl active_writer\n");
        exit(1);
    }
    if (semctl(semid, WRITERS_QUEUE, SETVAL, 0) == -1)
    {
        perror("Can't semctl write_queue\n");
        exit(1);
    }
    if (semctl(semid, READERS_QUEUE, SETVAL, 0) == -1)
    {
        perror("Can't semctl read_queue\n");
        exit(1);
    }
    if (semctl(semid, BIN_WRITER, SETVAL, 0) == -1)
    {
        perror("Can't semctl bin_writer\n");
        exit(1);
    }
    for (int i = 0; i < 3; i++)
```

```

{
    if ((childpid[i] = fork()) == -1)
    {
        perror("Can't fork writer\n");
        exit(1);
    }
    else if (childpid[i] == 0)
    {
        writer(shm, semid, i + 1);
        exit(0);
    }
}
for (int i = 3; i < 7; i++)
{
    if ((childpid[i] = fork()) == -1)
    {
        perror("Can't fork consumer\n");
        exit(1);
    }
    else if (childpid[i] == 0)
    {
        reader(shm, semid, i - 2);
        exit(0);
    }
}

```

Код Windows

```

HANDLE can_read;
HANDLE can_write;
HANDLE mutex;
LONG active_readers = 0;
LONG active_writer = false;
LONG waiting_writers = 0;
LONG waiting_readers = 0;
int number = 0;
int fl = 1;
void sig_handler(int signal)
{
    fl=0;
    printf("Signal %d is got\n", signal);
}

```

```

}
void start_read()
{
    InterlockedIncrement(&waiting_readers);
    if (active_writer || waiting_writers > 0)
    {
        WaitForSingleObject(can_read, INFINITE);
    }
    WaitForSingleObject(mutex, INFINITE);
    InterlockedIncrement(&active_readers);
    InterlockedDecrement(&waiting_readers);
    SetEvent(can_read);
    ReleaseMutex(mutex);
}
void stop_read()
{
    InterlockedDecrement(&active_readers);
    if (active_readers == 0)
    {
        SetEvent(can_write);
    }
}
DWORD reader(PVOID param)
{
    srand(GetCurrentThreadId());
    while (f1)
    {
        Sleep(rand() % 2000 + 5);
        start_read();
        printf("Reader %d read %d\n", GetCurrentThreadId(), number);
        stop_read();
    }
    return EXIT_SUCCESS;
}
void start_write()
{
    InterlockedIncrement(&waiting_writers);
    if (active_readers > 0 || active_writer)
    {
        WaitForSingleObject(can_write, INFINITE);
    }
    InterlockedDecrement(&waiting_writers);
}

```

```

        InterlockedExchange(&active_writer, true);
    }
}

void stop_write()
{
    ResetEvent(can_write);
    InterlockedExchange(&active_writer, false);
    if (waiting_readers > 0)
    {
        SetEvent(can_read);
    }
    else
    {
        SetEvent(can_write);
    }
}

DWORD writer(PVOID param)
{
    srand(GetCurrentThreadId());
    while (f1)
    {
        Sleep(rand() % 2000);
        start_write();
        number++;
        printf("Writer %d write %d\n", GetCurrentThreadId(), number);
        stop_write();
    }
    return EXIT_SUCCESS;
}

int main(void)
{
    int writer_amount = 3;
    int reader_amount = 5;
    DWORD thid[8];
    HANDLE pthread[8];
    void *handler = NULL;
    if ((handler = signal(SIGINT, sig_handler)) == SIG_ERR)
    {
        perror("Can't signal.\n");
        ExitProcess(EXIT_FAILURE);
    }
    if ((can_read = CreateEvent(NULL, FALSE, FALSE, NULL)) == NULL)
    {

```

```

        perror("Can't createEvent");
        ExitProcess(EXIT_FAILURE);
    }
    if ((can_write = CreateEvent(NULL, TRUE, FALSE, NULL)) == NULL)
    {
        perror("Can't createEvent");
        ExitProcess(EXIT_FAILURE);
    }
    if ((mutex = CreateMutex(NULL, 0, NULL)) == NULL)
    {
        perror("Can't createMutex");
        ExitProcess(EXIT_FAILURE);
    }
    for (int i = 0; i < writer_amount; i++)
    {
        pthread[i] = CreateThread(NULL, 0, writer, NULL, 0, &thid[i]);
        if (pthread[i] == NULL)
        {
            perror("Can't createThread");
            ExitProcess(EXIT_FAILURE);
        }
    }
    for (int i = writer_amount; i < writer_amount + reader_amount; i++)
    {
        pthread[i] = CreateThread(NULL, 0, reader, NULL, 0, &thid[i]);
        if (pthread[i] == NULL)
        {
            perror("Can't createThread");
            ExitProcess(EXIT_FAILURE);
        }
    }
    for (int i = 0; i < writer_amount + reader_amount; i++)
    {
        DWORD dw = WaitForSingleObject(pthread[i], INFINITE);
        switch (dw)
        {
            case WAIT_OBJECT_0:
                printf("thread %d finished successfully\n", thid[i]);
                break;
            case WAIT_TIMEOUT:
                printf("waitThread timeout %d\n", dw);
                break;
        }
    }
}

```

```
        case WAIT_FAILED:
            printf("waitThread failed %d\n", dw);
            break;
    }
}
CloseHandle(can_read);
CloseHandle(can_write);
CloseHandle(mutex);
return EXIT_SUCCESS;
}
```

Синхронизация и взаимоисключение

Синхронизация — один процесс заинтересован в выполнении другим процессом работы и получении результата работы. И только получив результат выполнения процесс может выполняться дальше.

То есть один процесс должен ждать, пока второй процесс выполнится и сформирует данные, чтобы их получить.

Или наоборот, первый еще не ждет данные, а второй уже сформировал данные и готов их отдать, первый же не готов их получить.

Взаимоисключение – запрет входа в критическую секцию, если в ней находится другой процесс.

Способ взаимодействия параллельных процессов: рандеву

- 1) Лучше всего реализовано в языке программирования Ada, который был разработан в военных целях
- 2) В языке Ada введено понятие задачи: Task – решает поставленную задачу
- 3) Задачи могут выполняться параллельно и независимо друг от друга, но имеют средства взаимодействия друг с другом.
- 4) При этом никак не регламентируется, как реализована поставленная задача. Сколько процессов, процессоров, что(?) нужно использовать.
- 5) В языке Ada Task интерпретируются как логические объекты, ведущие себя так, как будто выполняются на отдельных машинах, но не могут выполняться отдельно.
- 6) Task-и связаны с другими при помощи входов entry
- 7) ... *(тут кажется пропустил кусок лекции)*
- 8) Если task1 передало на вход task2, обе задачи теряют независимость и устанавливают рандеву, и пока рандеву действует, задачи синхронизируются.
- 9) Рандеву начинается с согласования фактических и формальных параметров, после чего осуществляется выполнение операторов.
- 10) **Свойство рандеву: последовательность операторов выполняется от имени обеих задач, находящихся в рандеву. То есть результат выполнения последовательности операторов получается обеими задачами одновременно.**

```
Task<имя> is entry<идент. входа>
(дискретный диапазон)
(формальная часть)
<другие объявления входов>
end<имя>
```

- 11) Если имеется entry, то в теле задачи должен быть хотя бы один оператор приема accept

```
accept<идент. входа>
(выравнивание)
<формальная часть>
```

```
do последовательность операторов
end
```

- 12) В рандеву есть проблема взаимодействия между процессами
- а) p1 выдал заявку на рандеву с другой задачей, нужно ждать ответа
 - б) p2 может не желать входить в рандеву

Распределенные системы

- 1) Сообщение — самое общее средство посылки и приема данных между процессами
- 2) Сокеты как средство взаимодействия параллельных процессов
 - а) Изначально реализованы в UNIX BSD
- 3) Распределенные системы — системы с разделенной памятью (каждый узел имеет собственную память)
- 4) Для синхронизации в распределенных системах нельзя использовать семафоры и мьютексы

Синхронизация по часам

- На каждой машине есть таймер. Процессы могут посылать друг другу сообщения. В ОС UNIX есть программа make
- Как правило в UNIX принято большие программы разбивать на файлы
- Изменение файла подразумевает перекомпиляцию только этого файла
- make проверяет время последней модификации
 - если исходный файл *name.c* имеет $t_{\text{посл.мод.}} > t_{\text{компиляции}}$, то система перекомпилирует его

```
<имя>.c 1224
```

```
<имя>.o 1223
```

```
требуется перекомпиляция
```

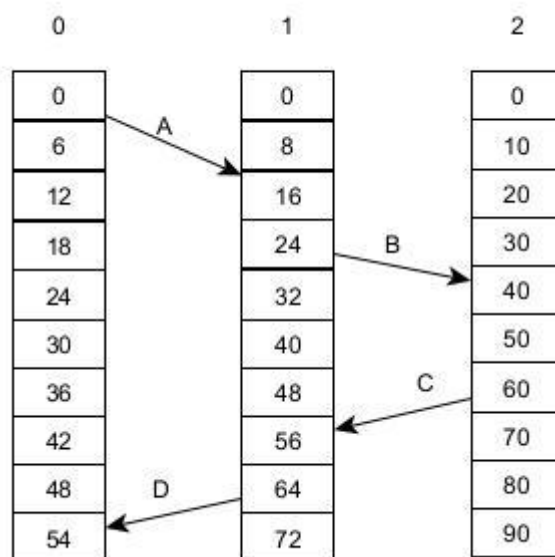
компьютер для перекомпиляции	1223	1224 создан <имя>.o	1225	1226
компьютер для редактирования	1222	1223 отредактирован <имя>.c	1224	1225

- При этом часы компьютера для редактирования отстают
- То есть внесенные в файл изменения не попадут и не приведут к перекомпиляции
- Каждый компьютер в распределенной системе имеет собственную микросхему CMOS RAM и собственные часы
- Работа часов основана на частоте тактового генератора, который входит в ПК. Являются высокопрочными, но имеют ограниченную точность. Ошибка генерации тиков в кварцевом генераторе составляет 10^{-5}
- Если генерируется 60 прерываний в секунду, в час будет сгенерировано 216000 тиков
- С учетом ошибки (точности) значение находится в диапазоне 215998 – 216002 тиков в час
- В современном мире существует служба точного времени
- Базовые понятия – солнечный день, секунда
- Солнечный переход — подъем солнца на максимальную высоту
- Солнечный день — промежуток между двумя солнечными переходами
- В 1948 изобрели атомные часы на атомах цезия 133
- Секунда — время, за которое атом цезия 133 совершит ровно 9192631770 переходов. Число переходов можно точно подсчитать.
- Международное бюро усредняет результаты 50 лабораторий, которые имеют атомные часы, и выдает глобальное время по атомным часам
- В реальном мире солнечный день удлиняется, а атомная секунда остается без изменений. Соответственно, если ничего не изменять, то полдень по атомным часам будет все раньше и раньше.
- Введена потерянная секунда (объявляется тогда, когда разница между TAI (International Atomic Time) и солнечным временем 800 мс)

- Весь мир перешел на универсальное согласованное время — UTC (Coordinated Universal Time)
- Большинство электронных компаний положили в основу своих 50-60 Гц часов систему UTC
- Когда международное бюро объявляют потерянную секунду, то часто меняется: 60->61 Гц, 50->51 Гц
- Сейчас если один ПК получается атомное точное время в локальной сети, то остальные ПК по некоторым алгоритмам корректируют свое время.
- Алгоритмы, по которым компьютеры получают точное время по глобальной сети
 - Алгоритм Кристиана
 - Алгоритм Беркли
 - Усредняющие алгоритмы
- В системах без глобальных доходов тоже нужна высокая точность времени: “случилось до”, “случилось после”. Для локальных систем имеет огромную важность правильный порядок событий, но подключиться к глобальной сети для них слишком дорого.

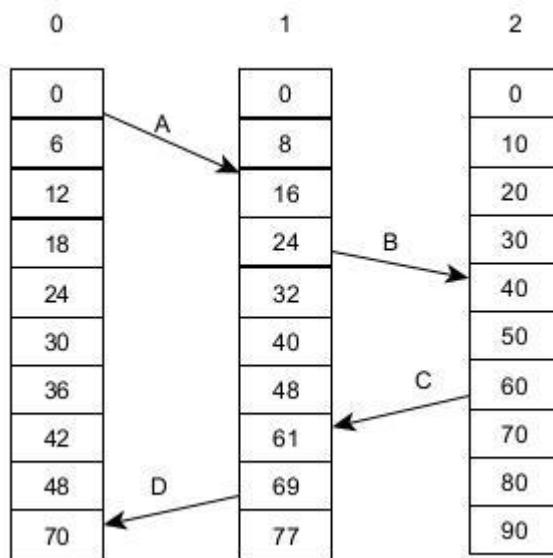
Алгоритм временных отметок Лампорта

- В 1978 г Лампорт предложил алгоритм синхронизации логических часов
- Рассмотрим 3 компьютера, на каждом свои локальные часы



- НА них выполняются процессы, значит взаимодействуют друг с другом путем отправки сообщений
- АВ: время получения > времени отправки — все ОК
- CD: $t_{\text{отправки}} > t_{\text{получения}}$, что невозможно
- Лампорт предложил добавлять в сообщения время отправки
- Получивший сообщение сравнивает время отправки и локальное время.

Если локальное время меньше, оно корректируется и устанавливается $t_{\text{лок}} = t_{\text{полученное}} + 1$

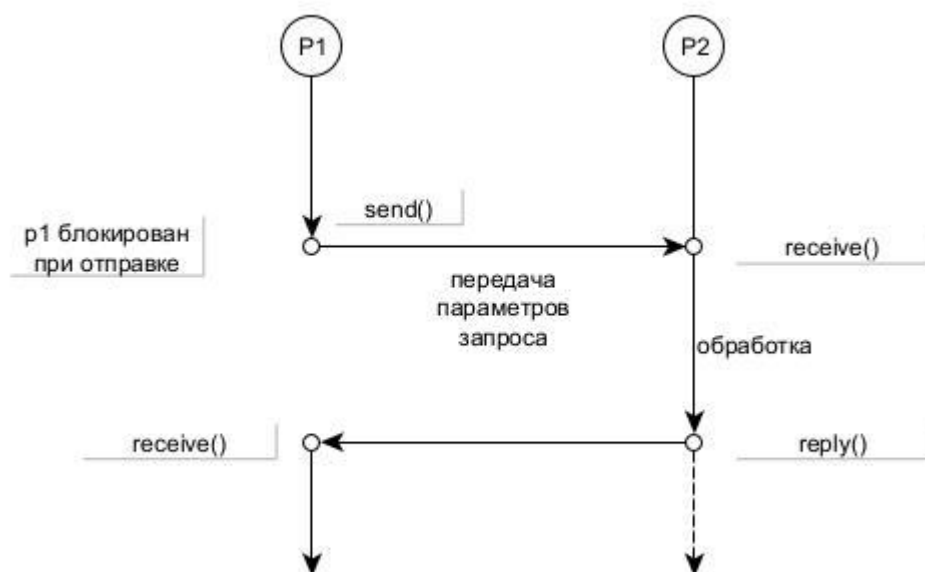


- В результате такой коррекции можно гарантировать соотношение “случилось до” – “случилось после” (причинно-следственное отношение)
- Реализация логических часов требует решения двух проблем:
 - локальную структуру данных для каждого процесса, представляющую локальное время
 - протокол, чтобы обновлять СД и обеспечивать условие согласованности
- Каждый процесс должен содержать структуры, которые позволяют использовать следующие две возможности:
 - локальные логические часы, обозначаемые как local clock (lc_i для процессора p_i)
 - глобальные логические часы global clock (gc_i для процессора p_i)
- Как правило, lc_i – часть gc_i

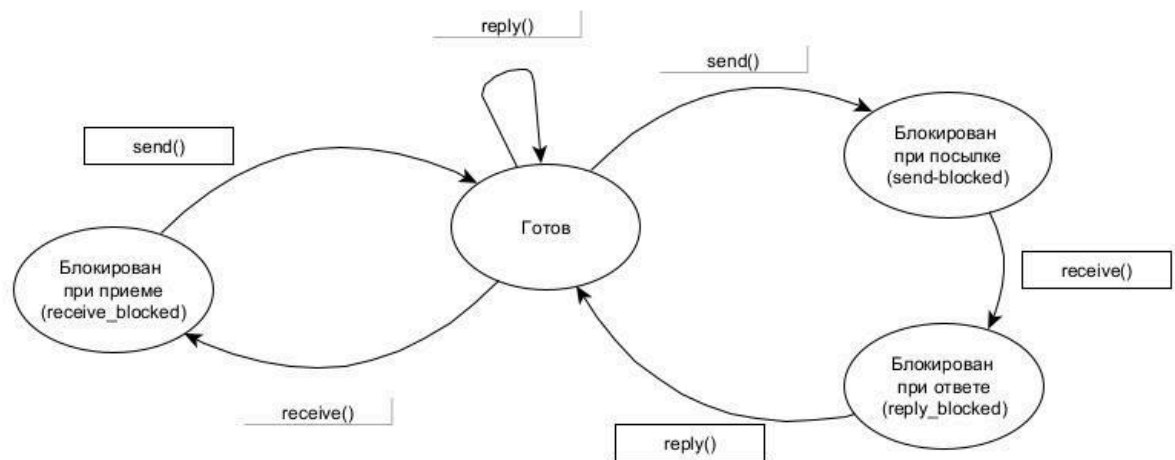
- Протокол гарантирует, что логические часы процессора и глобальное время должны быть согласованы
- Алгоритм Лампорта не единственный. Это всемирное(?) средство обеспечения правильного хода событий
- Векторные часы: Vector Clock
- Самые точные часы — в подмосковье. Государственный эталон времени и частоты: ГЭВЧ.
- Эти часы реализованы на Тулии (Хим. элемент Тулий)
- Они имеют частоту колебаний на порядок больше, чем у обычных атомных часов над цезием

Особенности передачи сообщений в распределенных системах

- Сообщения — самая общая форма взаимодействия процессов
- Рассмотрим сетевую подсистему
- Так как удаленное взаимодействие, сообщение дополняется дополнительной адресной информацией для доставки в сети (пакеты)
- send — сообщение послано
- receive — сообщение принято
- reply — ответ на сообщение (отправка ответа)
- процессы асинхронные, то есть выполняются с собственной скоростью. Предсказать, когда какой процесс придет в какую точку — невозможно.



- Если p1 еще не готов отправить сообщение, а p2 уже готов дать ответ
=> p2 будет блокирован в ожидании сообщения
- **3 состояния блокировки процесса при передаче сообщения**



- Существуют разные средства, исключающие часть этих блокировок
- Диаграмма для самого строгого протокола взаимодействия
- Блокировки неизбежны, когда необходимо строгое взаимодействие. Первый процесс запросил данные, и он заинтересован в их получении, иначе ему нечего делать.

Алгоритмы взаимодействия процессов в распределенных системах и подходы

1) Централизованный алгоритм

- Взаимодействие процессов построено в соответствии с взаимодействием процессов на отдельно стоящей машине
- Есть процесс-координатор
- Когда какой-то процесс хочет войти в критическую секцию (CR), он посылает координатору сообщение – запрос о входе в критическую секцию
- Координатор проверяет, есть ли в CR другой процесс или есть ли запрос на вход от других процессов
- Если в CR другой процесс, то запросивший процесс ставят в очередь
- Если два процесса одновременно, то координатор проверяет время, когда возникли запросы.
Если $t1_запроса < t2_запроса$, то координатор посылает разрешение процессу с более ранним запросом, а второй процесс ставит в очередь.

Алгоритм забияки

- Работоспособность алгоритма зависит от наличия процесса координатора
- Если он аварийно завершился, то система не работает
- Для восстановления работоспособности в такой системе запускается процесс выбора нового координатора
- Если какой-то процесс, отправив сообщение координатору, в течение установленного времени (таймаут) не может получить ответ (координатор отправляет процессу подтверждение получения запроса, подтверждение приема), то координатор отсутствует.
- Проблема установки таймаута (координатор жив, а мы считаем, что он умер)
- (Какой-то процесс обнаруживает отсутствие процесса-координатора (по таймауту) – не может в течение установленного времени получить подтверждение приема (имеется проблема установки таймаута))
- Любой процесс, обнаруживший отсутствие координатора, может инициализировать выборы нового координатора.
- Для этого процесс посылает специальное сообщение, обозначенное как “выборы”, в котором указывает свой номер (допустим, сетевой) (то есть посылает некоторую информацию о себе), всем заинтересованным процессам
- Получивший сообщение процесс сравнивает номер в сообщении со своим номером.
 - Если его (получившего сообщение процесса) номер больше, то он рассылает новое сообщение “выборы” со своим номером
 - Таким образом, процесс с меньшим номером прекращает выборы
- Процесс, который инициализирует выборы и имеет больший номер, должен их прекратить. В результате такой рассылки будет выбран процесс с наибольшим сетевым номером, и он становится координатором.
- Если процесс, выполнивший рассылку, не получил ни одного ответа, что чей-то сетевой номер больше, то он — координатор
- Проблемы:

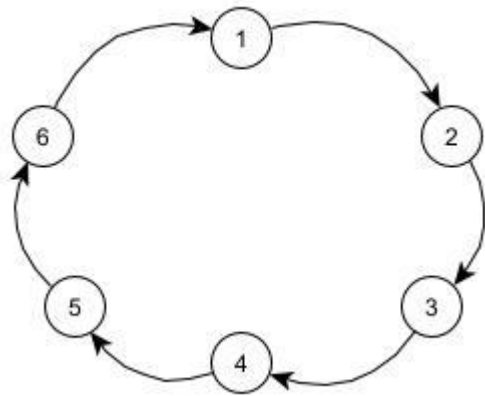
- все взаимодействующие процессы должны быть готовы стать координатором (нужно ПО координатора и ПО обычного процесса на каждой машине). Среди тех процессов, которые могут претендовать на роль координатора, должно быть ПО координатора.

2) Распределенный алгоритм

- Есть N взаимодействующих процессов $\Rightarrow$ могут обращаться к одним и тем же разделяемым ресурсам, и работа с разделяемыми ресурсами должна быть в режиме взаимоисключения.
- Когда любой из взаимодействующих процессов хочет войти в CR по какому-то ресурсу, он посылает $N-1$ сообщение с указанием критической секции и $t_{\text{отправления}}$ всем взаимодействующим процессам
- При получении такого сообщения другим процессом возможны три ситуации
 - а. Если процесс-получатель не собирается входить в данную CR, то он посылает процессу, отправившему запросу, сообщение-разрешение
 - б. Если процесс-получатель находится в этой CR, он не посылает ответ разрешение, а ставит процесс (запрос) в очередь
 - в. Если процесс-получатель хочет в данную CR, он сравнивает время в своем запросе и время в пришедшем сообщении.
 - Если его запрос возник позже, посылает сообщение разрешение
 - Иначе не посылает ответ разрешение, и ставит пришедший запрос в очередь
- $\Rightarrow$ чтобы процесс вошел в CR, он должен получить $N-1$ сообщение разрешение
- Используются отметки времени Лампорта
То есть сначала процессы должны синхронизироваться по времени.

3) Token ring

- Процессы образуют логическое кольцо
- Каждый процесс знает id визави, то есть того кому он должен отправить сообщение



- По этому кольцу движется токен — специальное сообщение, в котором указана CR, в которой может быть заинтересован взаимодействующий процесс
- Токенов столько, сколько критических секций
- Пусть по кольцу движется 1 токен. Действия процесса зависят от самоопределения относительно критической секции:
 - Если процесс в CR
 - Удерживает токен
 - Когда выйдет из CR, отправит токен дальше
 - Если процесс не заинтересован во вхождении в CR, то посылает токен сразу дальше

Выводы

- Самый надежный — централизованный — алгоритм записки
- Распределенный ненадежен
 - Если один из процессов завершился, запросивший никогда не получит N-1 ответ. Очень тяжело все восстанавливать.
- Проблема Token ring — разрыв кольца при завершении одного из процессов

Транзакции

- Транзакции неделимые по определению
- Неделимая — выполнение транзакции нельзя прервать
- Пример из бизнеса: процесс переговоров двух фирм
 - Условия могут меняться, но как только договор подписан, условия не меняются
- Процесс объявляет о желании создать транзакцию с одним или несколькими процессами
- Процессы могут создавать и удалять объекты(?)

- Затем инициатор заявляет, что хочет завершить транзакцию
- Если все согласны завершить транзакцию, то результат фиксируется и его нельзя изменить.
- Если какой-то из процессов не сообщил о согласии на завершение транзакции (мог завершиться сам процесс), то транзакция не фиксируется.
- Свойство транзакции: все или ничего.
- Транзакция — последовательность операций над одним или несколькими объектами (объекты БД, файлы), которые переводят систему из одного целостного состояния в другое целостное состояние
- Прimitives begin transaction, end transaction, read, write, abort

Свойства транзакций (ACID):

- Атомарность (Atomic) (Неделимость)
 - Если транзакция и все соответствующие ей действия выполнены успешно, то транзакция фиксируется (commit).
 - Если хотя бы в одном действии произошла ошибка, то выполняется откат (rollback)
 - Согласованность (Consistent)
 - Транзакция должна переводить данные из одного согласованного состояния в другое согласованное состояние
 - Пример: если товар куплен, то это должно быть зафиксировано в системе складского учета и числиться проданным.
 - Изолированность
 - Транзакция изолирована от других транзакций и процессов (то есть недоступна информация)
 - Данные не могут изменяться другой частью системы, которая не участвует в транзакции.
 - Устойчивость, или постоянство
 - При завершении транзакции все изменения фиксируются и никакой сбой не может привести к потере данных
-
- Согласованность, или упорядоченность
 - Пусть две или более параллельные транзакции выполняются в одно и то же время

- Конечный результат должен выглядеть так, если бы все транзакции выполнялись в некотором, заранее известном, порядке

2 механизма транзакций

Для обеспечения свойств транзакция существуют 2 механизма (2 подхода):

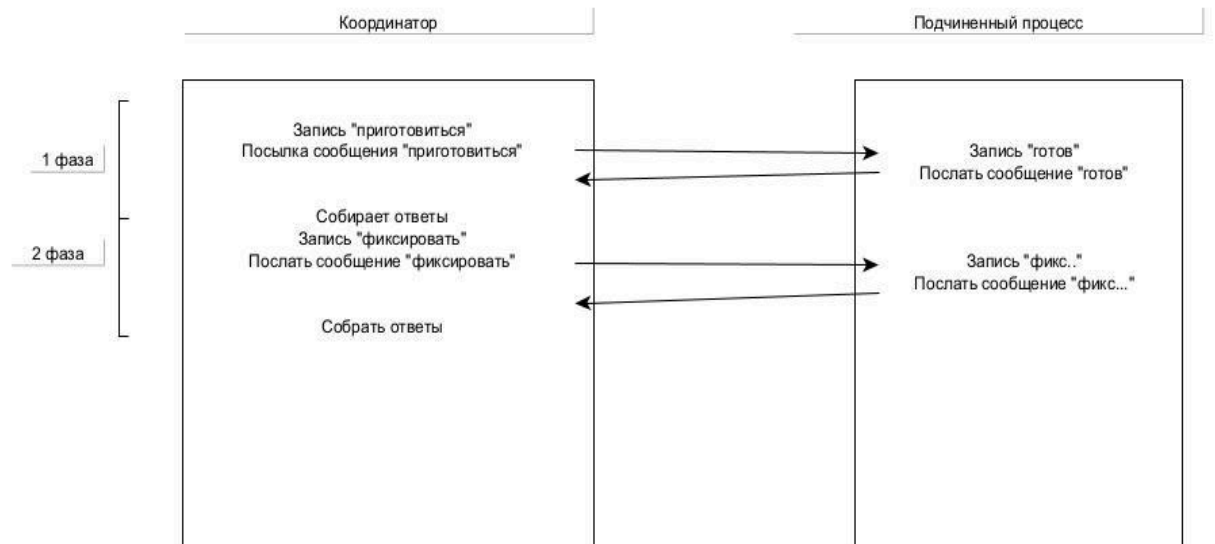
1. Путем создания для процесса, участвующего в транзакции, индивидуального рабочего пространства, в которое копируются все необходимые объекты.
 - a. В результате процесс выполняет все действия над копиями
 - b. Когда транзакция фиксируется, все изменения данных копируются в исходный объект
 - c. Данный подход требует больших накладных расходов: нужно много копировать одни и те же данные.
2. Список намерений
 - a. Модифицируются сами исходные объекты, а не копии.
 - b. Перед изменением любого объекта производится запись в специальный файл (журнал регистраций) значений исходного и измененного объектов
 - c. Только после этого производятся изменения
 - d. В этом журнале отмечается, какая транзакция изменяла данные, какой объект изменялся, а также время изменения.
<t_изменения, какой объект изменяется, какая транзакция изменяет>
 - e. После того, как транзакция фиксируется, все изменения остаются в силе, а транзакция фиксируется в журнале.
 - f. Если транзакция прерывается, происходит откат — все измененные объекты возвращаются в исходное состояние до транзакции

Протокол двухфазной идентификации транзакций

В распределенных системах фиксации транзакций могут потребовать взаимодействия нескольких процессов на разных системах (узлах). При этом у каждого узла свои бд, файлы => очень часто используется специальный протокол.

- Один из процессов выполняет роль координатора

- Координатор инициализирует транзакцию и делает в свой журнал регистрации запись об этом.
- После этого координатор посылает другим взаимодействующим процессам на узлах системы сообщение “началась транзакция”
- Какое-то время ПК выполняют действия над своими объектами
- В время t_0 координатор регистрирует завершение транзакции — посылает сообщение “приготовиться”

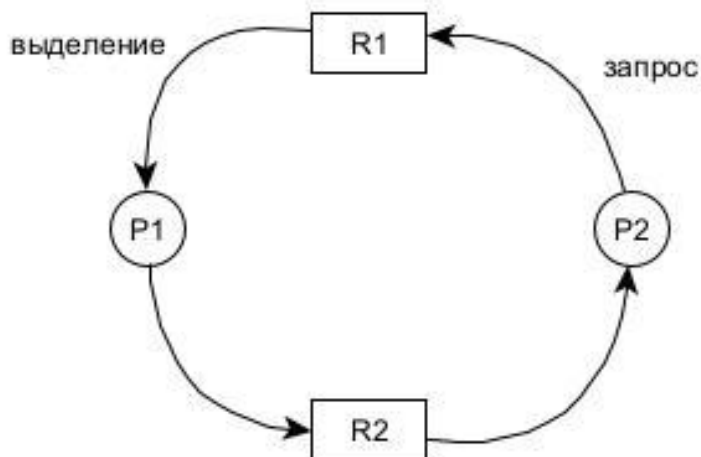


- Координатор должен получить N сообщений готов от N взаимодействующих процессов.
- Если удалось собрать N ответов, то выполняется запись “фиксировать”
- Если координатору не удалось собрать все ответы, то он рассылает об этом сообщения, и каждому нужно выполнить откат. (Если транзакция прервана, координатор рассылает сообщение, что надо совершить “откат”)
- Транзакции должны избегать тупиков
- Протокол(?) гарантирует, что если 2 или более транзакции выполняют параллельные действия, то конечный результат должен выглядеть так, как если бы все транзакции выполнялись в некотором наперед заданном порядке

Тупики



“Тупик становится проблемой, когда он возникает”

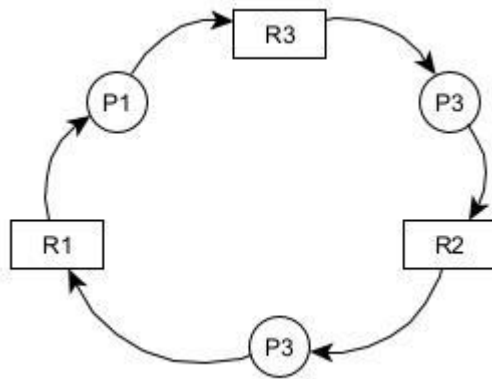


- Стрелка от процесса к ресурсу — запрос
- Стрелка от ресурса к процессу — выделение
- P1 запрашивает R1 и получает
P2 запрашивает R2 и получает
P1 запрашивает R2 и не может получить
P2 запрашивает R1 и не может получить
Они не могут получить эти ресурсы, так как они заняты. При этом они не могут продолжить выполнение, так как им нужны эти ресурсы. И освободить ресурсы не могут.

P1	P2
запрос R1	запрос R2
запрос R2	запрос R1

- Тупиковая ситуация, или тупик — это ситуация, возникающая в результате монопольного использования разделяемых ресурсов, когда процесс, владея ресурсом, запрашивает другой ресурс, занятый

непосредственно или через цепочку запросов другим процессом, ожидающим освобождения ресурса, занятого первым процессом.



- Ресурс может быть в нескольких экземплярах
- 2 типа ресурсов:
 - потребляемые ресурсы
 - сообщения
Такой ресурс перестает существовать, если его потребили, т.е. если сообщение было получено.
 - повторно используемые ресурсы
 - аппаратная составляющая ЭВМ
 - реентерабельные коды самой системы
 - объект ядра
- Потребляемые ресурсы — перестают существовать, когда их потребили
- Также к ресурсам относятся объекты ядра
- Свойства повторно используемых ресурсов
 - число единиц повторно используемых ресурсов ограничено и постоянно
 - например, процесс использует ограниченное число семафоров в наборе
 - изменение количества повторно используемых ресурсов — исключение, а не правило
- Вся теория тупиков строится на повторно используемых ресурсах

4 условия, необходимые и достаточные для возникновения тупика

- 1) взаимное исключение

- а) процесс монопольно использует предоставляемые ему ресурсы. Монопольный доступ реализуется методами взаимного исключения.
- 2) ожидание (hold and wait)
 - а) процессы удерживают полученные (занятые) ресурсы, и ожидают получения дополнительных ресурсов для выполнения
- 3) перераспределяемость
 - а) ресурсы нельзя отнять у процесса до его завершения или освобождения ресурса самим процессом
- 4) круговое ожидание
 - а) существует замкнутая цепь запросов процессов, в которой каждый процесс занимает ресурс, необходимый следующему(другому) процессу для дальнейшего выполнения

Подходы к борьбе с тупиками

- 1) Недопущение тупиков
 - а) создание в системе таких условий, при которых тупики невозможны
 - 2) Обход тупиков
 - а) Тупик возможен, в системе может быть несколько ситуаций, при которых возможен тупик.
Но предварительный анализ состояния системы позволяет такой тупик предотвратить.
 - 3) Тупик возможен
 - а) Система попадает в тупик
 - б) Необходимо обнаружить, что система попала в тупик и после этого восстановить состояние системы
 - с) Под системой понимается система взаимодействующих процессов
-
- Тупик не возникнет, если будет исключено хотя бы 1 из 4 условий возникновения тупика
 - Есть системы, которые в тупики попадать не могут:
системы реального времени, предназначенные для управления внешними системами или процессами (например, автопилот)

1 подход: Опережающее требование

- процесс должен запрашивать все необходимые ему в процессе выполнения ресурсы
- Исключается ожидание
- Минус: то есть процесс должен знать свою максимальную потребность (а обычно узнает по мере надобности) в каждом типе ресурса. А в наших системах ресурсы запрашиваются динамически.
- Существуют системы с хорошо изученными детерминированными процессами, когда действительно имеется информация о количестве всех типов ресурсов, которые используются процессами. Это системы специального, а не общего назначения.
- Плата: неэффективное использование ресурсов, так как ресурсы получают задолго до того, когда они будут необходимы. А это приводит к увеличению общего потребления ресурсов. Кроме того, такой подход может привести к бесконечному откладыванию.

Улучшение подхода 1

- Разделим процесс на несколько подзадач, которые будут запрашивать нужные им ресурсы (меньше ресурсов)
- Те же минусы: 1 задание выполнилось, а 2 откладывается

2 подход: Иерархическое распределение

- Ресурсы системы упорядочены, организованы в некоторую иерархию
- Реализуется правило, в соответствии с которым процесс может захватывать только ресурсы большие по номеру, чем он уже удерживает (исключает круговое ожидание)
- Например, ресурсы процесса: 1,2. Дальше может запросить 3,4,5.
- Для того, чтобы система была гибкой, можно добавить условие: если процесс не может получить ресурсы в нужном порядке (запросил 3, а нужны 1, 2), то он процесс может освободить ресурсы и запросить ресурсы заново в нужном порядке.
Это исключает потребность заранее запрашивать все ресурсы.
Но сейчас процессы выполнили часть работы, и должны вернуться в исходное состояние, и выполнять все заново.

А процесс может запрашивать ресурсы в порядке обратном потребностям: 5,4,3,2,1....

В итоге будет запрашивать/освобождать, пока не запросит все сразу.

- Нумерация ресурсов в системе решает проблему тупика.
- Нумерация ресурсов в системе – нетривиальная задача: ресурсов много, они самые разнообразные, поэтому определить наиболее эффективный порядок нумерации сложно.
- Как правило большие номера имеют более дефицитные ресурсы, но возникает вопрос, что назвать дефицитным ресурсом. Это зависит от системы.

3 подход

- Устранение условия неперераспределяемости ресурсов.
 - Если процесс не освобождает ресурсы, у процесса можно отобрать ресурсы
 - Подход может привести к тому, что процесс многократно запрашивает одни и те же ресурсы, что является тратой процессорного времени. Процесс фактически стоит на месте и не выполняется.
 - В действительности проводится мониторинг, и если обнаруживается такой процесс(видимо, который не освобождает?), то может его блок. (?). Но это неоднозначно, это может быть жизненно важный процесс.
 - Чтобы не возникало критических ситуаций системы специального назначения являются хорошо изученными, процессы в таких системах анализируются, т.е. все технические решения обоснованы. Мы же рассматриваем общие системы.

Классический алгоритм (Дейкстра) – алгоритм банкира

- Действия алгоритма подобны действиям банкира, дающего деньги заемщикам:
 - банкир не должен разориться => дает займ не всем.
 - кто-то может вернуть займ, только если ему дадут еще займ
 - заемщики — процессы, запрашивающие необходимые им для выполнения ресурсы
 - банкир — система, анализирующая запросы процессов

- Ограничения:
 - процессы должны указать в заявке свою максимальную потребность в каждом ресурсе
 - Количество процессов известно и ограничено
Замечание: в наших системах процессы создаются по мере необходимости, а ресурсы выделяются по мере надобности
 - Процесс не может запросить больше единиц ресурса, чем указано в его заявке
- Эти ограничения позволяют провести предварительный анализ заявки и решить, выделить ресурс или нет
- Процесс не может указывать в заявке больше единиц ресурса, чем имеется в системе
- Менеджер ресурсов по алгоритму банкира ищет последовательность процессов, которые могут завершиться
 - Если такая последовательность существует, то состояние системы безопасно относительно тупика => менеджер ресурсов удовлетворяет только те запросы, при удовлетворении которых ее состояние останется надежным

Пример относительно 1 типа ресурса

Процессы	Текущее распределение	Свободные единицы ресурса	Заявка
P1	4		12
P2	2		4
P3	4		8
-	-	2	-

- Показанное распределение безопасно, так как есть последовательность процессов (P2, P3, P1), которая может завершиться
- P2 не может запросить больше 2 единиц ресурса, так как он уже удерживает 2 единицы, а в заявке указано 4 единицы, и в системе есть 2 свободные единицы ресурса =>

- P2 завершится, освободит занимаемые им единицы ресурса, и вместе со свободным ресурсом это сможет удовлетворить максимальную потребность P3, так как максимальная потребность $P3 = 8$, то есть P3 может запросить не более 4 единиц, которые будут у системы
- Завершившись, P3 вернет в пул ресурсов ресурсы, и сможет удовлетворить запрос P1, которому нужно не более 8 единиц ресурса.
- Но безопасное состояние не всегда абсолютно.
Безопасное состояние может перейти в небезопасное относительно тупика, если процессу P1 выделена 1 единица ресурса (т.о. их всего 5), а у системы останется 1 свободная единица ресурса.

Процессы	Текущее распределение	Свободные единицы ресурса	Заявка
P1	5		12
P2	2		4
P3	4		8
-	-	1	-

- Нет необходимой последовательности
- Небезопасное состояние относительно тупика не всегда приведет к тупику, т.к. выполняющийся процесс может не запросить максимальное количество ресурсов
- Состояние системы является безопасным, если существует последовательность процессов, такая что:
 - первый процесс в найденной последовательности обязательно завершится, так как если он запросит даже максимальное заявленное число единиц ресурса, у системы имеется необходимое количество свободных единиц ресурса для удовлетворения запроса
 - второй процесс последовательности может завершиться, если завершился первый процесс и вернул ОС все занимаемые им единицы ресурса, что в сумме со свободными единицами

ресурса позволит удовлетворить максимальную потребность второго процесса

- i процесс в последовательности может завершиться, если все предыдущие $i-1$ процессы успешно завершатся, и вернут системе занимаемые им единицы ресурса, что позволит удовлетворить потребность i в ресурсе.
- Для определения состояния системы относительно тупика необходимо исследовать $n!$ последовательностей, прежде чем состояние системы будет отнесено к безопасному или небезопасному
=> алгоритм банкира имеет **неоспоримое(показал суть решения)** теоретическое значение, но его практическое применение затруднено в силу выполнения $n!$ проверок. Также очень много ограничений. Однако фактически задача обойти тупик решена.
- Так как существуют системы, которые не могут попадать в тупики, алгоритм Дейкстры привел к ряду аппроксимаций. Одна из аппроксимаций — алгоритм Хаберманна (Габерманна).

Алгоритм Хаберманна

- Обобщение алгоритма банкира Дейкстры
- Для обнаружения тупиков и для реализации обхода тупиковых ситуаций используется одна формализованная система.
- Система описывается как двудольный граф
- Есть два непересекающихся множества вершин:
 - множество вершин процессов
 - множество вершин ресурсов
- Граф является направленным
- С помощью дуг соединены вершины различных подмножеств
 - “Выделение” — дуга, выходящая из вершины подмножества ресурсов и входящая в вершину подмножества процессов
 - “Запрос” — дуга, выходящая из вершины подмножества процессов и входящая в вершину подмножества ресурсов
- В программе такой граф можно представить либо матрицей, либо связанным списком. Матрицами нагляднее, так что используются они. N процессов, M типов ресурсов.
- Как узнать, что состояние безопасное?
 - начинаем с заданного состояния

- имитируем выполнение каждого процесса, выделяя ему максимальное количество ресурсов
- если все процессы завершатся, то состояние безопасное
- Есть n процессов и m ресурсов

- Матрица доступных ресурсов

- Max-Avail matrix $A = (a_1, a_2, \dots, a_m)$

- Матрица заявок

- Max-Claim matrix B

$$\text{Max-Claim matrix } B = \begin{pmatrix} b_{11} & \dots & b_{1m} \\ \vdots & \ddots & \vdots \\ b_{n1} & \dots & b_{nm} \end{pmatrix} = \begin{pmatrix} B_1 \\ B_2 \\ \dots \\ B_n \end{pmatrix}$$

b_{ij} — максимальное количество единиц ресурса R_j , запрошенных P_i

- Матрица распределения

- Allocation matrix C

$$\text{Allocation matrix } C = \begin{pmatrix} c_{11} & \dots & c_{1m} \\ \vdots & \ddots & \vdots \\ c_{n1} & \dots & c_{nm} \end{pmatrix} = \begin{pmatrix} C_1 \\ C_2 \\ \dots \\ C_n \end{pmatrix}$$

где C_{ij} — количество единиц ресурса R_j , удерживаемых P_i

1. Для всех k $B_k \leq A$ — процесс не может запросить больше ед. ресурса, чем доступно
2. $C \leq B$ процесс не может запросить больше ресурса, чем указано в его заявке
3. $\sum_{k=1}^n C_k \leq A$

Никогда не может быть выделено ресурсов больше, чем доступно

4. Матрица доступных ресурсов

$$D = (d_1 \quad d_2 \quad \dots \quad d_m) = A - \sum_{k=1}^n C_k$$

d_i — количество единиц ресурса R_i , доступное в текущем состоянии

5. Матрица возможных запросов E

$$\text{Need matrix } E = \begin{pmatrix} e_{11} & \dots & e_{1m} \\ \vdots & \ddots & \vdots \\ e_{n1} & \dots & e_{nm} \end{pmatrix} = B - C = \begin{pmatrix} E_1 \\ E_2 \\ \dots \\ E_n \end{pmatrix}$$

6. Матрица запросов F

$$\text{Request matrix } F = \begin{pmatrix} f_{11} & \dots & f_{1m} \\ \vdots & \ddots & \vdots \\ f_{n1} & \dots & f_{nm} \end{pmatrix} = \begin{pmatrix} F_1 \\ F_2 \\ \dots \\ F_n \end{pmatrix}$$

7. Предварительное состояние (предполагаем, что удовлетворены все запросы и проведем анализ:

$$D \leftarrow D \cdot F$$

$$C_i \leftarrow C_i + F$$

$$E_i \leftarrow E_i - F$$

Запрос удовлетворен, если предыдущее состояние является безопасным.

Алгоритм:

- 1) Выбираем незавершившийся процесс $P_i: E_i < D$
 - а) То есть доступно больше ресурсов, чем нужно P_i
 - б) Если такого нет, переход на шаг 3
- 2) $D \leftarrow D + C$
 - а) Процессу выделены запрошенные единицы ресурса
 - б) P_i помечается как завершившийся
 - с) Выполняется как переход на шаг 1
- 3) Если все процессы отмечены как завершившиеся
 - а) система находится в безопасном состоянии
 - б) иначе в опасном

Если система в небезопасном состоянии, то запрос блокируется и состояние системы сбрасывается следующим образом:

- $D \leftarrow D + F_i$
- $C_i \leftarrow C_i - F_i$
- $E_i \leftarrow E_i + F_i$

Кроме того, процесс, помеченный на шаге 2, сбрасывается

- $D \leftarrow D - C$
- P_i помечается как незавершенный

Пример

max-Avail $A = (243)$ - матрица дост. ресурсов
 max-Claim $B = \begin{pmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{pmatrix}$ - матрица заявок

Allocation $C = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$ - матрица уже выданных рс.

Available $D = (0, 1, 1)$ - матрица доступных ресурсов

Возможные
заявки $E = \begin{pmatrix} 0 & 0 & 2 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix} \leftarrow B - C$

Пусть P_1 делает запрос единицы ресурса R_3
 $F_1 = (001)$

Предварительное состояние
 $D = (0, 1, 0)$, т.к. P_1 взял ед. рес. R_3
 $C = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$
 $\underline{\quad\quad\quad}$
 $\quad\quad\quad 2 \ 3 \ 3$

$E = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix} P_3$

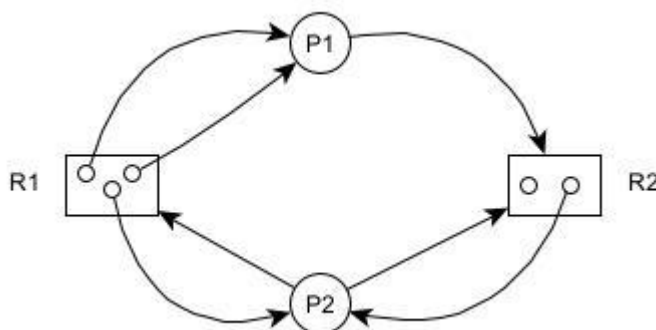
- Может завершиться P_3 , так как $E_3 \leq D$
- В результате P_3 вернет удерживаемые ресурсы в пул свободных ресурсов
 - $(1, 0, 1) \rightarrow D = (1, 1, 1)$

- Можно будет удовлетворить запросы процессов P_1 и P_2 и следовательно предварительное (?) состояние запроса (процесса ?) P_1 безопасное
- Следовательно, запрос F_1 может быть удовлетворен

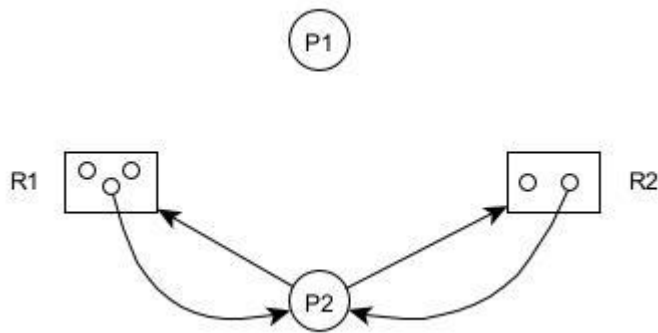
Существенным ограничением является определенное (известное) количество процессов.

Метод редукции графа

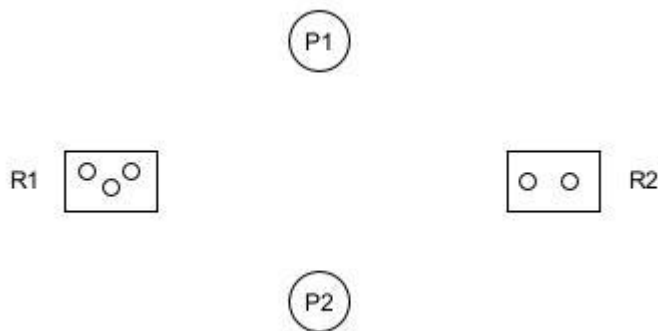
- Реализация на основе двудольного графа
- Есть два множества вершин:
 - множество вершин процессов
 - множество вершин ресурсов
- Граф является направленным
- С помощью дуг соединены вершины различных подмножеств
 - “Выделение” — дуга, выходящая из вершины подмножества ресурсов и входящая в вершину подмножества процессов
 - “Запрос” — дуга, выходящая из вершины подмножества процессов и входящая в вершину подмножества ресурсов
- Из графа удаляются все ребра, которые отражают, что ресурс может быть выделен процессу
- В результате после удаления ребер, если остаются только изолированные вершины — система не в тупике
- Пример:



- У системы есть еще свободная единица ресурса R2 и если она будет выделена P1, он сможет завершиться

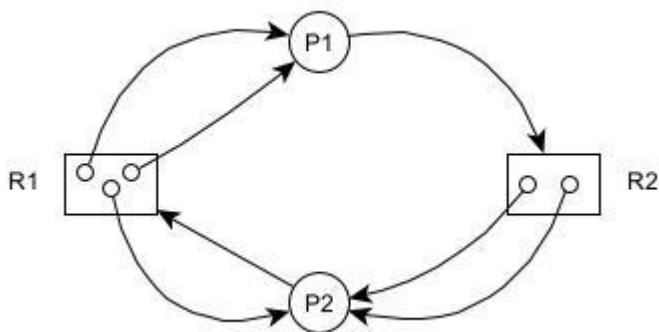


- Следующий шаг:



- Система не в тупике

Пример 2 (замкнутая цепь запросов)



Данный граф нельзя сократить ни по P1, ни по P2, так как у системы нет свободных ресурсов, требуемых P1 и P2

- Теорема 1: Граф является полностью сокращаемым, если есть последовательность сокращений, которая сокращает все дуги
- Теорема 2: Цикл в графе повторно используемых ресурсов является необходимым условием тупика
- Теорема 3: Если состояние S не является состоянием тупика и выполняется переход из состояния S1 в состояние S2 в результате

действия процесса P_i , причем $S2$ является тупиковой, то операция, выполняемая процессом P_i , является запросом

- Тупик возникает только в результате запроса

- Для представления двудольного графа минимум необходимы две матрицы и два связанных списка

- матрица распределения

p/r	1	2	3
1	0	1	1
2	1	3	0
3	1	5	0
	2	9	1

- матрица запроса

p/r	1	2	3
1	1	1	0
2	0	0	1
3	1	1	2

- Вектор свободных единиц ресурсов

$$F = \begin{array}{|c|c|c|c|} \hline r & 1 & 2 & 3 \\ \hline & 4 & 0 & 2 \\ \hline \end{array}$$

- $$R_{ij} = \sum_{i=1}^n b_{ij} + F_j$$

- $R = (6, 9, 3)$

- $P2$ имеет строку меньше $< F \Rightarrow$ он может завершиться

- Матрица распределения

p/r	1	2	3
1	0	1	1
2	0	0	0
3	1	5	0

- $F = (5, 3, 2)$

- $P1$ завершился

p/r	1	2	3
1	0	0	0
2	0	0	0
3	1	5	0

-
- $F=(5,4,3)$
- Система не находится в тупике $\Rightarrow$ завершился P3

p/r	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0

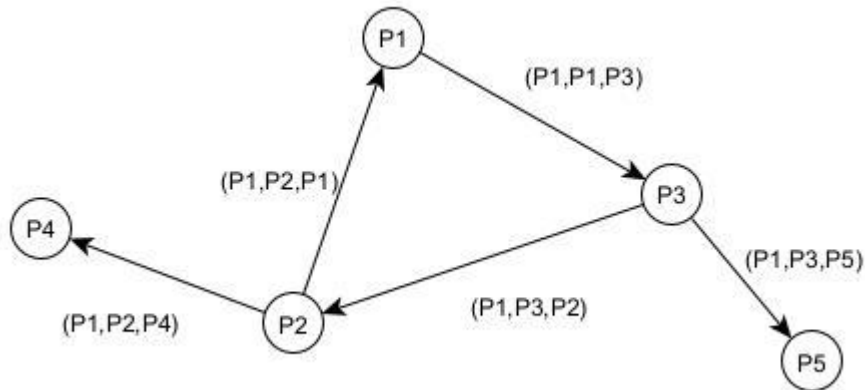
-
- $F=(6, 9, 3)$
- Системы:
 - известны процессы, формирующие сообщения и процессы, потребляющие сообщения
 - известны производители, неизвестны потребители
 - оба неизвестны

Алгоритм Канди-Мисра-Хаасс (на основе пробного сообщения)

- 1983 г — алгоритм Канди-Мисра-Хаасс
- Если процесс делает запрос на ресурс, который занят, то процесс генерирует сообщение-зонд и посылает его всем процессам, которые могут удерживать этот ресурс
- В этом сообщении:
 - id процесса, который оказался заблокирован и был инициатором сообщения типа зонд
 - id процесса, отправившего текущую версию сообщения (получил сообщение и тоже ждет этот ресурс)
 - id процесса, который должен получить сообщение типа зонд

инициатор	отправитель	получатель
-----------	-------------	------------

- Процесс, получивший сообщение-зонд, проверяет эти поля и если в 1 поле он обнаруживает свой id, то система находится в тупике



Управление оперативной памятью

-
- В ОС имеется разная по типу память
- Иерархия памяти строится с точки зрения близости к процессору
- Иерархия:
 - Оперативная память предназначена для поддержания работы процессора
 - В архитектуре фон Неймана процессор собственной памяти не имеет
 - Процессор может выполнять только ту программу, что загружена в память
 - Вторичная (внешняя память) предназначена для долговременного хранения
- Обычные файлы имеют разный формат и тип
- Программы, выполняемые процессором, рассматриваются как текстовые файлы
- Обращение к памяти производится через локальную шину памяти – циклы обращения к памяти (занимает определенное количество тактовых импульсов)
- 2 кэша 1 уровня принадлежат процессору
 - каждое ядро имеет 2 кэша
- 1 кэш 2 уровня, общий
- В этих кэшах хранится информация, к которой было произведено последнее обращение

- Существует кэширование в оперативной памяти, чтобы уменьшить число обращений к диску (так как это внешнее устройство: задействована системная шина. Обращения, связанные с прерываниями, аппаратно обрабатываются прерывания => большие затраты по времени при обращении к внешнему устройству)
- Два вида управления памятью:
 - вертикальное — передача данных между различными уровнями
 - горизонтальное
- Втор. зат. устр. при файловой системе, но(то?) адресное пространство внешней памяти делится на 2 части:
 - область, управляемая файловой системой
 - область своппинга – участвует в общем управлении памятью (под которым подразумевается управление оперативной памятью)
- Распределение памяти:
 - Связное распределение
 - Несвязное распределение

Связное распределение

Первое поколение

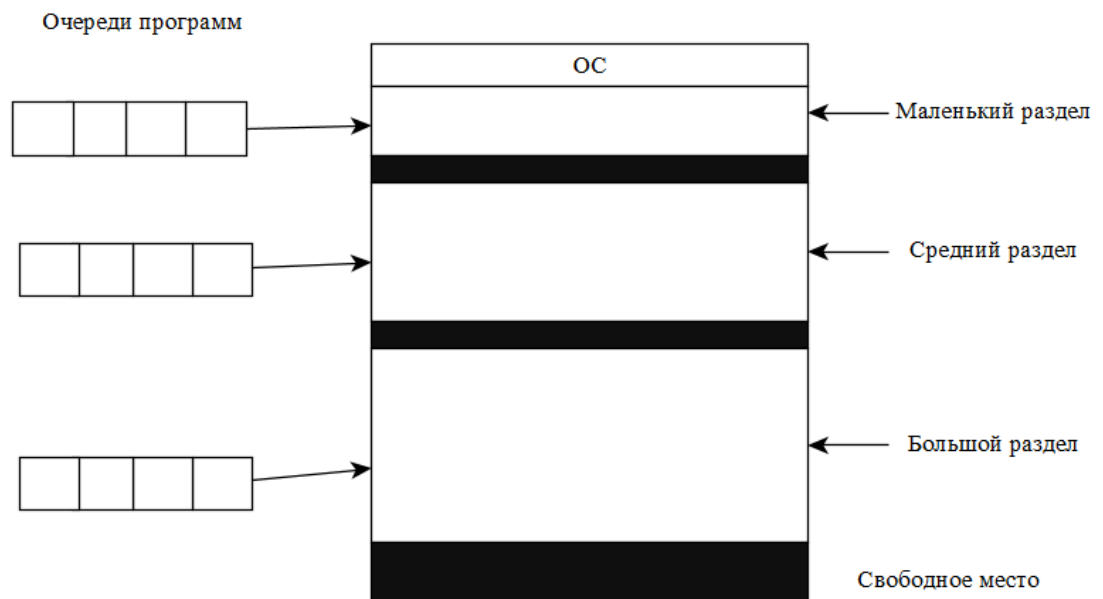
Реализовано первым: в первых однозадачных системах целиком вся программа загружалась в оперативную память. Объемы программ/решаемой задачи соответствовали вычислительным способностям.

Статическое распределение

- Во 2 поколении появилась возможность заменить действия оператора на специальную программу — ОС, но в памяти ПК какое-то время находилась все еще 1 программа, которая выполнялась от начала до конца (одионое связное распределение)
- Процессоры апнули => появилась возможность загружать в память несколько программ. Загружались программы так же целиком, но ОС умела переключаться между заданиями, то есть на это не тратилось время.
- В DOS программа тоже целиком загружалась в память, но делилась на 3 сегмента. По умолчанию сегменты выравнивались по началу

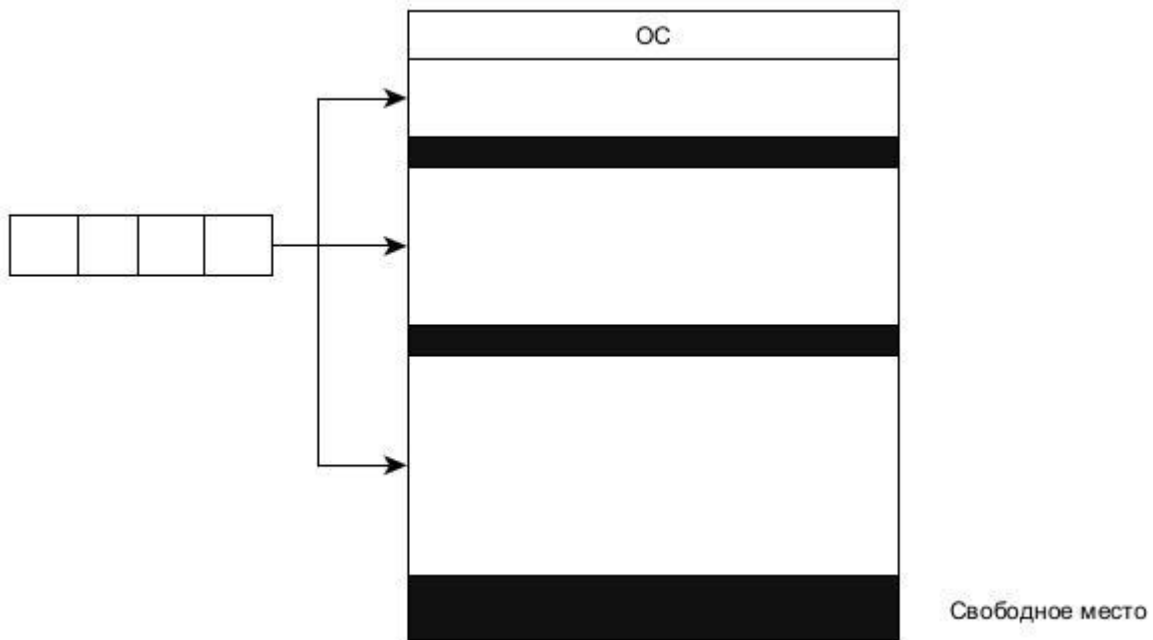
страницы(?). Остается идея связного распределения, когда программа целиком в памяти, все ресурсы предоставлялись одной программе, она выполнялась от начала до конца.

- Во 2 поколении в память загружалось большое число программ, каждая в своем разделе – распределение памяти разделами
- Изначально разделы были фиксированного размера, который определялся исходя из статистики размеров программ
- Раздельные очереди



Распределение разделами фиксированных размеров

- Найти программу, точно соответствующую размеру раздела сложно и маловероятно.
 - При загрузке программы возникает не использованное адресное пространство
 - Маленьких программ больше чем больших => очередь маленьких заполнена, а больших нет.
- Решение этой проблемы: распределение разделами с 1 очередью.
- Видим из рисунка, что память используется крайне неэффективно.
- Размеры разделов определяются из статистической информации о наиболее часто встречающихся разделах программ до запуска ОС, то есть статическое определение размера (Определение размеров разделов происходит до начала работы системы)

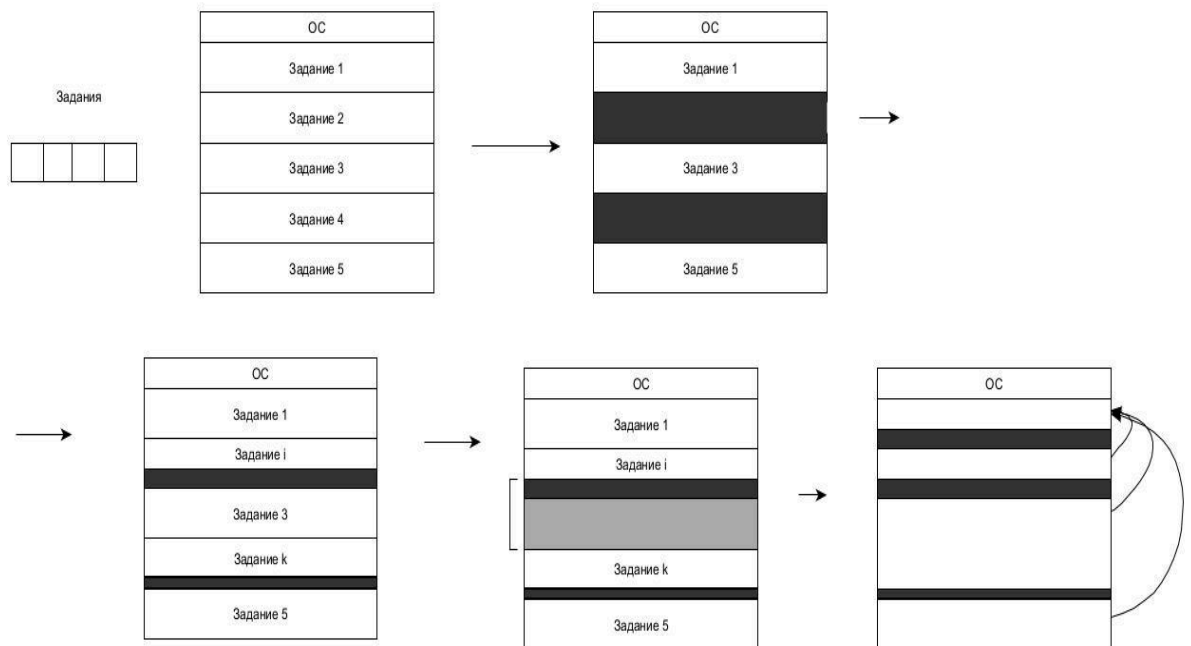


Распределение разделами. Одна очередь

Динамическое распределение

- До начала работы системы память на разделы не делится. В начале работы в память загружаются программы. В начальный момент времени имеется возможность загрузить одно задание за другим без потери оперативной памяти. (верхняя часть картинки)
- Но задания имеют свойство завершаться => освобождается раздел, занятый заданием.
- Существует очередь заданий, и в разделы надо поместить другое задание. Как искать задание для помещения в освободившийся раздел?
- Стратегии выбора раздела:
 - первый подходящий по размеру, не важно какой
 - самый тесный
 - => размеры должны быть упорядочены по возрастанию
 - самый широкий
 - из соображений, что в разделе останется место для еще одного задания

- В результате загрузки новых заданий(нижняя часть фото)

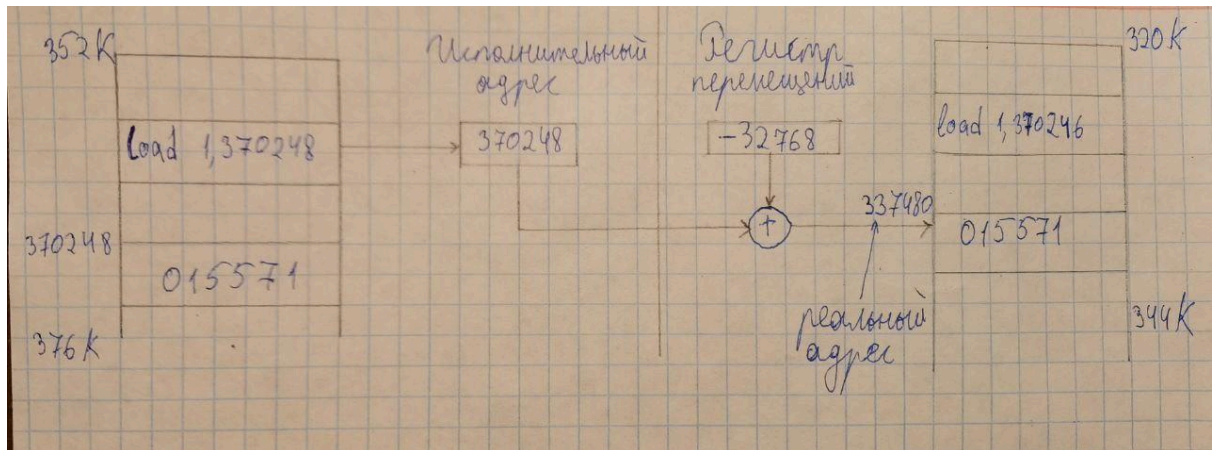


- Невозможно найти точно соответствующий размеру задания раздел, в результате чего возникает свободное адресное пространство. Для более эффективного использования нужно объединять свободные разделы.
- Но система функционирует без выключения длительное время. То есть задания загружаются, выгружаются, освобождая разделы. Соседние свободные разделы объединяются, но в итоге многократной загрузки и выгрузки в системе возникает большое количество очень небольших участков памяти, в которые ничего нельзя загрузить — “дыры” — фрагментация памяти.
- В результате этого явления может быть не распределено до 30% памяти. То есть $\frac{1}{3}$ объема памяти могла не использоваться.
- Как решить?
 - 1) Перезагрузить систему

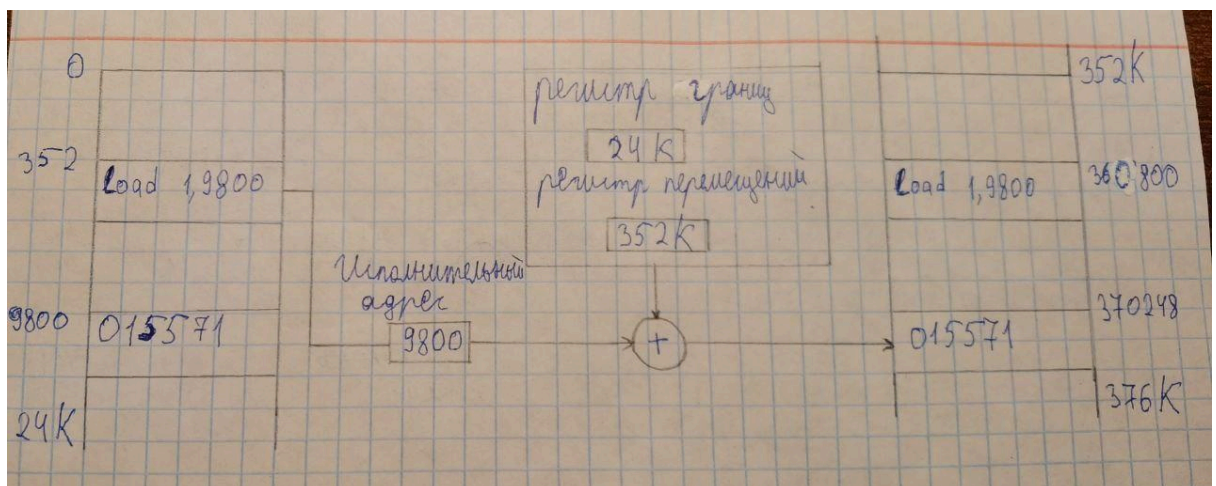
Цена: потеря проделанной работы. Задание выполняется, а в системе принято решение о перезагрузке, при этом проделанная заданием работа теряется.
 - 2) Перемещение разделов с целью освобождения памяти
- В итоге возникает свободное адресное пространство, которое сможет использоваться. Для этого нужны специальные средства.

Как будто все, что было до, подразумевает само по себе наличие логического ап

- При этом компиляция статическая. Когда задание компилируется — оно связывается с определенными адресами.
- Чтобы обратиться по реальному адресу нужно выполнить преобразование адреса — нужно переместить ... к младшим адресам раздела.
- Поддерживается аппаратно



- Было замечено, что механизм размещения задания в памяти фактически не связан с физическим размещением задания в памяти
- => Появилось логическое адресное пространство, адреса стали динамическими



- => Любое задание думает, что начинается с 0 адреса
 - В результате появилось смещение
- Регистр границ контролирует выход процесса за собственное адресное пространство

- Вопрос остался: когда выполнять перекомпоновку?
Перекомпоновка связана с накладными расходами. Два варианта:
 - сразу после освобождения раздела
 - если не найден раздел нужного размера. Во 2 случае выполняется реже

Несвязное распределение

- Сначала появилось страничное распределение
- Вся информация о разделе должна храниться в таблице – из таблицы выбирается информация и выполняется преобразование
=> почему бы не поделить адресное пространство на участки одинакового размера — страницы
=> размер задания должен быть кратен размеру страницы
- Каждое задание должно иметь соответствующую таблицу, в которой указан начальный адрес каждой страницы и сопоставлен с адресом загруженной страницы
- => физическое пространство стало делиться на страницы (фреймы)
- Пока что все страницы задания загружались в оперативную память
- Позже задания стали делить на логические единицы — сегменты
Их можно загружать в разброс, но сохраняется логика деления.
При делении на страницы делим просто на равные куски и загружаем в фреймы(нет логики деления).
- Число свободных фреймов должно быть не меньше числа страниц задания
- Нужно ли хранить в памяти все адресное пространство задания?
Если страничное распределение — нужно ли загружать все страницы адресного пространства процесса?
- От несвязного распределения до виртуальной памяти один шаг:
 - Нет необходимости держать в памяти все страницы/сегменты задания
 - В памяти должны находиться те части кода, к которым обращается процессор

Виртуальная память

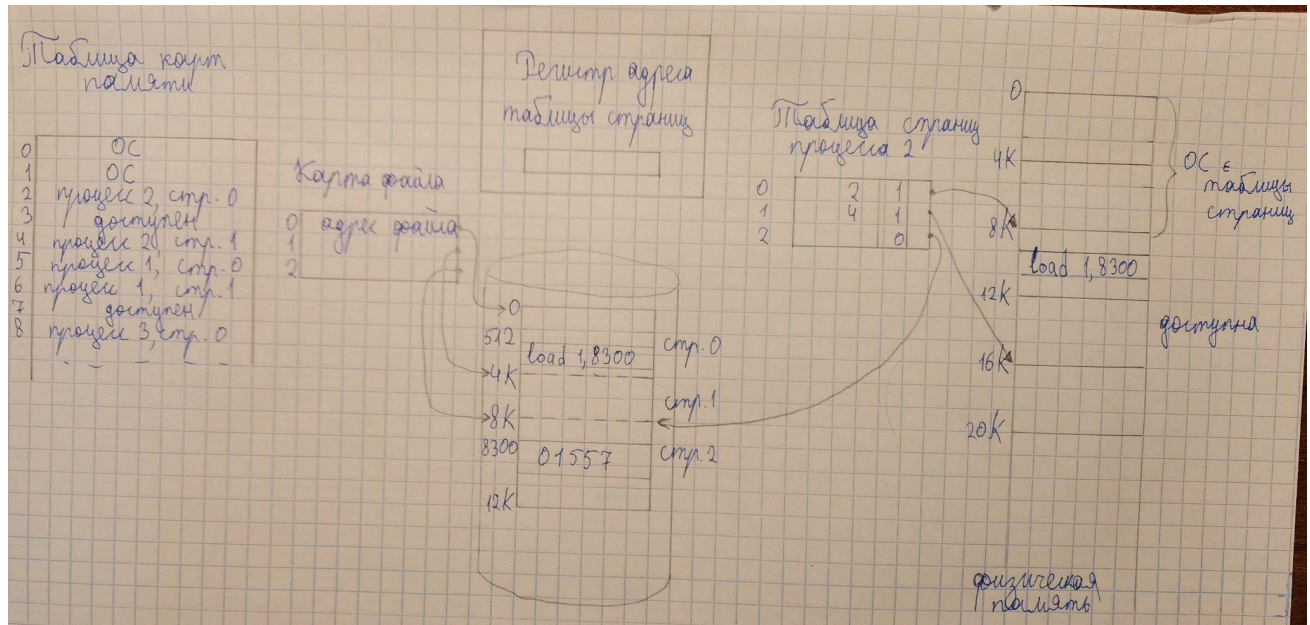
- Виртуальная память — память, объем которой превышает объем физического адресного пространства

- Устранилось требование загружать в память все адресное пространство задания: в памяти должна находиться та часть адресного пространства задания, к которой обращается процессор
- Управление виртуальной памятью:
 - страницами по запросу
 - сегментами по запросу
 - сегментами, разделенными на страницы, по запросу
- Память выделяется процессу по запросу
- По запросу => загрузка соответствующего адресного пространства процесса в ОЗУ производится по запросу.
Такой запрос возникает при обращении к адресу, отсутствующему в физической памяти.
- Эти запросы реализованы в виде прерываний
- В страничной памяти это страничные прерывания
- В результате страничного прерывания страница должна быть загружена в память, при этом выполняется преобразование
- Когда процесс создается, он идентифицируется (ему выделяется дескриптор), а затем ему выделяется минимально необходимое количество страниц
- Виртуальный адрес:
 - номер страницы
 - смещение

Страничное распределение

- Логическое адресное пространство:
 - виртуальное
 - пространство, которое формируется системой
Большую часть времени система оперирует виртуальными адресами.
При этом, как и до виртуального адресного пространства, адресное пространство процесса делится на страницы => оно должно быть описано соответствующей таблицей страниц
 - физическое

- условно, так как только часть адресного пространства в физической памяти
- До того, как страница попадет в физическую память, она должна находиться на диске



Существует таблица, которая связывает страницы адресного пространства процесса с дисковым адресным пространством. Также должна существовать таблица, которая отображает, какие виртуальные страницы загружены в физическую память. Бит присутствия — если страница загружена в память = 1, иначе = 0. Если в процессе выполнения программы производится обращение к адресу, отсутствующему в памяти, возникает страничное прерывание и страница будет загружена в память. В современных ОС — mmu — memory management unit — часть кода, решающая эти задачи. => необходимо выполнять преобразование виртуального адреса к физическому.

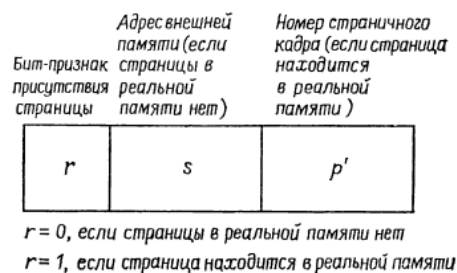


Рис. 8.11 Строка таблицы страниц.

3 способа преобразования

Прямое преобразование

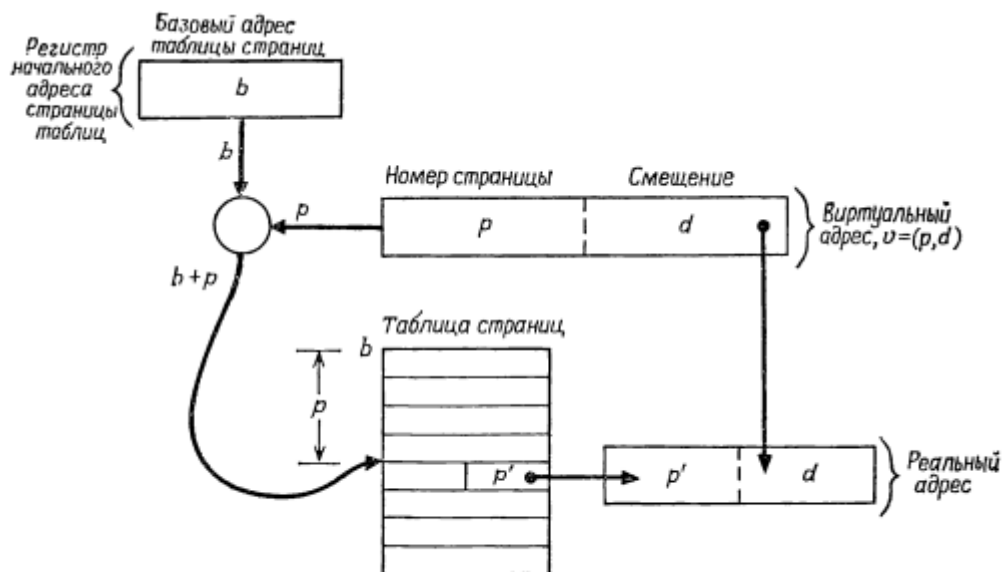
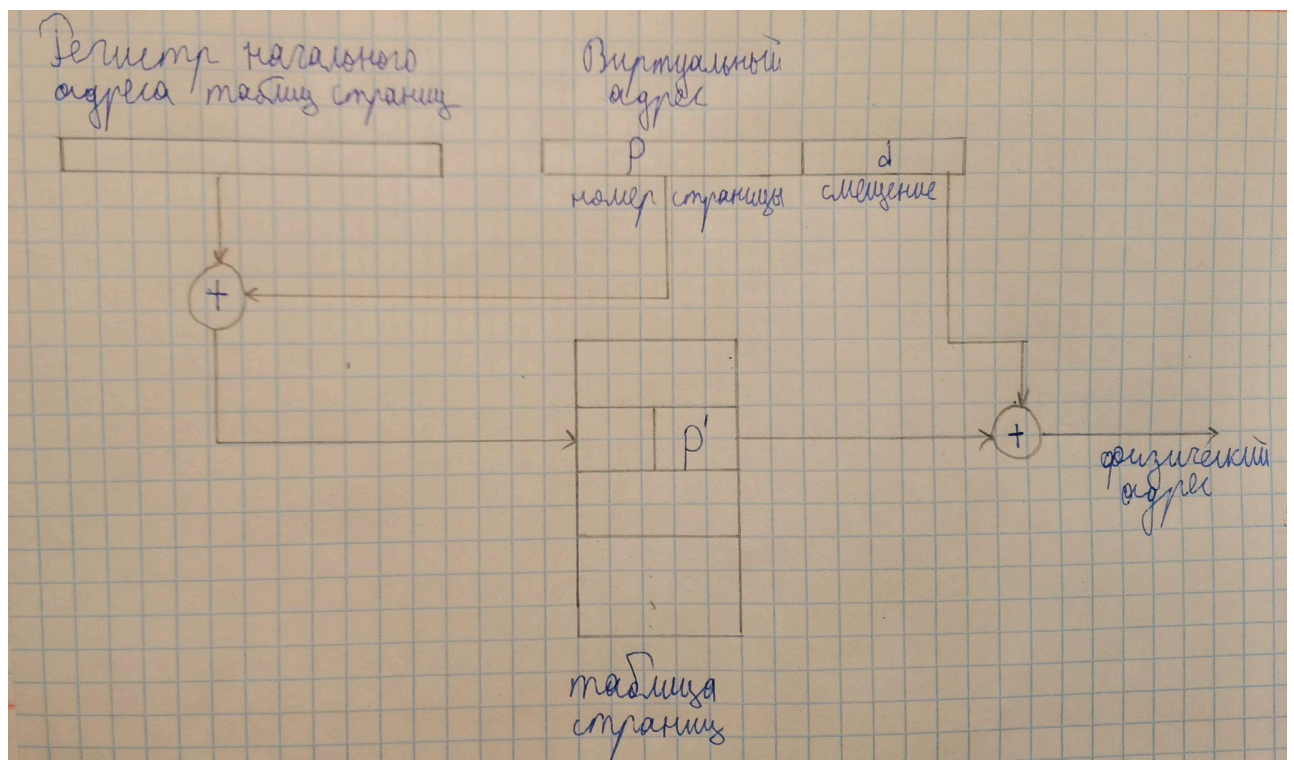


Рис. 8.12 Преобразование адреса страницы путем прямого отображения.



- Виртуальный адрес делится на 2 части: номер страницы и смещение
- Регистр базового адреса таблицы страниц = регистр начального адреса таблицы страниц
- Номер страницы – смещение к начальному адресу таблицы страниц

- Таблицы имеют один и тот же размер => добавляем смещение и формируем физический адрес
- Таблица страниц в оперативной памяти (в ней дескриптор страницы)
Системная таблица => находится в адресном пространстве ядра системы
- Если процесс имеет в виртуальной памяти n страниц, то таблица при прямом отображении для этого процесса содержит последовательные строки для страницы 0, страницы 1, страницы 2, ..., страницы $n-1$.
- Такое преобразование выполняется на каждой команде обращения к оперативной памяти, а то и несколько раз.
 - Считать команду
 - Операнд – получить адрес операнда
 - Косвенный адрес – преобразовать адрес операнда
- Обращение к памяти — затратное действие
- Адрес команды, адрес операнда, м.б. косвенный адрес

Ассоциативное отображение

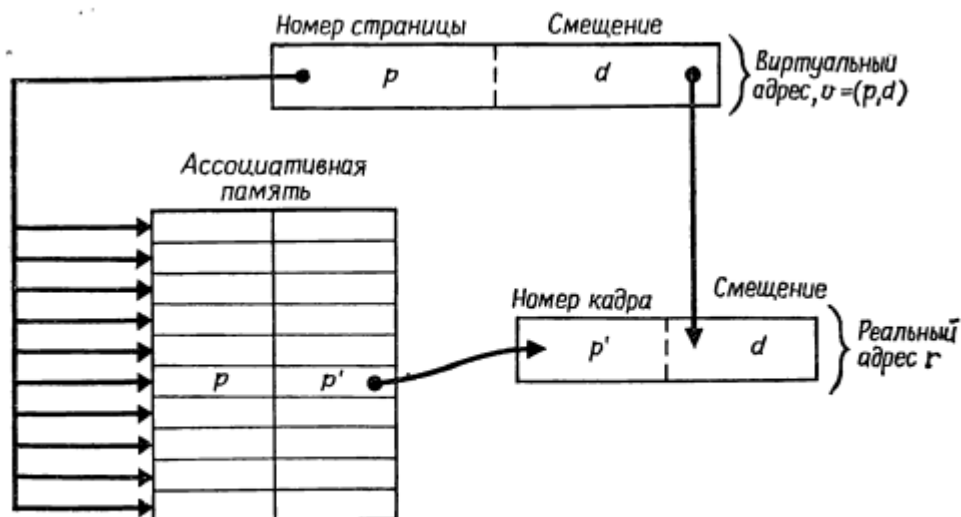
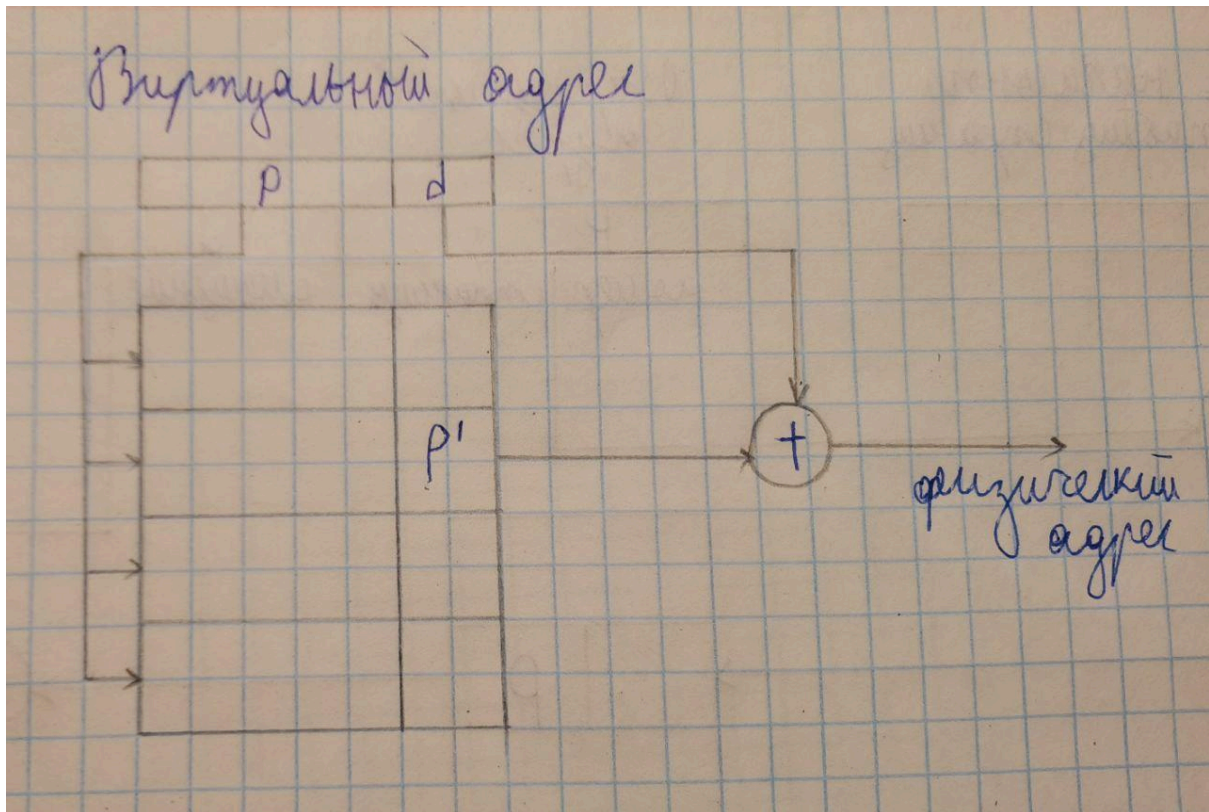
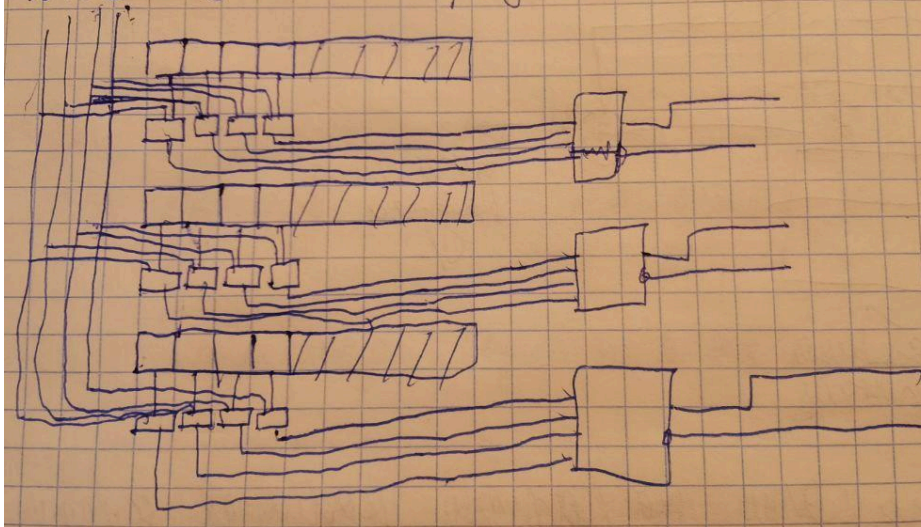


Рис. 8.13 Преобразование адреса страницы при чисто ассоциативном отображении.



- Вся таблица преобразования страниц размещается в ассоциативной памяти (которая работает на порядок быстрее первичной)
- Ключом является номер страницы:
все строки таблицы, хранящейся в ассоциативной памяти, одновременно сравниваются с адресом страницы p .
- Ассоциативная память выдает значение p' как адрес страничного кадра, соответствующего странице p
- Ассоциативная память позволяет сравнивать ключ со своим номером в таблице
- Минусы: стоимость ассоциативной памяти высока
- Что из себя представляет ассоциативная память:

что из себя представляет ассоциативная память



Ассоциативное прямое отображение

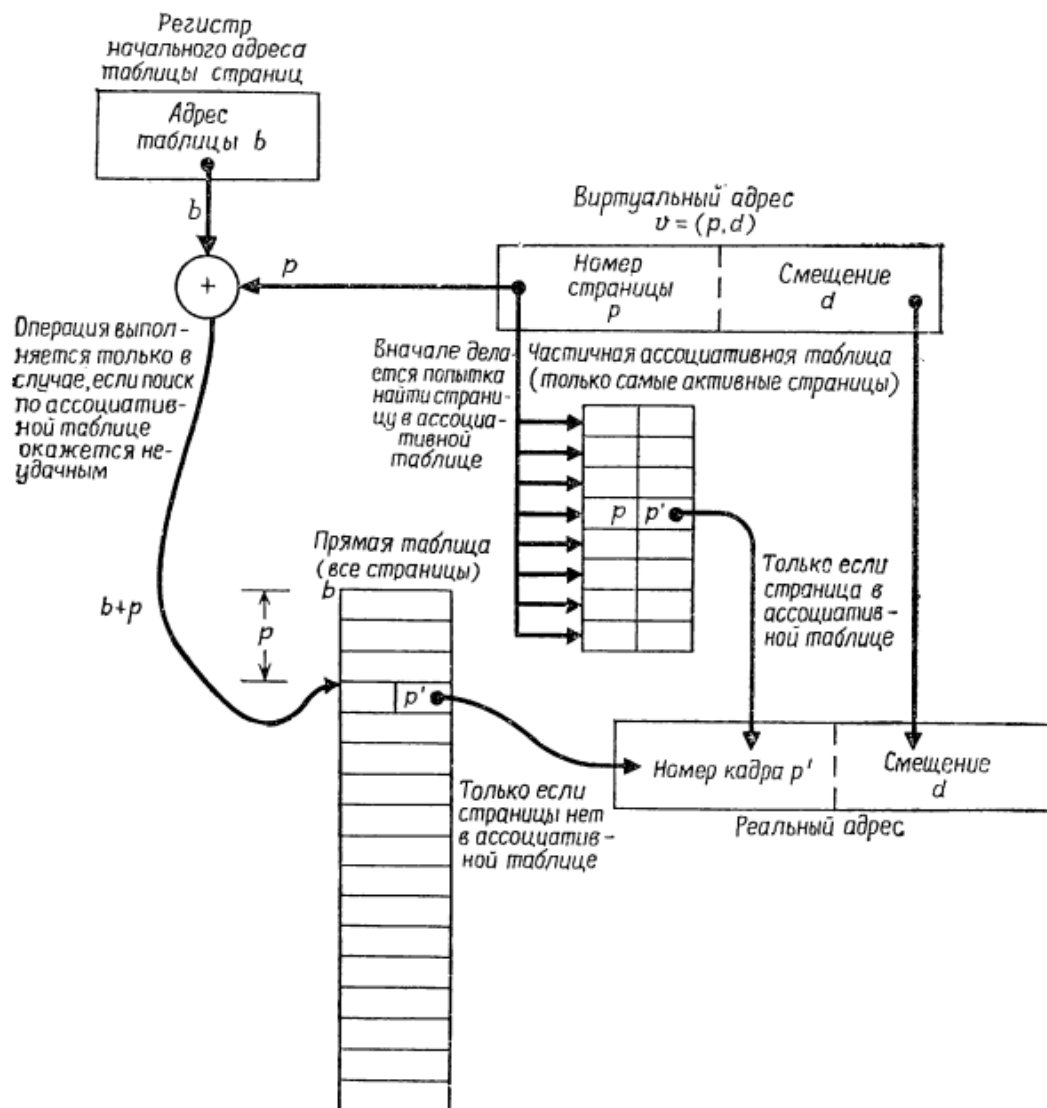
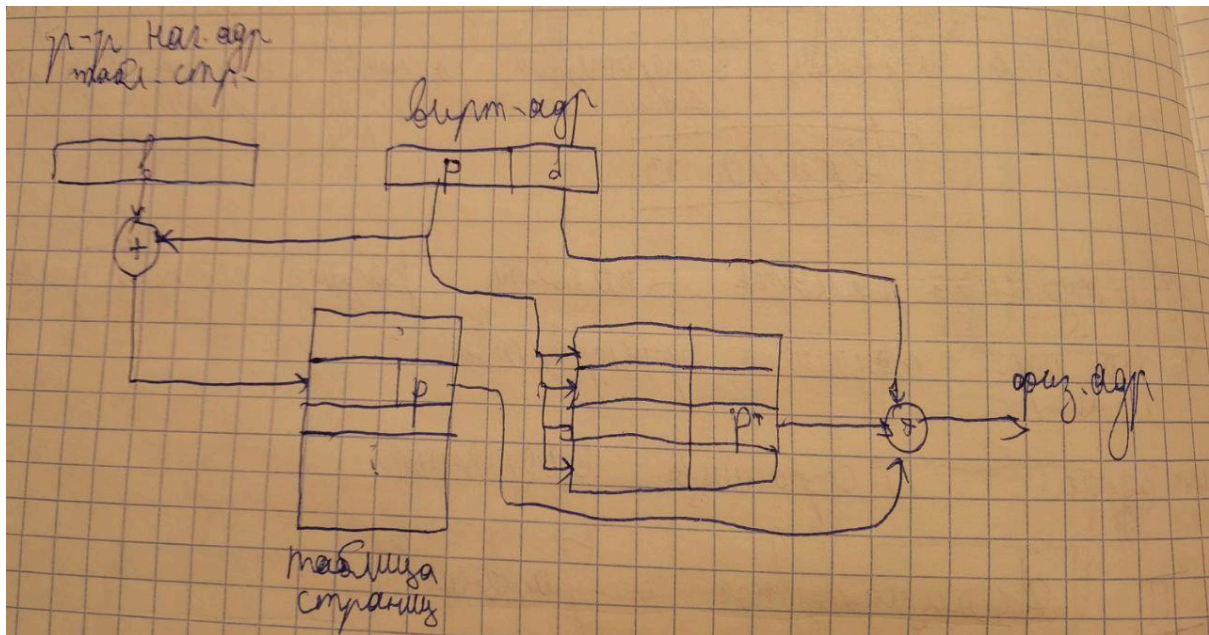


Рис. 8.14 Преобразование адреса страницы при комбинированном ассоциативно-прямом отображении.

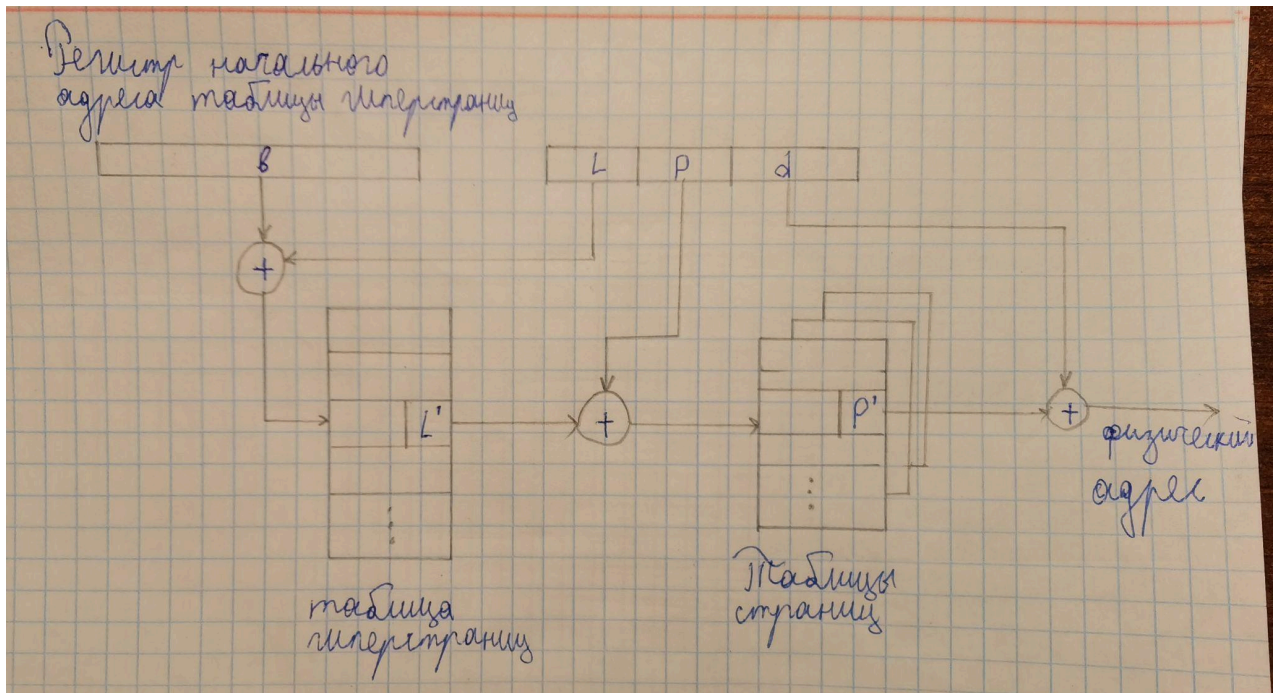


- Идея: в ассоциативной памяти хранится лишь небольшая часть полной таблицы отображения страниц для процесса. В строках этой таблицы отображаются только те страницы, к которым были последние ссылки, — это делается с учетом того эвристического соображения, что к странице, к которой осуществлялось обращение в недалеком прошлом, будет, по всей вероятности, новое обращение в ближайшем будущем.
- Программа обращается по виртуальному адресу (p, d): сначала механизм преобразования адресов выполняет поиск страницы p в ассоциативной памяти (ассоциативном кэше)
- Если страница p найдена в таблице в ассоциативной памяти, то ассоциативная таблица выдает p' как номер кадра, соответствующего виртуальной странице p', $r = p' + d$
- Если не найдено, то в таблице
- Обычно такой ассоциативный буфер удовлетворяет большинству запросов
 - В нем адреса страниц, к которым было последнее обращение
 - TLB буфер

Схема страничного управления памятью с гиперстраницами

- Никогда процесс одновременно не обращается ко всем своим страницам
- Нет необходимости держать в памяти все таблицы страниц

- В памяти должны быть только актуальные таблицы страниц, с которыми сейчас работает процесс
- Поэтому в IBM 370 реализована схема с гиперстраницами



- Таблица гиперстраниц хранится в пространстве ядра системы

Замещение страниц

- Физическая память имеет ограниченный размер
- Возникает ситуация, когда вся физическая память распределена, но процесс запрашивает путем страничного прерывания новую страницу, которую необходимо загрузить в память
- Значит, какую-то страницу придется вытолкнуть на диск

1. Выталкивание случайной страницы

- Выбирается первая попавшаяся страница
Выбирается случайная страница – то есть все страницы, находящиеся в основной памяти могут быть выбраны для выталкивания с равной вероятностью, в том числе даже следующая страница, что наиболее нецелесообразно
- Достоинства:
 - характеризуется малыми накладными расходами

- не является дискриминационной по отношению к каким-либо конкретным пользователям, НО
- Недостатки:
 - может быть вытолкнута часто используемая или только загруженная страница

2. FIFO

- Может быть реализован двумя способами
 - связный список
 - указание временной метки загрузки страницы в память
- Будут вытолкнуты страницы, дольше всего находящиеся в памяти
- Недостаток:
 - может быть вытолкнута часто используемая страница (Стратегия FIFO с достаточно большой вероятностью будет приводить к замещению активно используемых страниц, поскольку тот факт, что страница находится в основной памяти в течение длительного времени, вполне может означать, что она постоянно в работе.)
- Аномалия FIFO:
 - Казалось бы, чем больше фреймов выделяется процессу, тем с меньшим числом прерываний по отсутствию нужной страницы в памяти он будет выполняться.
 - Однако Билейди, Нельсон и Шедлер установили, что в стратегии FIFO определенные последовательности обращений к страницам приводят в действительности к увеличению количества прерываний по отсутствию страницы при увеличении количества страничных кадров, выделяемых процессу.

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4+	3+	5+	5	5	2+	1+	1
M=3		4	3	2	1	4	3	3	3	5	2	2
			④	③	②	①	4	4	④	③	5	5
F		+	+	+	+	+	+			+	+	

Частота страничных переходов $f = \frac{9}{12} = 75\%$

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4+	3+	5+	4+	3+	2+	1+	5+
M=4		4	3	2	2	2	1	5	4	3	2	1
			4	3	3	3	2	4	5	4	3	2
				4	4	④	③	②	①	⑤	④	3
F		+	+	+	+		+	+	+	+	+	+

$f = \frac{10}{12} \approx 83\%$

3. Least recently used (LRU)

- Вытесняются те, к которым дольше всего не обращаются
То есть для вытеснения выбирается та страница, которая не использовалась дольше других.
- В голове списка страниц страницы, к которым дольше всего не было обращения
- Или используются временные метки, обновляемые при каждом обращении к странице

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4+	3+	5+	4	3	2+	1+	5+
$M=3$		4	3	2	1	4	3	5	4	3	2	1
			(4)	(3)	(2)	(1)	4	3	(5)	(4)	(3)	2
F	+	+	+	+	+	+	+			+	+	+

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4	3	5+	4	3	2+	1+	5+
$M=4$		4	3	2	1	4	3	5	4	3	2	1
			4	3	2	1	4	3	5	4	3	2
				4	3	(2)	(1)	(1)	(1)	(5)	(4)	3
F	+	+	+	+			+			+	+	+

- Стековые алгоритмы $\leftarrow$ LRU
- Обладают свойством локальности:
 - Если обращение было к странице, то наиболее вероятно, что следующее обращение будет к этой же странице
 - Следует из того, что в программах наиболее вероятно следование
- Но очень затратный — при каждом обращении редактируется временная метка

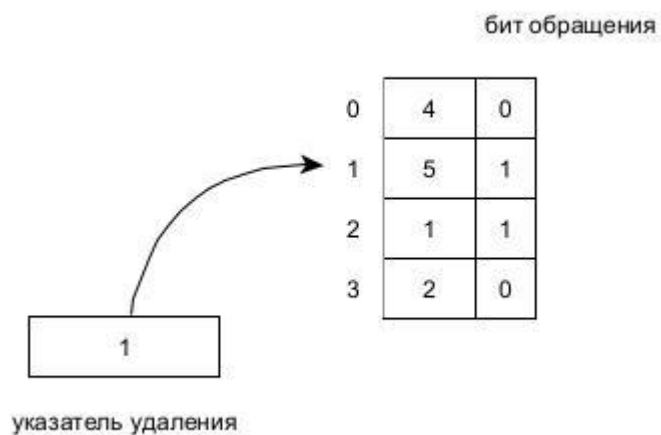
4. Least Frequently Used (LFU)

- Выталкивается наименее часто(наименее интенсивно) использовавшаяся страница
- Для реализации контролируется частота обращения к странице
- Недостаток: может быть вытолкнута только что загруженная страница, так как она еще не набрала обращений

5. Not Used Recently (NUR)

Выталкивание не использовавшейся в последнее время страницы

- К страницам, которые в последнее время не использовались, вряд ли будут обращения и в ближайшем будущем, так что их можно заменить на вновь поступающие страницы
- Желательно заменять ту страницу, которая в период нахождения в памяти не изменялась(почему....?) => вводятся два аппаратных бита-признака на страницу.
- Для каждой страницы записывается бит обращения



- Изначально биты-признаки обращения и модификации для всех страниц устанавливаются в 0.
- Бит обращения устанавливается при обращении к странице
- Страница для выгрузки в первую очередь ищется среди страниц, у которых бит обращения 0
- Если таких страниц нет, то ничего не остается, кроме как выталкивать страницу, к которой были обращения
- Память работает активно, и рано или поздно у большинства страниц признак обращения будет установлен, и мы не сможем отличать те страницы, которые наиболее целесообразно вытолкнуть
- => биты-признаки обращений периодически сбрасываются
- Если к странице были обращения, то проверяется, подвергалась ли она изменениям: бит модификации
- Если страница не модифицировалась, то выталкиваем ее, поскольку выгоднее выталкивать немодифицированные страницы, так как исключается копирование (на диске есть ее точная копия)

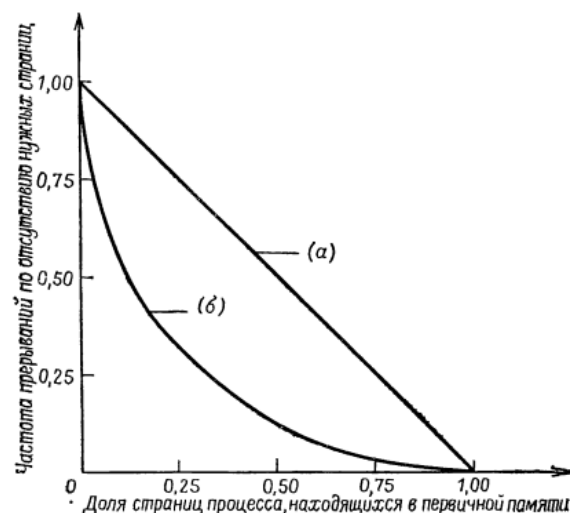
Описанный алгоритм предусматривает существование четырех групп страниц:

Группа	Бит обращения	Бит модификации
1	0	0
2	0	1
3	1	0
4	1	1

- Страницы группы с меньшими номерами следует выталкивать в первую очередь, а с большими - в последнюю.
- (Пояснение ко 2 строке) — модификация произведена до сбросов битов обращения
- Если нет соответствующей страницы в кэше при схеме с гиперстраницами, будет 2 обращения вместо 1
- Для реализации выталкивания большие затраты
- Виртуальная память требует больше затрат (4 уровня таблиц)
- От виртуальной памяти не отказались для увеличения мультизадачности (чтобы в памяти было большое число программ)

Теория рабочего множества

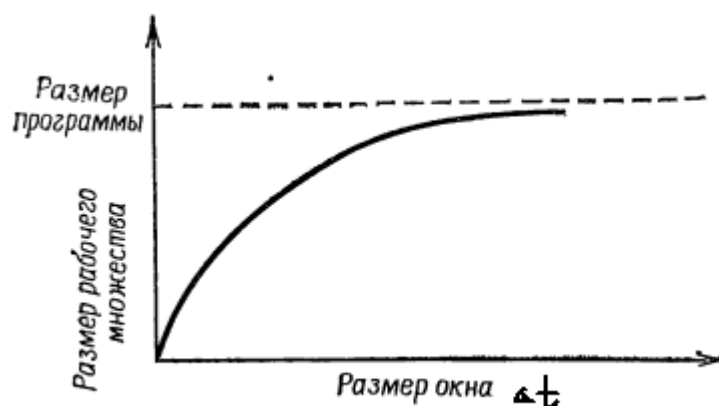
- Рассмотрим зависимость числа страничных прерываний от объема физической памяти



- Является мерой обобщенного поведения программ
- Неформально: рабочее множество – подмножество страниц, к которым процесс активно обращается. Для эффективного выполнения программы необходимо, чтобы ее рабочее множество находилось в первичной памяти
- В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за время Δt
- То есть, рабочее множество страниц процесса $W(t, \Delta t)$ в момент времени $t + \Delta t$ — набор страниц, к которым этот процесс обращается в течение интервала времени процесса Δt .
Время процесса – время, в течение которого процесс имеет в своем распоряжении центральный процессор.



- Чем больше Δt , тем к большему числу страниц обращается процесс
- При увеличении Δt размер рабочего множества стремится к пределу



- Программа должна иметь возможность загрузить в память рабочее множество страниц
 - То множество страниц, к которому в текущий интервал времени обращается процесс
- Если программа не имеет такой возможности, то пробуксовка (trashing)
 - Пробуксовка — загрузка и выгрузка одних и тех же страниц
- (Совершенная ОС стремится использовать все процессорное время на себя саму, а на приложения времени не остается – нужен компромисс)
- Page — виртуальная страница
- Frame — физическая страница

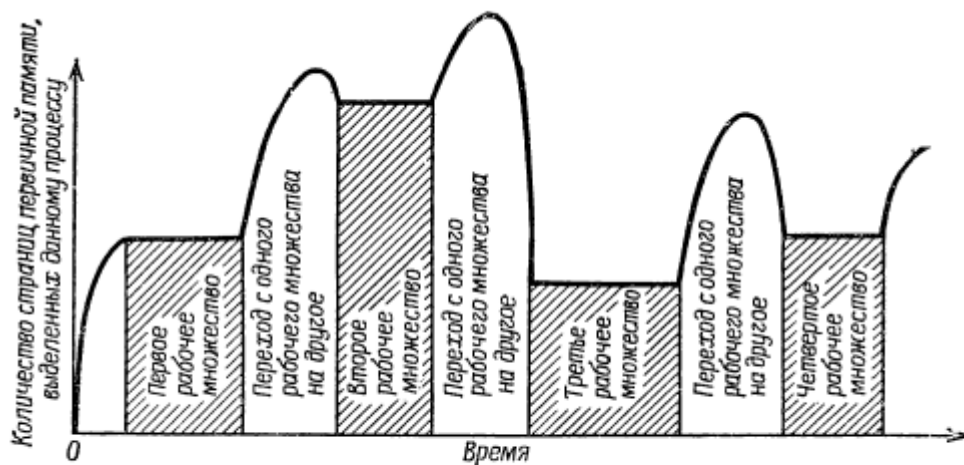


Рис. 9.6 Выделение основной памяти при управлении памятью на основе рабочих множеств.

1. Вначале, поскольку процесс запрашивает страницы своего рабочего множества не все сразу, а по одной, он постепенно получается достаточный объем памяти для размещения текущего рабочего множества.
2. Затем на какой-то период времени этот объем стабилизируется, поскольку процесс интенсивно обращается к страницам своего первого рабочего множества.
3. Со временем процесс перейдет на следующее рабочее множество, что показывает кривая линия от 1 ко 2 раб. множеству. Вначале эта кривая поднимается выше уровня первого рабочего набора, поскольку для процесса по его запросам быстро производится подкачка страниц нового рабочего множества.

При этом система не может сразу определить, то ли процесс просто расширяет свое рабочее множество, то ли процесс переходит на новое.

4. После того, как процесс начнет стабильно обращаться к страницам своего следующего рабочего множества, система видит, что в течение выбранного окна происходит обращение к меньшему количеству страниц, чем ранее, и сокращает объем памяти, выделяемой процессу, до числа страниц этого второго рабочего множества.
5. Каждый раз, когда осуществляется переход с одного рабочего множества на другое, кривая линия поднимается, а затем спадает, показывая, каким образом система адаптируется к подобному переходу.

Проблема: рабочие множества меняются во времени, причем следующее рабочее множество процесса может существенно отличаться от предыдущего. Стратегия управления памятью должна обязательно учитывать этот факт, чтобы избежать перегрузок первичной памяти и возникающего вследствие этого трешинга.

График зависимости длительности периода между прерываниями (кривая линии жизни)

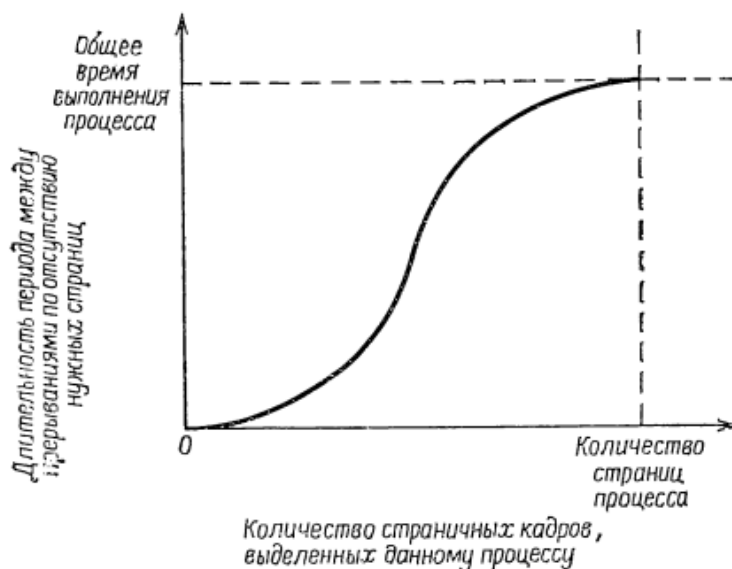
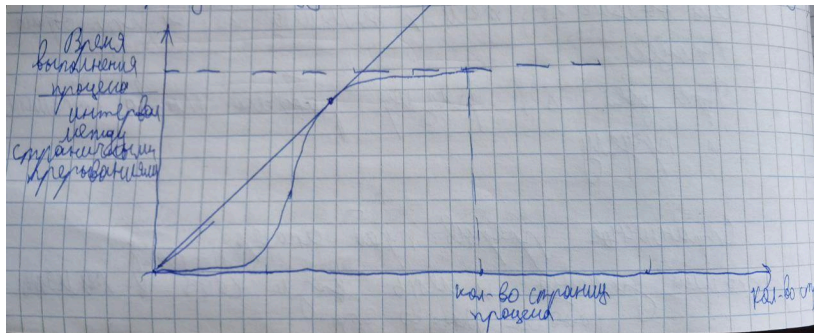


Рис. 9.12

- На графике показано, каким образом меняется среднее время между прерываниями по отсутствию нужной страницы в памяти при увеличении числа фреймов, выделяемых процессу.
- Кривая монотонно растет – чем больше фреймов имеет процесс, тем больше интервал между прерываниями по отсутствию нужных страниц.
- Перегиб происходит, так как в памяти оказывается все рабочее множество процесса
- Поначалу величина интервала между прерываниями растет очень быстро, поскольку растет часть рабочего множества, которая оказывается в первичной памяти.
- После того, как выделенный объем первичной памяти оказывается достаточным для размещения всего рабочего множества, кривая делает перегиб, указывая на то, что выделение дополнительных

фреймов не приведет к существенному увеличению интервала между прерываниями.

Влияние размера страниц при одном и том же размере физической памяти

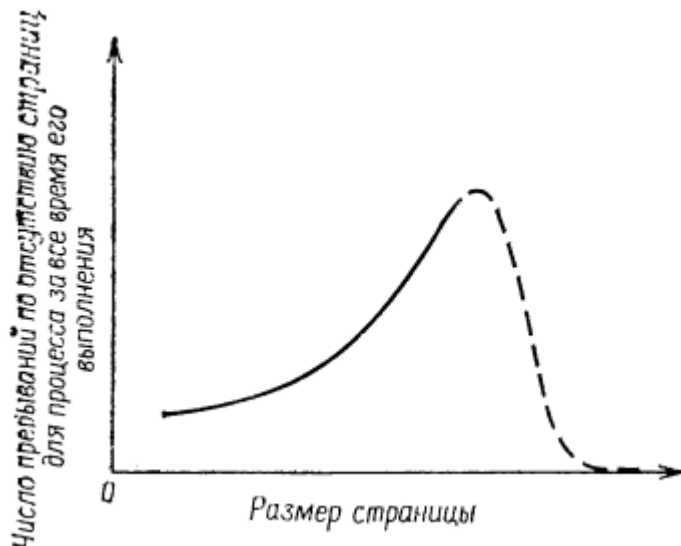
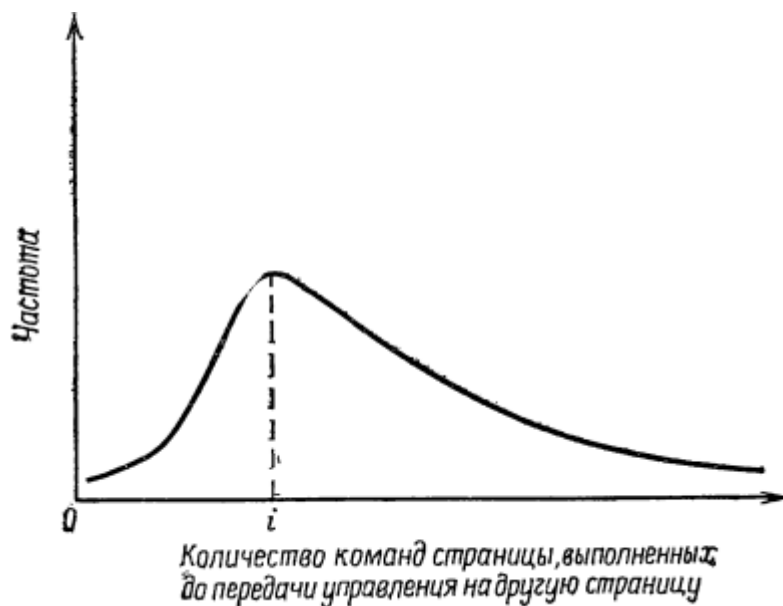


График показывает влияние размера страниц на число страничных прерываний

- С ростом размера страниц сначала число прерываний растет, так как в память попадет большее число процедур и данных, к которым не будет обращений. Так как никогда не выполняются все команды на странице, всегда выполняется только какой-то процент.
- Чем больше страница, тем меньше будет выполнено команд
- Падает, так как размер страницы стремится к размеру программы

Зависимость частоты обращения к странице от количества команд на странице, выполненных до передачи управления на другую страницу



- На графике показано, какой процент команд данной страницы выполняется до того, как управление будет передано другой странице.
- Максимум указанной кривой пришелся на точку, соответствующую 200 командам страницы размеров 4096Б.
- По-видимому нужно уменьшать размер страниц – Дейтелл
- Рабочее множество от Windows называется квота
- Так как предсказать размер рабочего множества невозможно, то каждому процессу перед началом выполнения выдается квота — число страниц, которые процесс может запросить в ОЗУ.
 - Вопрос: сколько страниц выдавать?
- Производится контроль за числом страничных прерываний
 - Если идет рост страничных прерываний, то система работает неэффективно, так как страничные прерывания — затратные действия => квота для данного процесса мала
 - При малейшей возможности квота увеличивается, чтобы процесс мог их загрузить в память, то есть загрузить все рабочее множество.

Управление сегментами по запросу

- Было реализована в первых UNIX
- Связано было с тем, что на уровне страниц теряется принадлежность страниц к процессу и возникает проблема.
- Проблема коллективного использования страниц — несколько процессов заинтересованы в одних и тех же страницах
 - Одни в исходном содержимом
 - Другие в редактировании
- =>дополнительные расходы: приходилось указывать, что страница разделяемая и выдавать права доступа процессам. Нужно тщательно выполнять контроль.

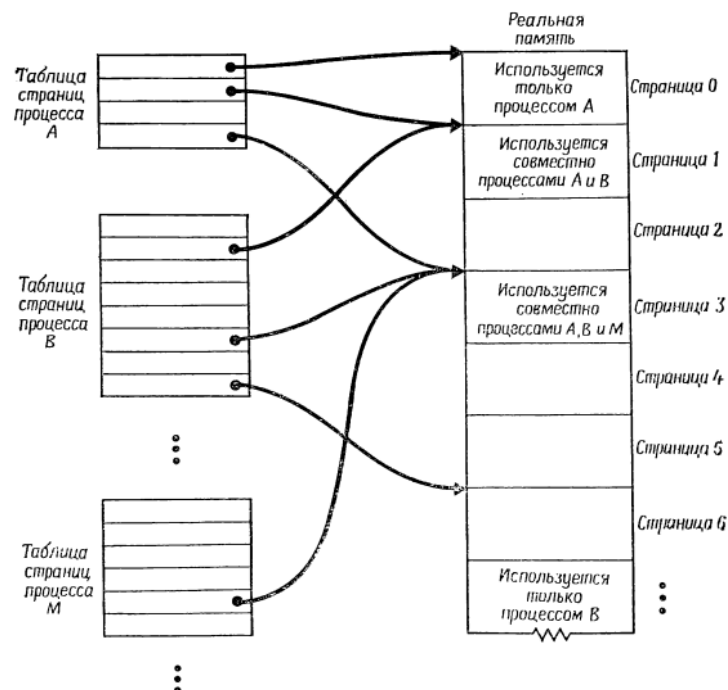
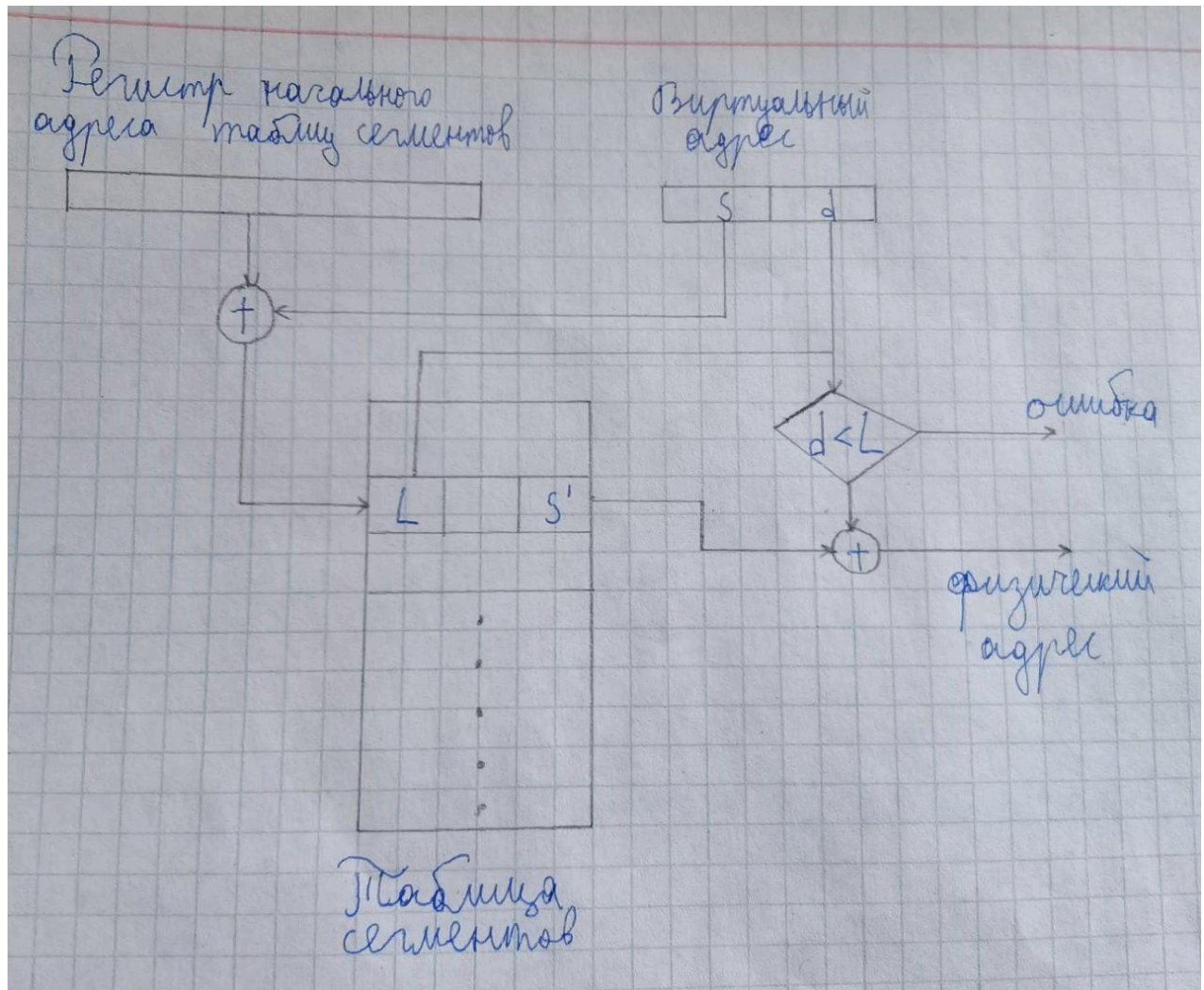


Рис. 8.15 Коллективное использование программ и данных в чисто страничной системе.

- Несмотря на то, что при управлении памятью страницами по запросу, страница меняется на страницу, так как страница является физической единицей, то есть при замене страницы проблем не возникает, но проблемы с коллективным использованием делают более привлекательной схему управления сегментами по запросу.

- Сегмент является единицей логического деления памяти, размер определяется размером программного кода
- Коллективное использование сегментов логично.
Данные сегмента разделяются несколькими процессами, так как в этом коде заинтересованы разные процессы.(?)



- ОС должна иметь информацию о размере сегмента, чтобы контролировать его выход за собственное адресное пространство.
- В таблице сегментов - дескриптор сегментов.
Дескриптор содержит размер, флаги, адрес сегмента
- Размер сегмента в любой системе ограничен
- При управлении сегментами по запросу преобразования выполнялись по дескрипторам
 - S — номер сегмента
 - S' — начальный адрес сегмента
 - L — размер сегмента

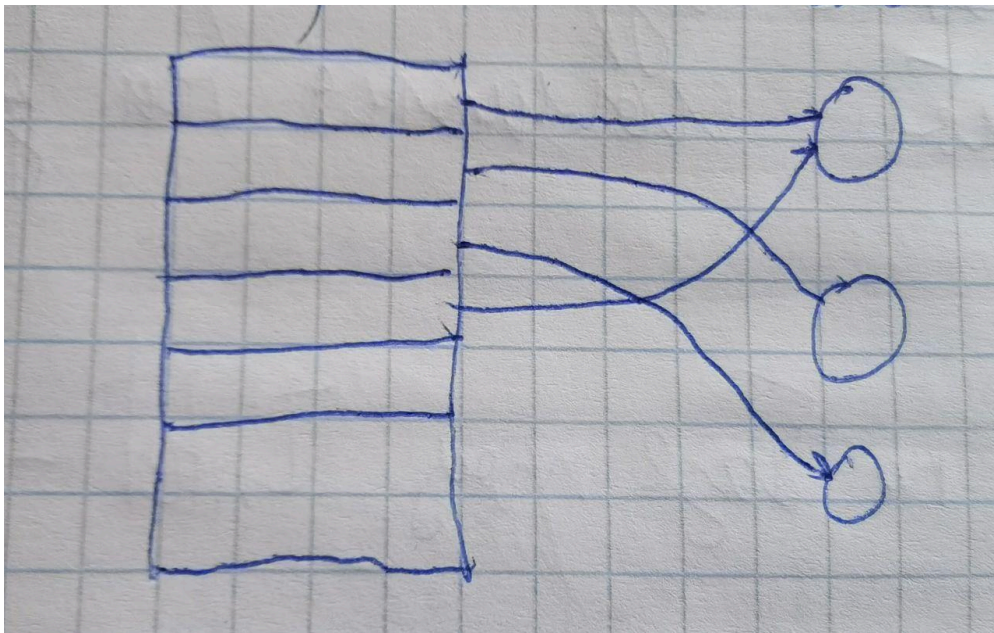
- Смещение 20 разрядов
- Если $D \geq L$ – то прерывание по выходу за пределы сегмента, иначе получаем физический адрес
- В режиме long сегментация не используется вообще, но глобальная таблица осталась
- (В сегментные 16-разрядные регистры в защищенном режиме записываются смещения к дескриптору)

3 способа организации таблиц сегментов

1) Единая таблица

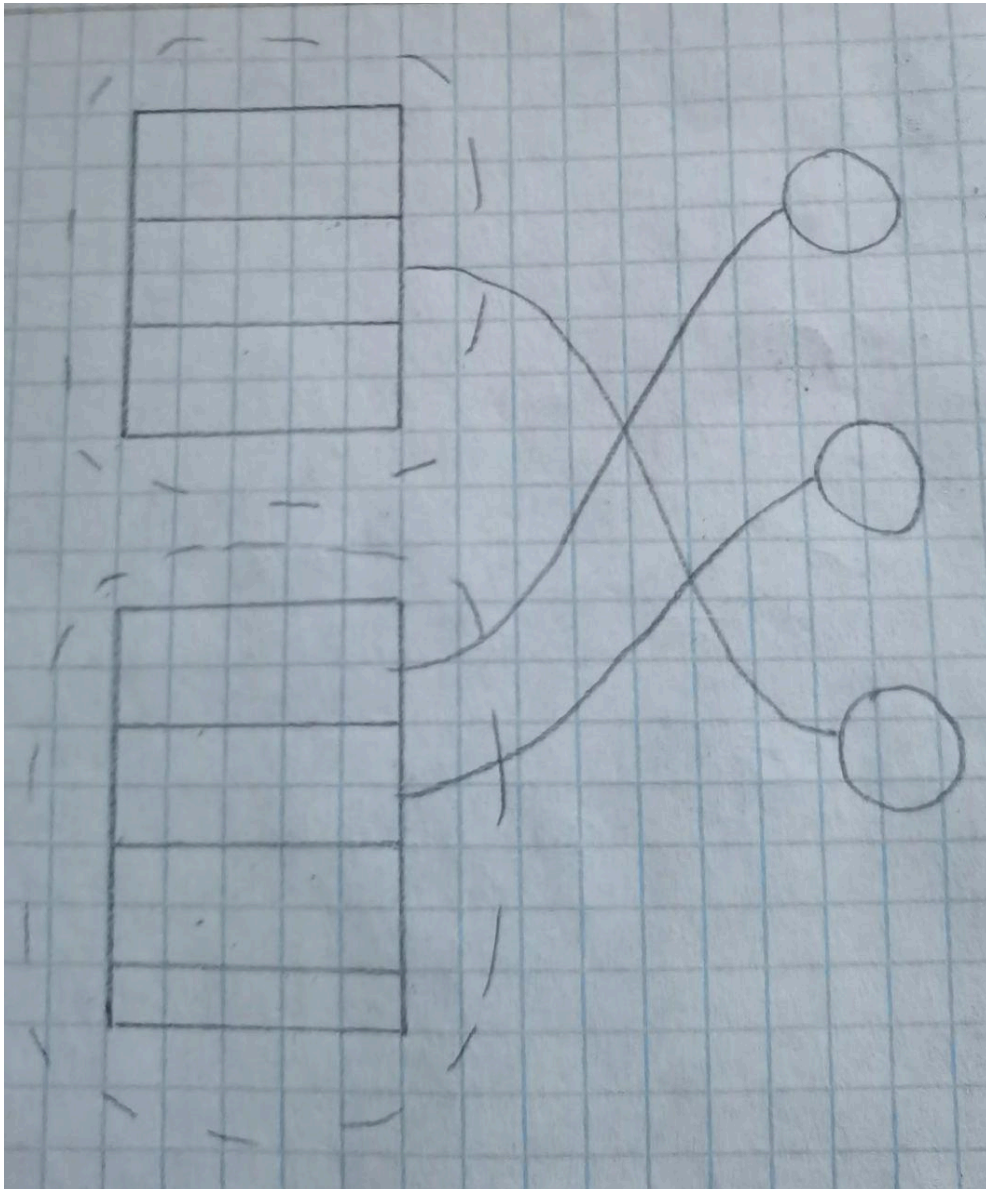
- a) Существует одна единая таблица сегментов, описывающая сегменты всех процессов
- b) Все сегменты имеют единственное имя, которое называется глобальным

=> глобальная таблица дескрипторов



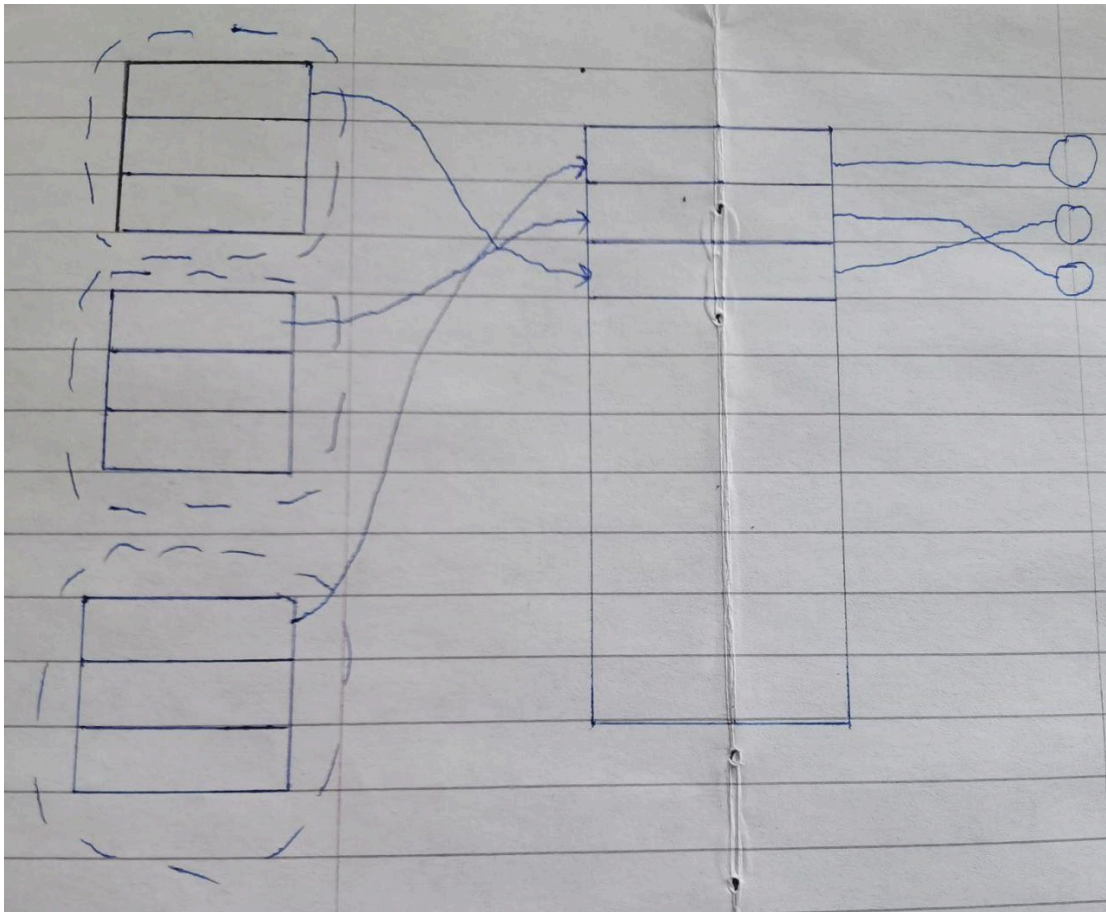
- c) Единая таблица содержит все дескрипторы сегментов
- d) При этом единственное имя сегмента является индексом его дескриптора в таблице
- e) Дескриптор должен содержать список прав доступа всех процессов, работающих с данным сегментом.
- f) Данный тип управления памятью не удобен при совместном использовании сегментов, так как все процессы, работающие с данным сегментом, должны использовать его единственное имя, то есть должны его “знать”.

2) Локальные таблицы



- а) Локальная таблица описывает адресное пространство процесса
- б) Каждый процесс имеет собственную локальную таблицу.
- с) Сегмент физической памяти, доступный разным процессам, должен иметь в каждой локальной таблице дескриптор (то есть будет иметь несколько имен?)

3) Локальные таблицы + глобальная



- a) Есть 1 глобальная таблица, описывающая все сегменты физической памяти
- b) Есть локальные таблицы, описывающие адресное пространство отдельных процессов
- c) Дескрипторы локальных таблиц ссылаются на дескрипторы глобальной таблицы
- d) Данная схема в защищенном 32 битном режиме на база процессора Intel

Проблемы с управлением сегментами по запросу

- 1) Сегменты – единицы логического деления, имеют разные размеры. При замещении сегмента ОС должна выбрать сегмент для замещения. Используются те же алгоритмы замещения, как и для страниц – LRU — вытесняются те, к которым давно не было обращений, по свойству локальности.
- 2) Если есть свободный сегмент — нужно выбрать, в какой свободный сегмент загрузить сегмент. Стратегии выбора раздела те же: первый подходящий, самый тесный, самый широкий.

3) При swapping

Нашли давно не используемый сегмент. Может оказаться, что он недостаточного размера для загрузки нашего сегмента => придется выгрузить еще один сегмент — в этом огромная проблема.

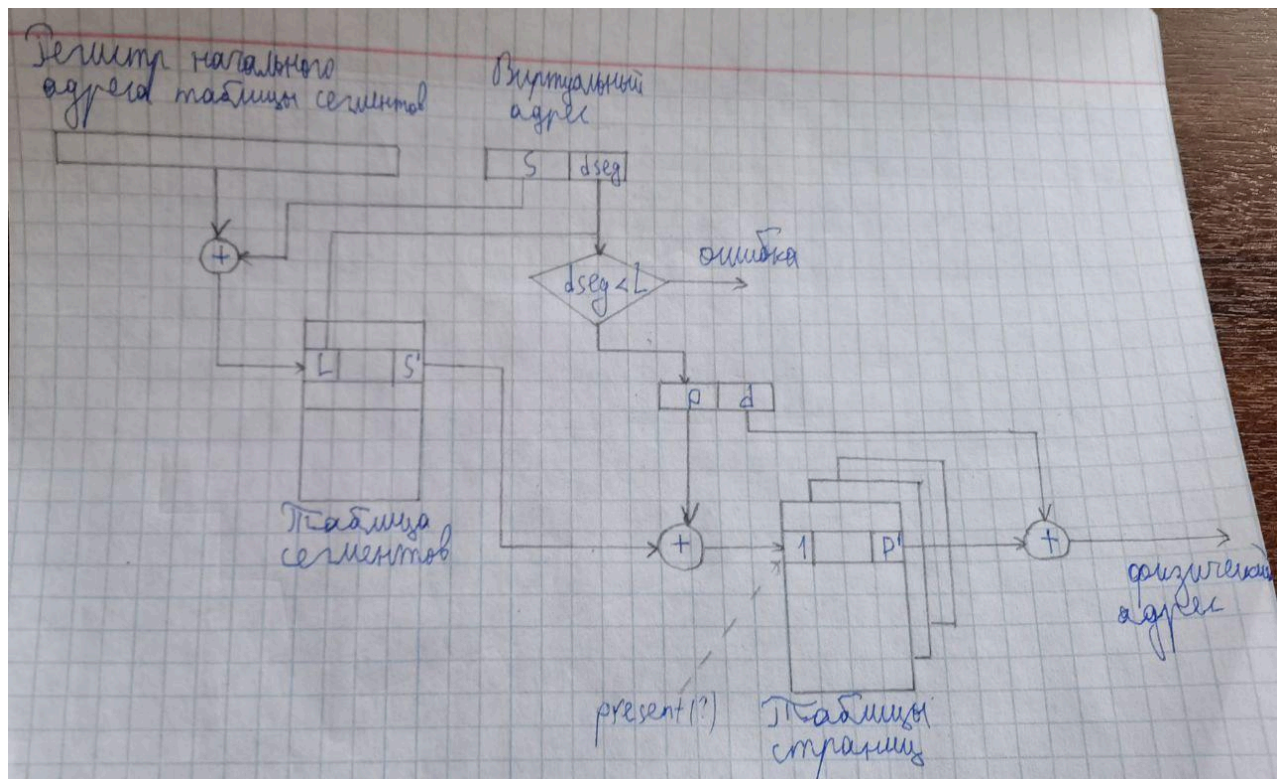
Также возникает фрагментация которая требует передвижений сегментов — дополнительные расходы.

(В страницах существует скрытая фрагментация — последняя страница заполнена не полностью — статистически на $\frac{1}{3}$. Но при использовании(?) страниц на это не обращают внимания)

Таким образом, swapping сегментов затруднен из-за разных размеров.

Управление сегментами, разделенными на страницы, по запросу

- Попытка объединения преимуществ двух подходов(страницами по запросу и сегментами по запросу)
- Каждый процесс имеет одну таблицу сегментов и много таблиц страниц
- При этом размер сегмента должен быть кратен размеру страницы
- Сегментно-страничная организация была реализована в системе Multics



- Обозначения:
 - s – номер сегмента
 - p – номер страницы
 - d – смещение в рамках страницы, по которому находится нужный элемент
- Логика получения адреса:
 - $b+s$ – получаем адрес строки для сегмента s в таблице сегментов первичной памяти. В ней указан адрес базовый адрес s' таблицы страниц для сегмента s
 - $p+s'$ – адрес строки в таблице страниц для страницы p сегмента s . Строка содержит номер фрейма p' .
 - $p'+d$ – реальный адрес
- Просмотр таблицы сегментов может показать, что сегмента s в первичной памяти нет — прерывание по отсутствию сегмента. ОС найдет сегмент во внешней памяти, сформирует для него таблицу страниц и загрузит соответствующую страницу в первичную память. (Возможно вместо какой-то существующей)
- Когда сегмент находится в первичной памяти, обращение к таблице страниц может показать, что нужной страницы в первичной памяти нет — прерывание по отсутствию страницы. ОС найдет страницу во внешней памяти и загрузит ее в первичную память.
- Адрес виртуальной памяти может выйти за рамки сегмента и произойдет прерываний по выходу за пределы сегмента.
- Системные таблицы

Размеры приложений достаточно большие

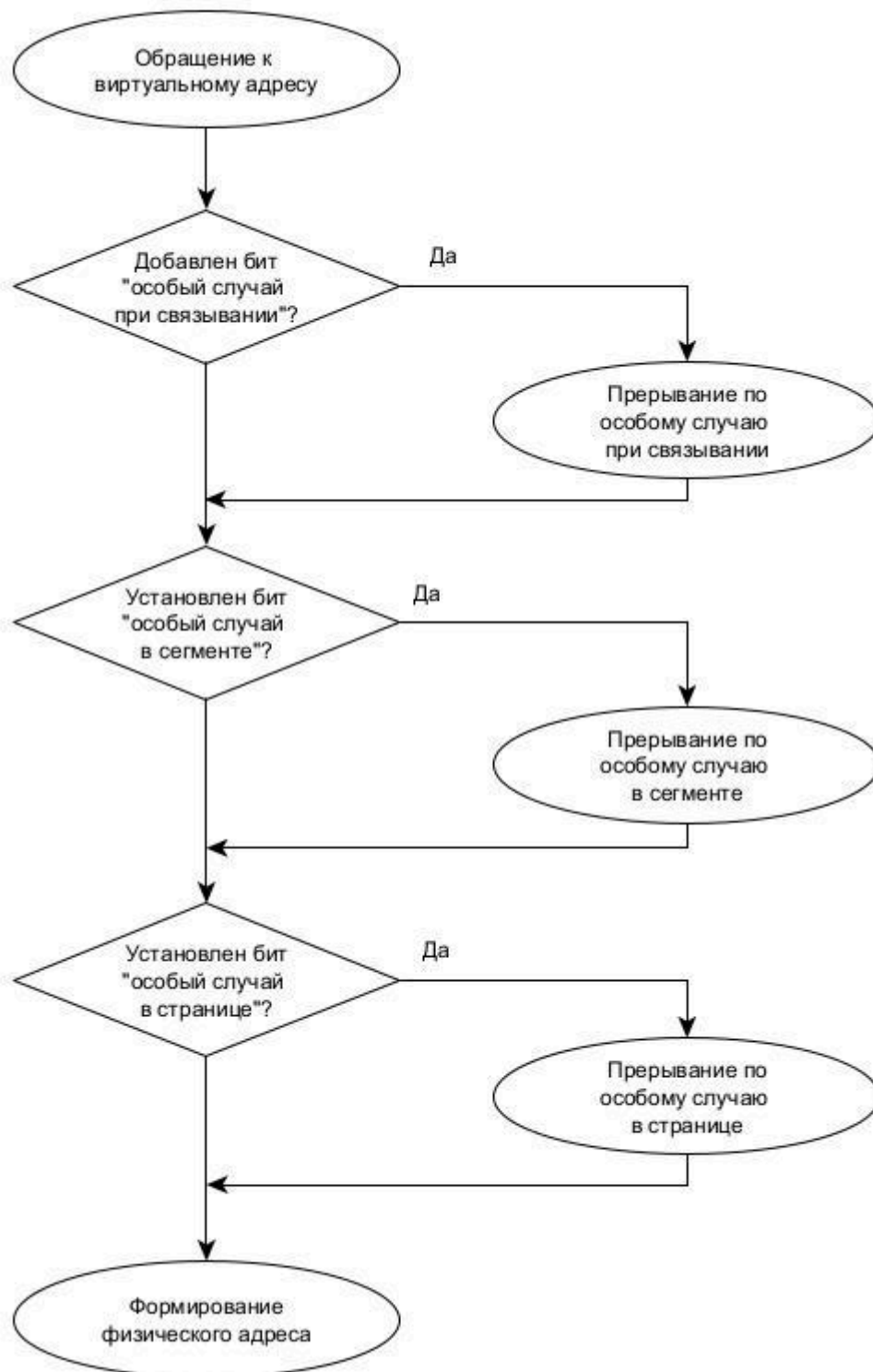
В результате таблица сегментов могла быть большой и сама занимать несколько страниц физической памяти.

При этом таблица сегментов процесса должна находиться в физической памяти.

Если сегмент большого размера, то он мог описываться несколькими таблицами страниц.

И если к сегменту осуществлялось обращение, то страницы тоже должны были находиться в физической памяти.

Вопрос динамического связывания и вопрос отложенного связывания



- Система делает только необходимое в текущий момент действие. Если к сегменту не было обращений, то ему не создается дескриптор

— отложенное действие. Заранее ОС ничего не делает, ведь может оказаться, что обращений к данному сегменту(странице) вообще не будет.

- 1) Особый случай при связывании: когда к сегменту выполняется обращение и оказывается, что нет дескриптора данного сегмента, создается дескриптор
Только при первом обращении!
- 2) Особый случай в сегменте:
сегмент имеет дескриптор, но для данного дескриптора не созданы таблицы страниц
 - а) Создается таблица страниц
 - б) Адрес созданной таблицы страниц будет записан в дескриптор сегмента
(на схеме в дескрипторе сегмента находится начальный адрес таблицы страниц)
- 3) Особый случай в странице:
страница не находится в физической памяти
 - а) должны быть выполнены все необходимые действия по загрузке страницы в физическую память — надо найти свободный фрейм, если нет — вытеснять

Возникают:

- 1) Стр.
- 2) Сегм.
- 3) Деск.

Обрабатываются в обратном порядке.

Огромные затраты!

Поздно выгружать страницы из памяти, когда нужно загружать новые. Это делается заранее. Демон страниц — вызывается по таймеру и анализирует таблицы страниц, чтобы найти те, к которым не было обращений.

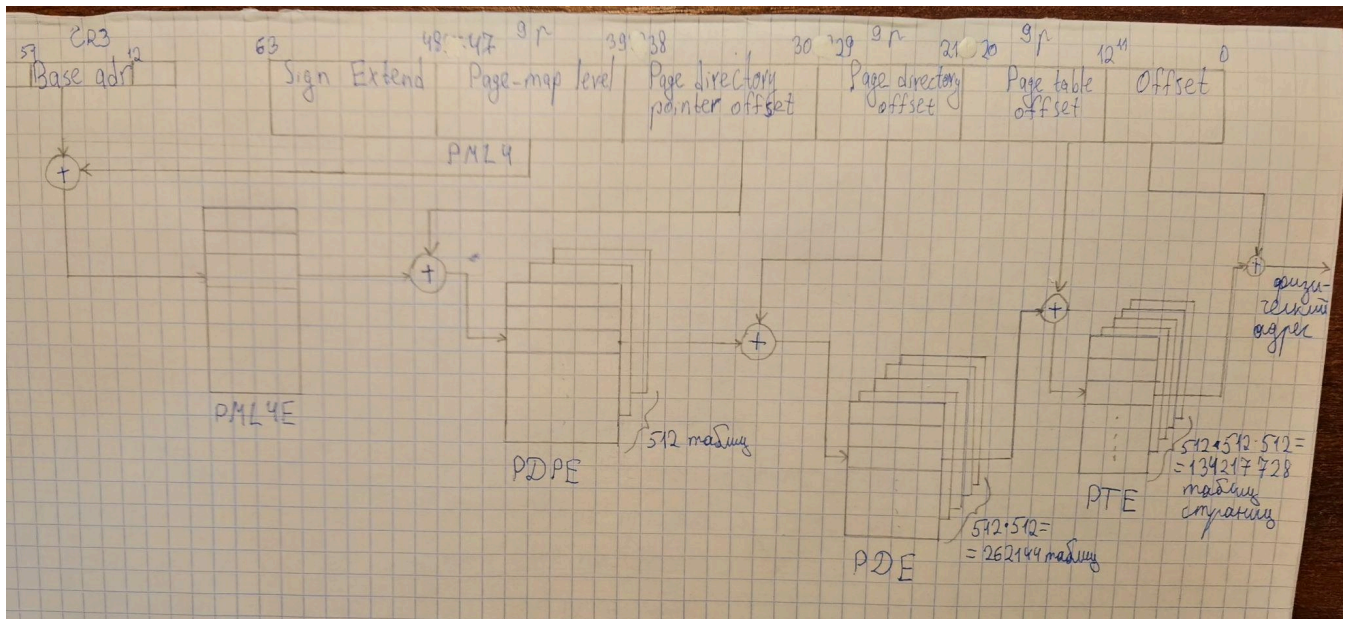
Это делается, чтобы сократить время обработки страничного прерывания.

- В современных системах страничное управление, т.к. решили вопрос коллективного использования страницы с помощью флага copy-on-write

Совместно использование страницы — изменяются права на ONLY READ и устанавливается флаг COPY ON WRITE.

- В режим long реализовано только управление памятью страницами по запросу.
- В режиме long — схема PAE (physical address extension)

PAE



- PTE — таблицы страниц
- PDE — каталоги таблиц страниц —
- PDPE — указатель на каталоги таблиц страниц — максимум 512 табл.
- PML4E — указатель на указатель — такая таблица в системе 1
- Дескрипторы инициализируются по мере надобности
- Размер дескриптора — 8 байт
- Таблицы по 2^9 строк $\Rightarrow$ каждая таблица в памяти занимает одну страницу
- Когда процесс создается — ему выделяется минимально необходимое число страниц.
- То есть выполняется отложено.

Например, при создании процесса создается 3 страницы: точка входа, страница сегмента данных, страница сегмента стека.

Все остальные страницы создаются по мере надобности при возникновении системных прерываний.

- Наши машины могут работать в следующих режимах в связи с обратной совместимостью:
 - реальный режим — работает как компьютер на базе 8086 DOS 6.11, 16 разрядные регистры, 20 разрядная шина данных
 - Защищенный
 - 32 разрядный, регистры 32х разрядные, 32 разрядная шина данных.

Многооперационный режим с поддержкой управления памятью:

- сегментами по запросу
- страницами по запросу (но отключить сегментное преобразование нельзя, используется плоская модель памяти. Каждому процессу создается адресное пространство и оно делится на страницы).

Тут впервые появилась схема PAE, но можно было использовать и дескрипторы для таблиц 4 уровня(?)

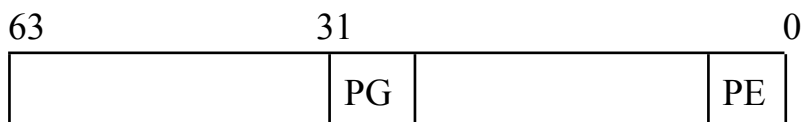
Стандартный защищенный режим: 4гб памяти, обычная двухуровневая схема преобразований: pte, pde

- long
 - 64 разрядный режим.

В этом режиме поддерживается режим обратной совместимости – можно выполнять 32 разрядные. Но нельзя выполнять 16 разрядные. В 64 разрядном режиме поддерживаются только аппаратные преобразования. Части регистров: 8, 16, 32 — аппаратно доступны => будут выполняться команды соответствующей разрядности.

В процессорах Intel начиная с pentium 5 управляющих регистров.

1. CR0 – режим слова состояния



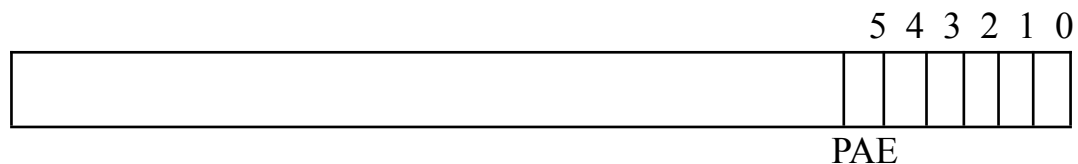
- a. PG – paging enable – страничное преобразование
- b. PE – protected enable
- c. PG может быть установлен, если установлен PE
2. CR1 — не используется
3. CR2

- a. Линейный адрес страничной ошибки — адрес, на котором возникло страничное прерывание
- b. Он нужен, так как в результате страничного преобразования система должна выполнить большой объем действий, вызовов....

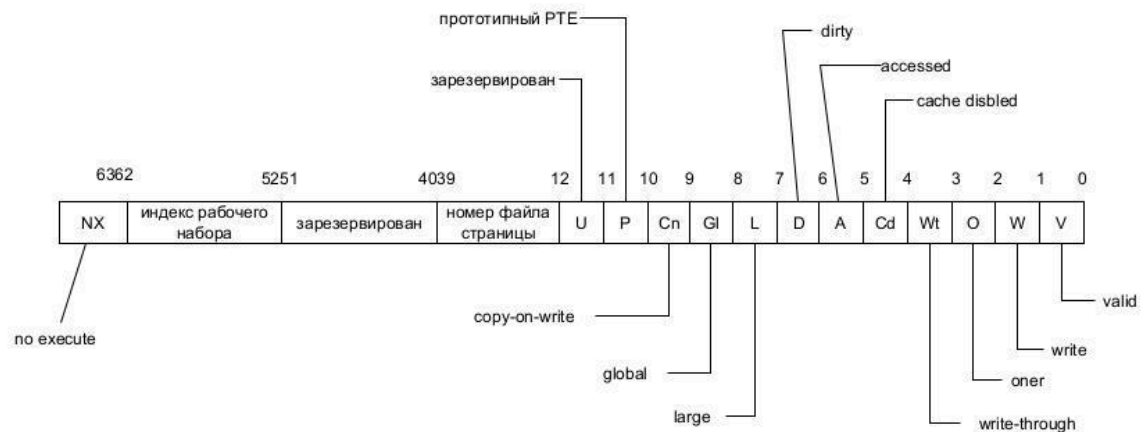
4. CR3

- a. В 64-разрядном режиме в CR3 записывается начальный адрес таблицы 4 уровня.
- b. Изменение регистра CR3 связано с переключением полного контекста (с переходом на новые таблицы страниц)
- c. В общей теории: CR3 — базовый адрес. Конкретный уровень таблицы определен разрядностью.
В long это 4 уровень.

5. CR4



- Дескриптор PTE — дескриптор в таблице страниц
Первые 12 разрядов не участвуют в адресации => отводятся для флагов



- Аппаратное ограничение — не все биты решили использоваться для аппаратной адресации
- Access — аппроксимация LRU
Время от времени сбрасывается.
Устанавливается при загрузке страницы и при каждом обращении.

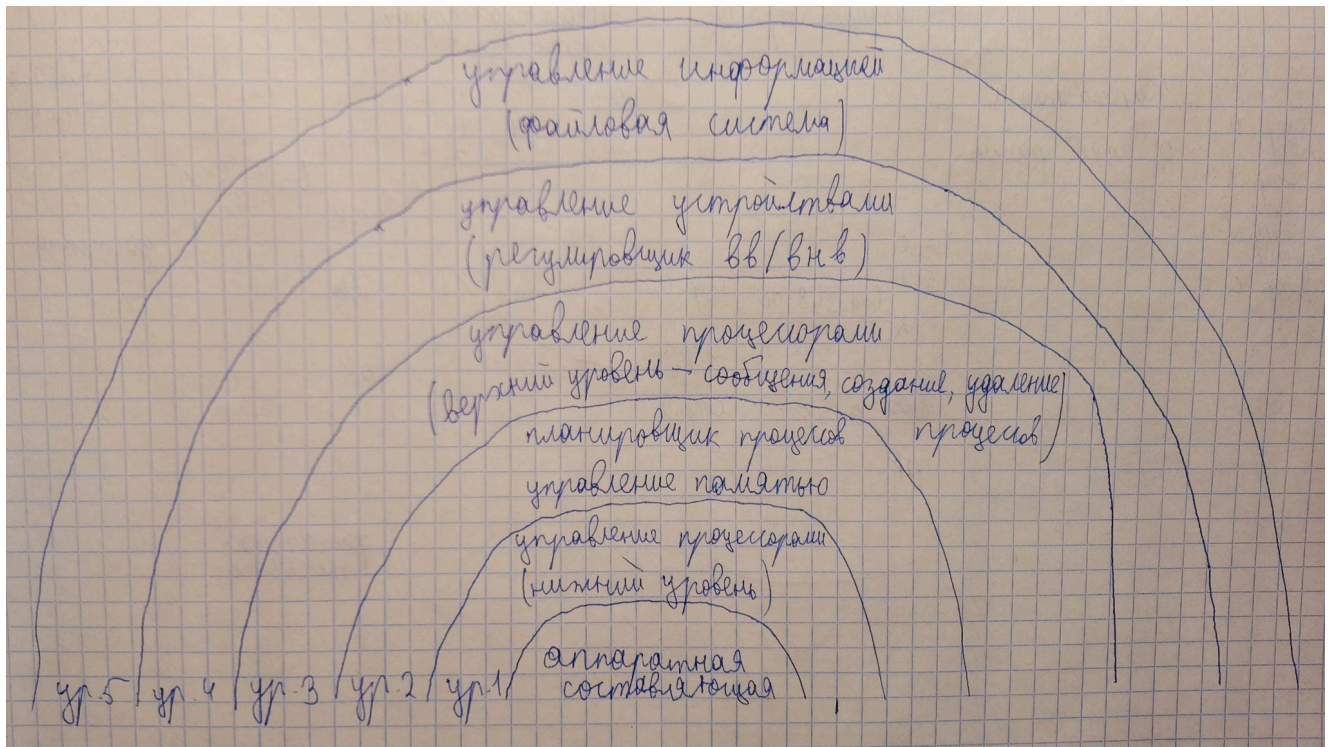
- Dirty — устанавливается, когда страница была изменена (Обращение с целью записи в страницу приведет к установке)
- COPY ON WRITE — работа программного обеспечения. Когда страница определяется как разделяемая, для устанавливается этот флаг и права доступа только на чтение.
- При обращении к странице будет страничное исключение и флаг COW заставит менеджер памяти создать копию страницы в адресном пространстве процесса, который ее хотел изменить
- Менеджер памяти должен создать копию страницы — должен создать дескриптор страницы. После этого система может работать с этой страницей, выполнять действия над этой страницей.
- Для создания страницы может оказаться необходимым создать дескриптор таблицы страниц.

Структура ядра ОС

- Две базовые архитектуры ядер:
 - монолитное ядро
 - программа, имеющая модульную структуру, то есть состоящая из подпрограмм
 - каждое изменение ядра требует перекомпиляции
 - микроядерная архитектура
 - реализовано микроядро, в которое включены только самые низкоуровневые действия, выполняемые ядром
 - остальные компоненты реализованы в виде отдельных модулей, которые могут выполняться в режиме пользователя
- UNIX Linux — системы с монолитным ядром
Unix/Linux имеют минимизированное ядро. То есть какие-то действия вынесены из ядра.
В Windows же, например, GUI входит в ядро, а в UNIX/LINUX из ядра вынесено.
- Изначально все операционные системы строились как операционные системы с монолитными ядром.
- Соответственно, в какой-то момент потребовалось определить места каждого важнейших в системе действий по отношению друг к другу

Иерархическая машина Медника-Донована

- Появилась в конце 60-70 годов, но актуально и сейчас,
- так как именно на его основе выделены функции, которые могут быть выполнены в режиме пользователя и те, которые необходимо выполнять в режиме ядра.
- Компьютер — программно управляемое устройство. Железо и софт — неотъемлемые части => уровни иерархии выстроены относительно аппаратной составляющей, которую когда-то кто-то из классиков назвал “голая машина”.



- Уровень 1 — выделение кванта процессорного времени процессам (диспетчеризация)
 - Уровень 2 — определение приоритетов, помещение в очередь готовых процессов
 - Иерархия построена по отношению к процессу, но классически называется “управление процессорами”. Под этим понимается, что именно к процессорам процессы выстраиваются в очередь.
 - На основе этой машины было определено понятие прозрачного/непрозрачного мейнфрейма
 - Появление структуры обосновано стремлением структурировать код ОС, также одна из целей — параллельная разработка коллективом программистов оп. систем
- Структуризация позволяет выделить модули, которые могут выполняться в режиме пользователя в структурах ядра.
- Ядро — в большой ОС разбитый на несколько уровней, каждый из которых ориентирован на свои функции управления системными аппаратными средствами.
 - Ядро является самым низким уровнем.
 - Предназначается для выделения аппаратных ресурсов процессам самой ОС и программам, выполняемым под ее управлением
 - UNIX 4.4 BSD

Управление терминалом		1	2	3	4	Сокеты	Обработка сигналов	Создание и удаление процесса
Необработанный телетайп	Обработанный телетайп	файловая система	Виртуальная память		Сетевые протоколы			
	Дисциплины линии связи	Буферный кэш	Страничный кэш		Маршрутизация		Планирование процессов	
Драйверы символьных устройств		Драйверы блочных устройств			Драйверы сетевых устройств		Диспетчеризация процессов	
Аппаратура								

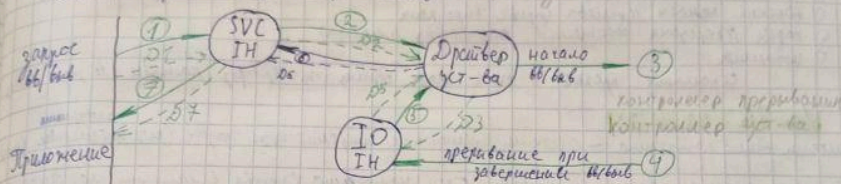
- 1 — символьный уровень — позволяет именовать файлы понятными человеку названиями — строка символов
- 2 — именование файлов
- 3 — отображение ядра (преобразование виртуального адреса в физический)
- 4 — страничные прерывания
- Все устройства вторичной памяти — блочные
Флешка — блочное устройство.
Жесткий диск — поделен на 2 неравномерные части – область своппинга и большая область хранения данных.
С помощью своппинга поддерживается механизм виртуальной памяти.

Микроядерная архитектура

Аех-16 - 13.12.23

□ Проблемы внеш. уст-ли (ИЗ)
Драйвер-спец. модуль ядра, управляющ. ВУ (отв. за ОС-драйверы)

□ Запрос приложения ввода-вывода



SVC - super-visor-call (super-visor - ОС в стадии выполнения)

ИН - interrupt handler (обработчик) (системный вызов)

Дл - data (т.е. данные к-рые передаются при вызове)

Нужно драйверные драйверы (функции) написать разработчиками ВУ
и драйвер ВУ имеет свои под-прер. (т.е. функции драйвера)
работает от прер-а передавая данные от буфера уст-ва в буфер ядра

□ Микроядро - ОС Mach

Unix/Linux - загрузочные модули ядра

Windows - многоуровневые драйверы

средства вычисления чисел
без перемножения

Может клиент-сервер (Минус: загрузка, к-рые разлет. ОС вынуждены в рел. ядре)

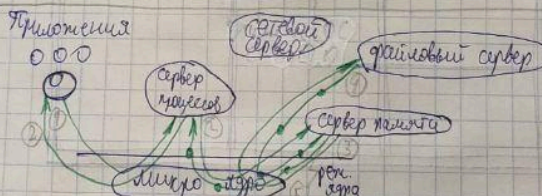
Приложение
○○○○
○

Ядро ОС
○○○○○○○○

Серверы ОС
○○○○

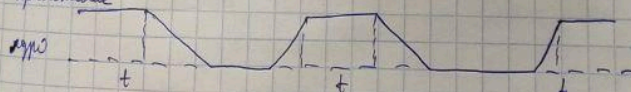
В ядре-те спец. вызов от приложения формируется
соедин-е к серверу ядра
Как между собой взаимодействуют
блокировки

микро ядро
рел. ядро



В микро ядре - минимум 4
к ядру и обратно
с функциями переключения контекста

• Приложение аппаратного контекста в микроядер. ср-е



Примитивный
" " " "
Низкоуровневые

ОС Mach - микро ядро, в ядре сервисы ядра, потоки, переключение контекста, виртуальный канал и драйверы уст-в - все остальное (api, драйверы системных устройств - рел. ядро)

• ОС на микро ядре низко пороговые из-за затрат на передачу сообщений



Однако, ОС с микрозерной архитектурой можно построить по конкретным условиям без переименования ядра

• Абстракция ядра Mach

- 1) процесс // основной цикл, в я-клет процессит выполнение (virt op, текст. код и данные, etc)
- 2) ядрос // часть Mach процесс, к-рый может вызвать ядро (сервисное расширение к ядру)
- 3) объекты ядрос // основа управл. вирт. ядром
- 4) порты // передача сообщений сараб. вирт. ядру
- 5) планировщик

Страничные прерыв. в Mach: ядро обрабат. и серверу памяти в рет. ядре

Лек-14 22.12.23

Minix

микро/ядро

Система общего назначения — системы, решающие широкий круг задач (есть микро/ядро)

Mach — управление в/р. ресурсами на ф/а. в вирт. ядре

Системы реального времени — системы спец. назначения (управл. процессом, в/р. ресурсами, к системе проз. или уст-м)

- Реальное время (RTOS) — в ОС это способность ОС обеспечить требуемый уровень сервиса в определенном промежутке времени (заданный интервал времени, в течение к-го мод. должен обслужить запрос)

Детерминированный, т.е. основанный на строгих контроле времени, характер работы — классическая обработка сервисов ядра ОС реал. вр. т.е. интервал времени известен, выполняются задачи

- VxWorks RTOS v6.2 (микро ядро) система реального времени

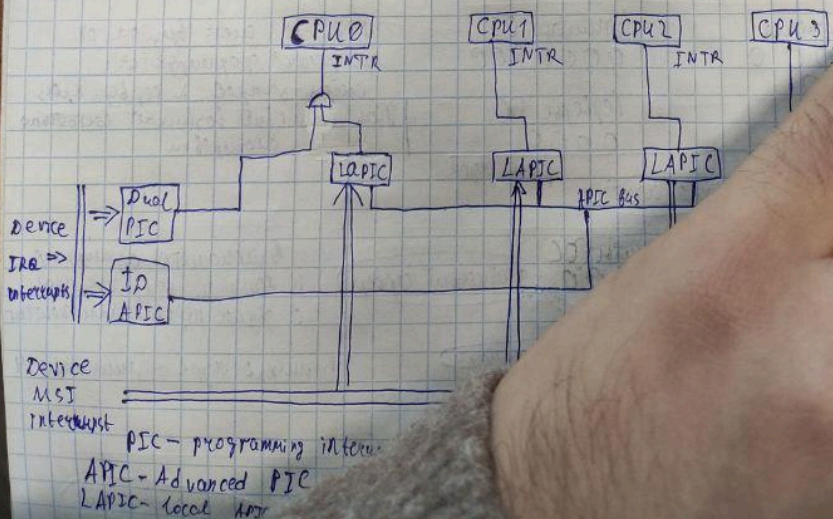
перерыв. сообщ. обеспеч. ядром

VxWorks предоставляет патентов. ал. планирования: FIFO, RR, sporadic scheduling

+ Разработка соответствует ал. планирования; в/р. ресурсы; как ядро, назначение периф.

- Прерывания. MSI (Message Signaled Interrupts; прерывания, сигнализирующие)

в отличие от векторных прерываний



- Низкая эффективность (отзывчивость системы — реакция на действия пользователя)
- Система общего назначения — решающая общий круг задач. Взаимодействие с пользователем играет огромную роль.

- Не существует системы общего назначения с микроядерной архитектурой
- Микроядерная архитектура востребована в системах реального времени: системах специального назначения — предназначены для управления внешними системами и устройствами
- Реальное время (определено в POSIX 1) в ОС — способность ОС обеспечить требуемый уровень сервиса в определенный промежуток времени
- Промежуток времени — заданный интервал времени, в течение которого система должна обслужить запрос. Запросы поступают в виде пр.
- Отличием сервисов ядра систем реального времени является детерминированный, то есть основанный на строгом контроле времени, характер (обслуживание ОС запроса должно быть выполнено за интервал времени заранее известной величины — по POSIX)
- Высоко приоритетные прерывания могут прервать мало приоритетные прерывания
- QNX Neutrino RIOS v.6.2
 - Можно пользоваться бесплатно, устанавливается в качестве учебной. Ограничение: нельзя продавать изделия, в которой установлена эта система, зарубеж.
 - Вся система построена на микроядре: минимизированное микроядро, из которого вынесены в отдельные процессы файловая система, доступ в сеть, ...
 - Основа любой микроядерной архитектуры — модель клиент-сервер, основана на передаче сообщений
 - Ядро обеспечивает передачу сообщений в любой системе
 - Предоставленные алгоритмы планирования:
 - FIFO
 - RR
 - sporadic scheduling (случайное, но на самом деле возникающее особенно)

(может быть соб. в ОС, кот. не имеет никакого периода)

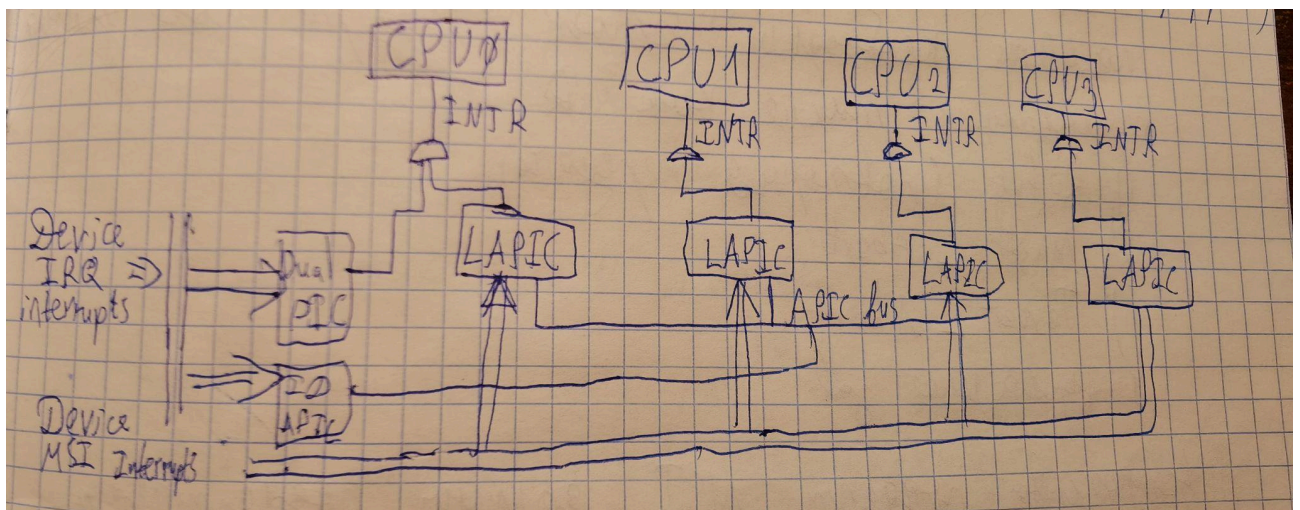
На основе sporadic планирования в UNIX есть возможность разработать собственный алгоритм планирования. Для этого в системе введено понятие начального бюджета и периода потока

Начальный бюджет, период потока

- Начальный бюджет потока — время, за которое поток может быть выполняться с начальным приоритетом до момента, когда его приоритет будет понижен
- Для потока определено максимально возможное число текущих пополнений — это число ограничено числом операций пополнения потоком => число ресурсов, выд.
- Для высокоприоритетных процессов можно исключать paging
- Развитые системы реального времени позволяют исключить paging

MSI

- MSI — message signaled interrupts
- Прерывание, сигнализируемое сообщением
- В отличие от векторных прерываний (?)
- Векторное прерывание — так как в результате пришедшего на контроллер прерывания, контроллер формирует вектор прерывания.



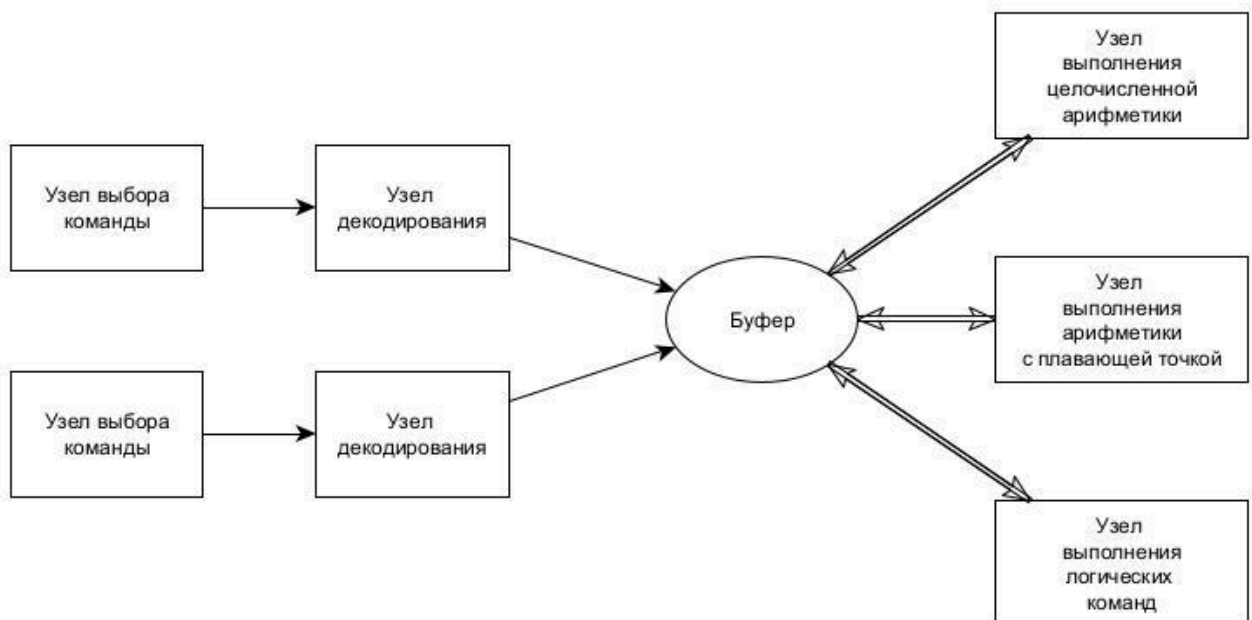
Обозначения:

- PIC — peripheral interface controller
- APIC — advanced PIC
 - контроллер для 32-разрядных систем
- LAPIC — local APIC
- MSI — для современных систем
- Старый прогр. контроллер прерываний для поддержки обратной совместимости 16 разрядных систем

- Схема связана с обратной совместимостью
 - присутствует контроллер для 16-разрядных систем
- К современным системам можно подключить, используя слоты шины ISA, старые устройства
- ISA — первая шина, индустриальный стандарт.
С ней связана работа программного контроллера прерываний.
- Медленная, затем PCI
- MSI — вместо использования выделенных линий прерываний
 - Устройства сигнализируют о прерываниях, записывая/отправляя сообщение по определенному адресу в памяти
- Используется механика захвата шины — bus mastering(?)
Механизм чаще всего описывается по отношению к DMA
- DMA — direct memory access
- Для записи такого сообщения устройство может иметь до 32 закрепленных за ним областей памяти
- В шинах PCI X, PCI express вообще не используются физические линии прерываний, прерывания доставляют как MSI
(PCI – для нее в UNIX были определены структуры и они существуют в ядре до сих пор, хотя в чистом виде PCI не исп., PCI EXPRESS)
- Системы, предоставляемые пользователю – абстракции, приведение к старому подходу.
Сделано для обратной совместимости.
- Так как MSI генерируется в форме записи в память, необходимо, чтобы поддерживались все условия транзакций, такие как:
 - Retry (повторная передача)
 - Master abort
 - Target abort
 - Нормальное завершение
- Устройства, поддерживающие MSI, должны поддерживать (pin) IRC с целью обеспечения обратной совместимости
- Аналогично маскированию прерываний в контроллере прерываний, маскированные прерывания, которые пришли из PIC, в MSI могут быть замаскированы.
Все это указано в драйвере устройства.

Точные и неточные прерывания

- Современные компьютеры являются суперскалярными.
Сначала появились конвейерные системы. Их появление обусловлено тем, что любая команда имеет 3 стадии выполнения.
- Каждая команда имеет 3 стадии выполнения
 - выборка
 - дешифрация
 - выполнение
- За счет разделения этих этапов можно обеспечить быстроедействие.
- Следовательно, есть 3 узла
 - выбора
 - декодирования
 - выполнения



- Разные узлы — выбираются разные команды.
- Команды выполняются не в том порядке, в котором они находятся в коде
- Аппаратура должна гарантировать, что результат выполнения должен получиться такой, как при последовательном выполнении. Это особенно важно при появлении прерываний.
- Если не предпринять действия, то после возвращения из прерывания система не может просто начать заполнять конвейер со значения

счетчика команд. Его значение является ключевым для продолжения выполнения.

- 1, 2, 3, 5, 8 — выполнились
- 4, 6, 7, 9, 10 — не выполнились
- Также очень важно при монопольном доступе.
А. о. может отн. вып. п. к.
Но очень часто это недопустимо для получения правильного результата: барьеры
- Есть понятие “барьеры”
- Точное прерывание — оставляет машину в строго определенном состоянии
- Свойства точных прерываний:
 - PC(program counter) указывает на команду, до которой все команды полностью выполнились
 - Ни одна команда из тех, что после указанной в PC, не выполнялась (?)
 - Состояние команды, на которую указывает PC, известно
- Ничего не сказано, что команды после той, что находятся в PC, не могут начаться
 - Все изменения, связанные с этими командами, должны быть отменены до обработки прерывания
- При аппаратном прерывании PC указывает на команду, которая не начала выполняться (?)
- При исключении PC указывает на команду, на которой возникло прерывание типа исключение
- Неточное прерывание — прерывание, не удовлетворяющее перечисленным требованиям
- Машины с неточными прерываниями обычно выгружают в стек огромное число данных, чтобы ОС могла определить, что происходило в момент прерывания.
- Сохранение большого объема данных только замедляет вход в обработчик прерываний. И возобновление после прерывания является сложным и медленным.
- => Для задач реального времени сверхбыстрые суперскалярный компьютер не применим из-за медленных прерываний

- Все преемники Pentium Pro поддерживают точные прерывания
 - Цена — сложная внутрипроцессорная логика обработки прерываний
 - Некоторые прерывания(обычно исключения) совсем не обязательно реализуются как точные, поскольку возникает ошибка, программа завершается и не важно, адрес какой команды содержал счетчик команд РС — ошибочная ситуация. (Если же речь не о страничном прерывании, то обычно выполняются свойства)

Режимы работы устройств

Режимы разделения времени всегда являются мультизадачными

- 1) Реальный режим (16-разрядный)
 - a) 20-разрядная шина адреса
 - b) DOS Intel 8086
 - i) дисковая операционная система
 - ii) не система пакетной обработки, но при этом однозадачная (объем памяти 1 МБ)
 - iii) в памяти находится 1 программа, которая выполняется от начала до конца
- 2) Защищенный режим (32-разрядный)
 - a) Основа многозадачности – защита адресного пространства процессов. Иначе бы многозадачная ОС не могла бы функционировать, т. к. любой мог бы насрать в память другому.
То есть “защищенный” — было всем известно, не открытие Microsoft.
 - b) Многозадачный режим с поддержкой виртуальной памяти
 - c) 2 типа виртуальной памяти
 - i) управление сегментами по запросу
 - ii) управление страницами по запросу
 - d) Для каждого процесса виртуальное адресное пространство 4 ГБ
 - e) мультизадачный режим (то есть с разделением времени)
 - f) Основой многозадачности является защита адресного пространства
- 2a) Специальный режим защищенного режима - v86
 - a) режим эмуляции DOS
 - b) в этом режиме как задачи запускаются ОС реального режима (виртуальные машины реального режима)
 - c) В каждом из них выполняется одно приложение реального режима. Может быть запущено много V86, т. к. защищенный режим многозадачен.
 - d) Пример: консоль

3) Режим long (64-разрядный)

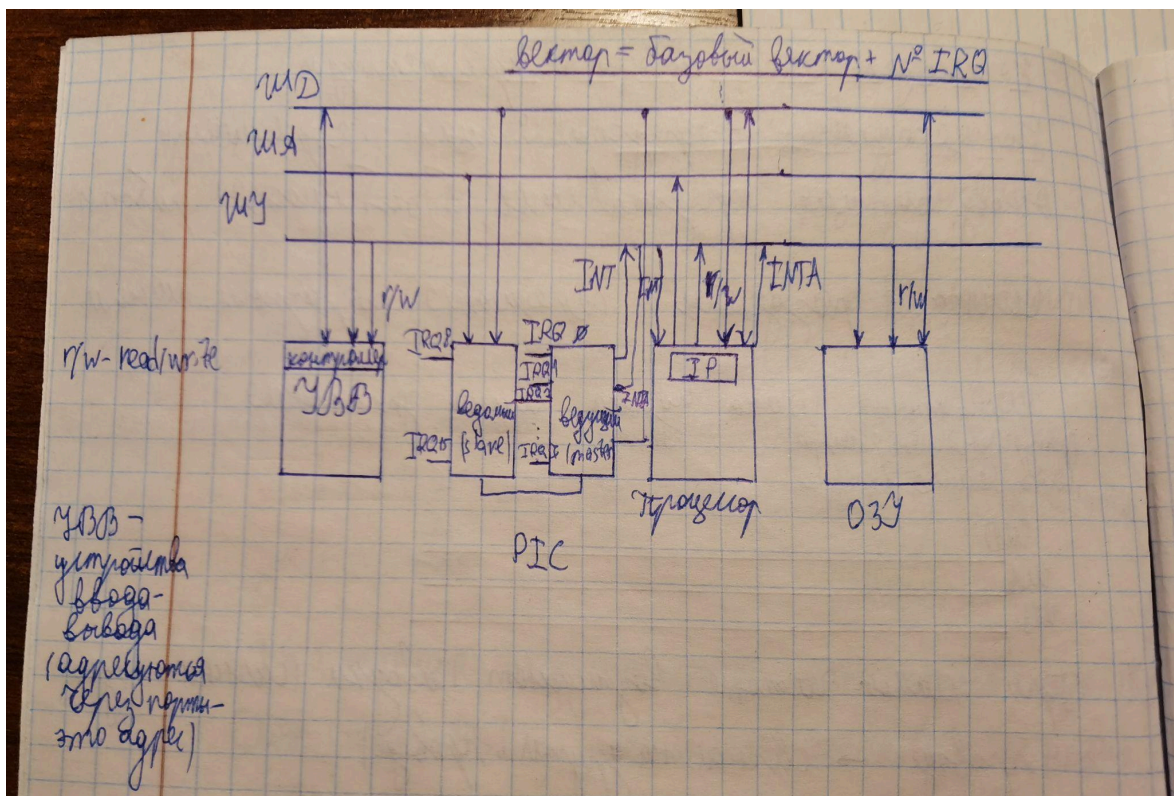
- a) В этом режиме поддерживается режим compatibility — можно выполнять 32-разрядные программы
Это следствие принятия Intel и Microsoft соглашения о предоставлении возможности выполнять старые программы на новых компьютерах – обратная совместимость.
Поддерживается аппаратно.
Сделано в результате анализа фирмами рынка США. Когда появились 386, 486, когда произошел технический прорыв, стали появляться большие интегральные схемы, оказалось, что большой сегмент рынка(фирмы, производства, военные), заинтересованы в выполнении старого ПО на новых компьютерах. В результате стали поддерживать обратную совместимость.
- b) Нельзя выполнять 16-разрядные программы
- c) Поддерживается только страничное преобразование
- d) Младшие части регистров аппаратно доступны
- ISA — шина индустриального стандарта (в старых ПК)
(оставлена в новых, чтобы можно было подключать старые внешние устройства)
- Слот шины BSI — более быстрая шина в более новых компьютерах
- USB-шина
- 3 типа сигналов:
 - Данные и команды
 - Адреса
 - Сигналы управления

Концептуальная трехшинная архитектура

- Концептуальная — представляющая теоретическую основу, дающая представление об особенностях работы
- В технике любая концепция должна быть реализована.
- Сигналы разделены и передаются по разным шинам
- Шины:
 - шина данных:
 - имеет разрядность регистров
 - параллельная передача: по каждому проводу передается 1 разряд.

(При 3.5 ГГц параллельная передача невозможна — возникают наводки: сигнал 1 провода создает наводки на другой. Для технической реализации необходимо экранировать каждый провод) => в наших ПК шина — (определение ниже)

- шина адреса
- шина управления
- Шина — аппаратное устройство, м.б. имеющее свой тактовый генератор
- Возможна только последовательная передача — так называемая витая пара.
- Для передачи информации используется чистая медь.
- На эти шины вешаются все устройства системы
- Основная концепция архитектуры фон Неймана — концепция хранимой программы
 - Программы и данные хранятся в одной памяти (в последовательных адресах).
Команду от данных отличить невозможно. Команды программы записаны в памяти последовательно.
 - Команда не должна содержать адрес следующей команды



Все остальные устройства в ОС являются внешними.

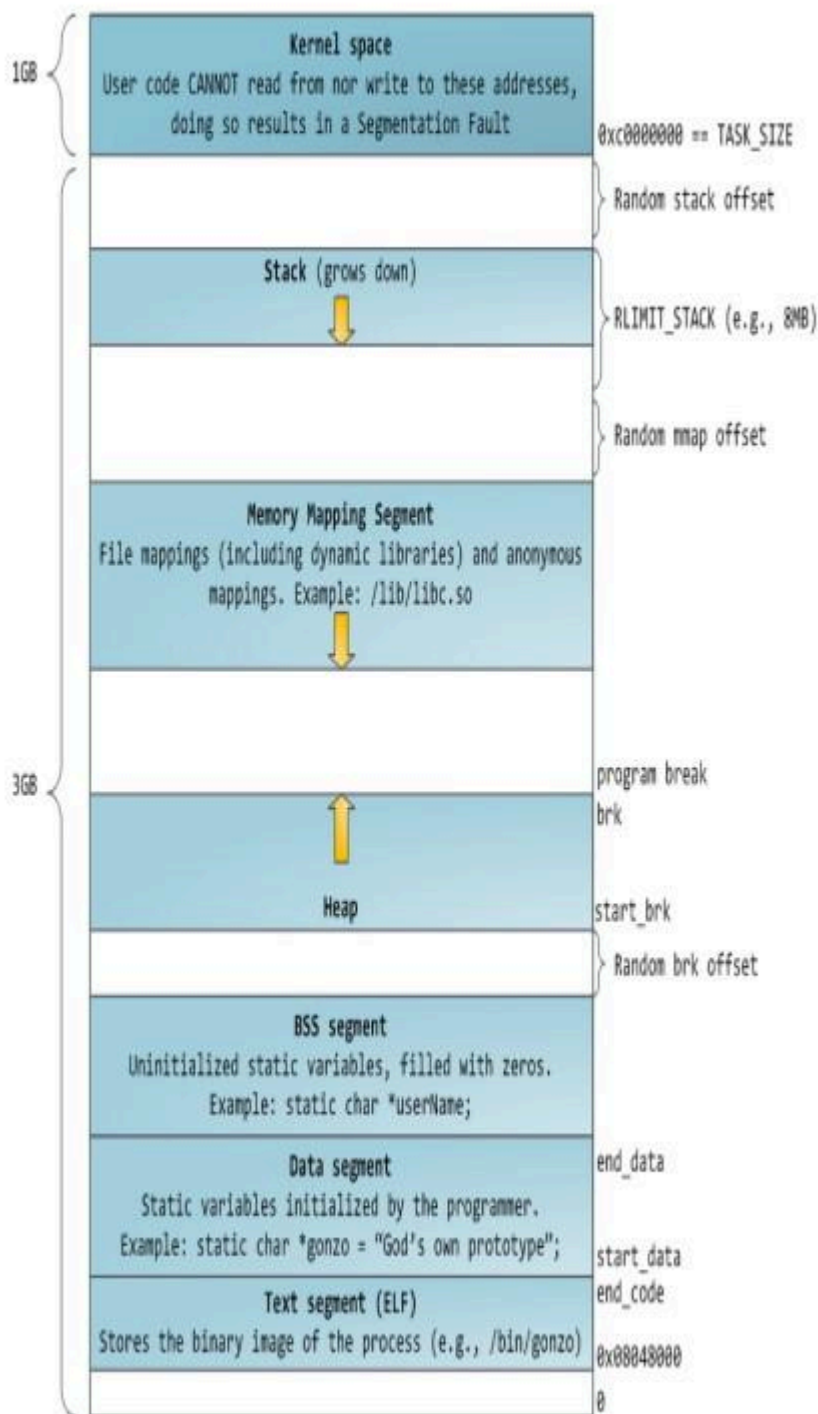
Внешние устройства адресуются через порты ввода/вывода. Порт – это адреса.

Процессор адресует устройство, указывая порт.

Видеокарта взаимодействует с процессом через ОЗУ(отображение в память)

Внешние устройства: 1) медленные 2) клавиатура и мышь(зависят от пользователя)

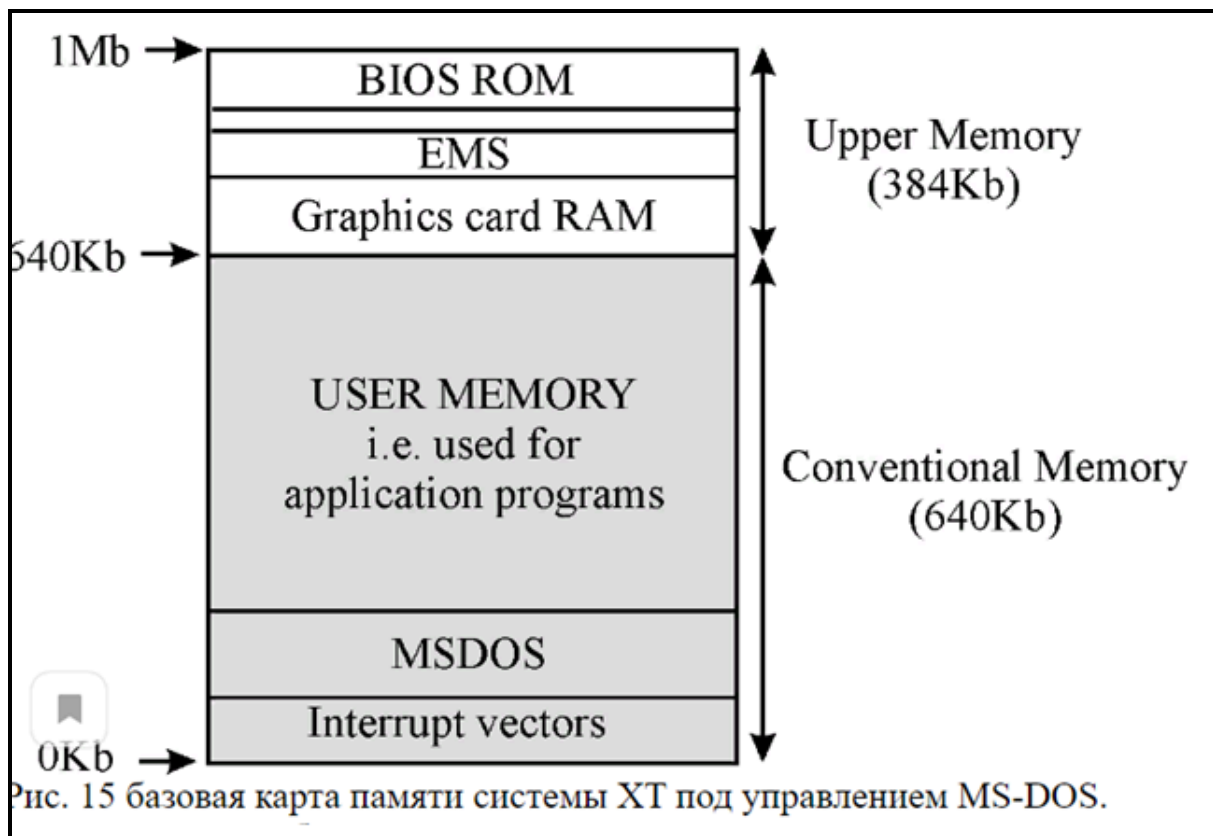
- В IBM 360 – принцип распараллеливания функций
- Процессор освобожден от управления внешними устройствами. Управление передано специальным пр.
- В IBM 360 – контролируют каналы
- В нашей архитектуре(шинной) — контроллеры — программно управляемые устройства. По шине данных получает от ЦП команду. Контроллер — устройство, входящее в состав
- Адаптер — контроллер на материнской плате
- Возникла необходимость информировать процессор о завершении операции ввода-вывода(чтобы он обрабатывал данные, полученные в результате операций. Не важно I или O, в наших системах будем получать информацию об успешности)
 - именно для информирования процессора об окончании IO в систему прерываний были включены аппаратные прерывания
- Виртуальное адресное пространство процесса в архитектуре x86



Прерывания

- Система прерываний:
 - системные вызовы (программные прерывания)
 - исключения

- аппаратные прерывания
 - от системного таймера (единственное периодическое)
 - от внешних устройств
 - от действий оператора (Ctrl+C)
- Система прерываний организуется через контроллер прерываний.
- Контроллер прерываний PIC(program interrupt controller) состоит из 2 микросхем, каждая по 8 входов, каскадное соединение.
Входы контроллера – линии IRQ(interrupt request) (линии прерываний). Имеют номера. IRQ2 используется для каскадного соединения.
 - Линии IRQ связаны с внешними устройствами
 - IRQ – interrupt request
 - IRQ0 – тики
 - IRQ1 – от клавиатуры PC/2
 - IRQ2 – от мыши
- 1 контроллер – ведущий, master
- 2 контроллер – ведомый, slave
- Когда на контроллер прерываний приходит прерывание от внешнего устройства, контроллер формирует INT, которые по шине управления поступает на выделенную ножку процессора.
- Цикл выполнения команды
 - → Выборка → Дешифрация → Выполнение
- В конце цикла выполнения команды ЦП проверяет наличие сигнала INT на выделенной ножке. Если он присутствует, то процессор по шине управления посылается INT A(answer)
- Получив этот сигнал от процессора(информирует о готовности обл. с к. д.) контроллер по шине данных процессору посылает вектор, который формируется как сумма базового вектора + номер линии IRQ.
- вектор = базовый вектор + №IRQ
- В реальном режиме базовый вектор ведущего контроллера равен 8, отсюда INT8h — $8+0=8$;
INT9h — $8+1=9$; клавиатура



- На контроллер прерываний посылаются маскируемые прерывания
 - процессор отправляет маску на контроллер, чтобы запретить прерывание
 - Маска посылается по шине данных на контроллер => нужно указать адрес =>
- Контроллер имеет порт (то есть адресуемое устройство)
- В наших компьютерах смешанное адресное пространство
- Системная шина — локальная шина памяти. ЦП и ОС соединены отдельной быстродействующей шиной. Память более чем на порядок медленнее, чем процессор.
- => В процессоре 4 кэша (L1-L3, TLB)
 - L1 и L2 – у каждого ядра. L3 — общий.
- В кэше – команды и данные, к которым были последние обращения. ЦП обращается к ним, и если там нет, лезет в ОЗУ.
- ЦМОС-микросхема
 - энергонезависимая
 - питается от аккумуляторной батарейки
 - благодаря этой микросхеме верное время после выключения компьютера

- Стартуют компьютеры в реальном режиме, чтобы сократить объем стартового кода. Потом система уже загружается в ОЗУ.
- ОС – программа, лежащая на диске => ОС должна быть загружена в ОЗУ.
 - никогда не называется файлом
 - на диске во вторичной памяти

Функции INT8h

- 1) инкремент счетчика реального времени (
 - под реальным временем подразумевается физическое суточное поясное время (время, в котором мы живем).
 - Счетчик находится в области данных BIOS по известному адресу. Сегментный адрес 40h. Это оперативная память (энергонезависимые). Когда компьютер выключается, память теряется.
 - 2) Декремент счетчика реального времени до отключения моторчика дисководов
 - Первые компьютеры на INtel 80 не содержали винчестеры, поэтому всю информацию писали на гибкие диски, которые нужно раскрутить до определенной скорости. Это выполнял моторчик дисководов.
 - На то, чтобы раскрутить нужно время. Остановился, нужно записать? Снова раскручиваем. Поэтому контроль моторчика был возложен на счетчик. В него писали 2 секунды, после чего посылался сигнал отключения в порт дисководов.
 - Порты используются для адресации внешних устройств.
 - 3) Вызов пользовательского прерывания 1CH
 - В чистом виде заглушка, чтобы разработчики вешались на него, а не на 8-го прерывания.
 - Если повесят слишком много на 8-е прерывание: это прерывание самое приоритетное, его никто не может прервать. В результате страдает так называемая **отзывчивость системы**.
- Когда вызывается обработчик 8 прерывания? По тикеру

- Тактовый генератор. У компьютера указана тактовая частота. Он генерирует сигнал, в котором длительность низкого напряжения равна длительности высокого напряжения.
- Из него с помощью делителей и линий задержки выделяются импульсы с определенным периодом.
- Тик в ДОСе сформирован так, что он повторяется 18.2 раза в секунду. По этому тик и происходит вызов обработчика 8 прерывания.
- По каждому тик вызывается обработчик, который в первую очередь инкрементирует счетчик реального времени
- Прерывание от таймера - аппаратное прерывание, единственное периодическое прерывание в системе.
- Вывод: в обработчиках прерываний циклов быть не может!!! Их нужно завершать как можно быстрее, иначе будет страдать отзывчивость системы. Это общее правило для всех обработчиков аппаратных прерываний.

Программный канал в Unix

- 1) Именованный программный канал – виден в файловой системе как специальный файл, помечен p.
- 2) Неименованный программный канал:
 - a) были изначально
 - b) создается системным вызовом `pipe()`
 - c) имеет дескриптор(но не имеет имени)
 - d) На неименованном программном канале задается массив дескрипторов: `int fd[2];`
Этот массив передается системному вызову `pipe`.
 - e) `Pipe()`, как и все системные вызовы, в случае ошибки возвращает -1.

Программный канал

- Буфер типа FIFO в системной области памяти – в области данных ядра системы
- Поточковая модель передачи сообщений

- Программный канал имеет встроенное средство взаимного исключения
 - нельзя читать, если пишут
 - нельзя писать, если читают
- В силу того, что программный канал не имеет имени и имеет только дескриптор, то взаимодействовать через программный канал могут только родственные процессы, так как потомок получает дескрипторы открытых файлов
- Перед тем, как писать в канал, его закрывают на чтение.
- Если выполняется чтение, а канал пуст, то канал блокируется
- Выбор размера трубы в 1 страницу, так как передача данных 4096 байт (paging) оптимизирована в системе
- Информация, считанная из pipe, перестает существовать
- Программный канал поддерживается и управляется файловой системой.

Труба буферизуется на трех уровнях:

- 1) Труба буферизуется в системной памяти – в области данных ядра системы. При переполнении системной памяти те буферы, которые имеют наибольшее время существования, переписываются на диск. При этом используются стандартные функции управления файлами.
- 2) Если процесс записывает более 4096Б (то есть больше размера 1 страницы), то труба буферизуется во времени, приостанавливается процесс до тех пор, пока данные не будут прочитаны. То есть система приостанавливает процесс, который пишет в канал, если канал переполнен. Таким образом write работает, если в пайпе есть место. Иначе блокировка. Такая ситуация может быть, если читающий процесс запаздывает.
- 3) третьего уровня у меня не было...

Вызов ps

ps -ajx

- a — вывод процессов, которыми владеют другие процессы
- x — вывод процессов, не имеющих управляющего терминала
- j — вывод дополнительной информации

ls -al

Process flags:

- 0 – нормальное выполнение процесса
- 1 — forked but not exec
(то есть установлен флаг Copy On Write(в том случае если child не вызвал exit), который находится в дескрипторе страницы)
- 4 — used super-user privileges
 - Процесс с правами суперюзера имеет доступ к функциям и структурам ядра
 - (Помните, что для системы вы не существуете)

Process State Codes

- D (device) — непрерываемый сон
 - обычно блокировка процесса в ожидании завершения ввода-вывода
 - нельзя прервать
- R — running or runnable (выполняется или может выполняться)
 - может выполняться – то есть находится в очереди готовых процессов
 - система не различает, выполняется процесс или находится в очереди готовых процессов
(при этом для планировщика это важно, поэтому есть структура, которая позволяет узнать, сколько процесс был в очереди)
- S — прерываемый сон
 - сон в ожидании завершения события
- Z — зомби
- T — terminate

7 типов файлов

- 1) — — обычный файл
 - a) файл, в котором пользователь хранит информацию длительное время (во вторичной памяти)
 - b) Обычный файл — поименованная совокупность данных, м.б. бессмысленная
- 2) d — директория
 - a) хранится во вторичном запоминающем устройстве

- b) может содержать поддиректории(другие директории) и обычные файлы
- 3) p — именованный программный канал (pipe, труба)
 - a) модель потоковой передачи данных
 - b) mknod [функции] <имя> p
 - c) буфер типа FIFO
- 4) l — soft link (гибкая ссылка)
 - a) специальный файл, содержащий массив символов, то есть строку, представляющую путь к файлу
 - b) ln -s /path/to/file /path/to/symlink
- 5) s — сокет (в пространстве файловых имен)
- 6) c — специальный файл символьного устройства (character)
- 7) b — блок
 - a) специальный файл блочного устройства

7 типов файлов отражают парадигму UNIX. В Unix – все файл. Устройства тоже файл.

~~И ты файл... Хотя нет, тебя для системы не существует.~~

Hardlink, softlink

- **hardlink** — еще одно равноправное имя файла
- **ln file1 newlink**
- В UNIX Linux имя файла не является его идентификатором
- Идентификатором файла является inode
 - Это структура, имеющая номер, метаданные
 - описывает физические файлы
- Один и тот же файл может иметь много разных имен hardlink
- Количество hardlink на файл указывается
- Файл можно удалить из системы, когда остался последний hardlink
- **softlink** — специальный файл, содержит массив символов, то есть строку, которая является путем к файлу, т.е. полным именем к файлу
- Система работает с полным именем файла, т.е. путь_к_файлу + shortname
- Для получения доступа к файлу ОС проходит весь путь к файлу
- Короткие имена возможны, так как поддерживается текущая директория
- В текущей директории можно работать с короткими именами. Система дописывает к короткому имени путь к файлу

Приоритеты

- PRI — системная составляющая приоритета
- NI — пользовательская составляющая приоритета (может только понизиться – любезный)
- **renice** – только в сторону уменьшения приоритета
- Чем больше значение приоритета, тем он ниже

IPC (inter process communication)

К средствам межканального взаимодействия в System V относятся:

- сигналы
- семафоры
- именованные и неименованные программные каналы
- сегменты разделяемой памяти
- очереди сообщений
- файлы, отображающиеся в память (принято относить к более поздним средствам взаимодействия)

Сигналы

- **Сигнал** — это программное средство информирования процесса о событиях, которые могут произойти как внутри процесса, так и вне его(то есть в системе)
- Как правило(обычно) получение процессом сигнала указывает ему на необходимость завершиться
- В классическом UNIX было определено не более 20 сигналов. Сигналы, как и другие предопределенные константы, представлены в двух формах:
 - в мнемонической (большими буквами) форме
 - в числовой форме
- Информация хранится в `signal.h`

SIGHUP	1	Разрыв связи с терминалом
SIGINT	2	Завершение выполнения процесса, посылается при нажатии Ctrl+C
SIGKILL	9	Системный вызов kill
SIGSEGV	11	Выход процесса за его адресное пространство
SIGPIPE	13	При записи в Pipe, когда нет читающего процесса. Информирует о недопустимой операции
SIGALARM	14	Прерывание от системного таймера
SIGTERM	15	Выполнение команды kill в командном режиме. К адресуемому PID
SIGUSR1	16	При системном вызове kill. К адресуемому PID
SIGUSR2	17	При системном вызове kill. К адресуемому PID
SIGCLD	18	Информирование процесса о завершении процесса-потомка
SIGPWR	19	Падение напряжения. Система переходит к выполнению необходимых действий для сохранения необходимых значений
SIGPOOL	22	Событие в stream-устройстве

- Два макроса
 - SIG_DFL
 - SIG_IGN
- Средствами послыки и приема сигналов в UNIX служат системные вызовы:
 - kill
 - signal

```
int kill(int pid, int sig)
```

- pid в kill — не обязательно id процесса.
Если это Id процесса, то сигнал получит конкретный процесс.
Если pid <= 1, то сигнал будет послан группе процессов:
Если pid равен 0, то она отправляется каждому процессу в process

group вызывающего процесса.

<https://man7.org/linux/man-pages/man2/kill.2.html>

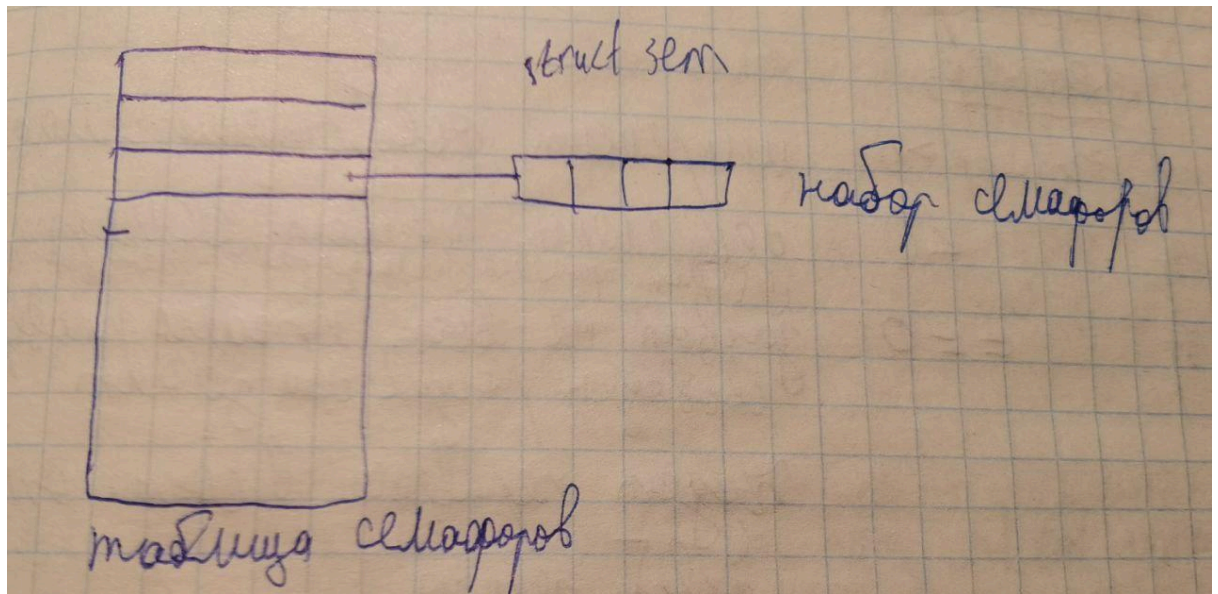
- Процессы в UNIX определяются в группы по разным критериям. Например, терминальная группа процессов
- Родитель является лидером группы, создает группу процессов.
- `kill(getpid(), SIGALRM);` — сигнал будет послан процессу, который вызвал `kill`.

```
void *signal(int sig, void (*handk(int)))
```

- `handle` – стандартное название обработчика
- Реакцией процесса, вызвавшего `signal()` с аргументом `handl`, на сигнал `sig` будет вызов этого обработчика
То есть с помощью этого системного вызова можно на сигнал установить собственный обработчик.
- Системный вызов `signal` не входит в POSIX 1
- POSIX 1 перечисляет около тысячи системных вызовов, которые должна поддерживать операционная система, чтобы выполнить переносимость ПО.
Это не единственный стандарт. Наиболее интересны стандарты, связанные с аппаратным ПО: X/Open, XPG4
- Системный вызов `signal` возвращает указатель на предыдущий обработчик данного сигнала
 - => его можно использовать для восстановления обработчика
- Сигналы возникают в ответ на внешние к процессу события
- Процесс будет реагировать на внешние асинхронные события. Внешние асинхронные события сопровождаются сигналами, которые получает процесс. Процесс может поменять ход своего выполнения.
 - Асинхронно — независимо от того, что делает процесс:
Пример: процесс что-то делает, нажимаем `ctrl+c`. Независимо от того, что делает процесс, он меняет ход выполнения.
- Одни и те же сигналы могут получать процессы одной группы
- Процесс, вызвавший `fork()`, создает свою группу

Семафоры UNIX

- Семафоры UNIX вносят дополнения в семафоры Дейкстры.
- В современных системах поддерживаются наборы семафоров
- Наборы семафоров представлены массивами семафоров
- В UNIX LINUX семафоры поддерживаются системной таблицей(находится в адресном пространстве системы) семафоров. Каждый элемент таблицы описывает один набор семафоров (то есть это дескриптор)
- Дескриптор набора семафоров хранит:
 - имя (целое число) – присваивается процессом, который создал набор семафоров. Другие процессы по этому имени могут получить доступ к нему.
 - uid
 - идентификатор создателя наборов семафоров и идентификатор его группы.
Процесс, uid которого совпадает с uid созд. может удалить набор и изменить его существующие права(?)
 - права доступа
 - количество семафоров в наборе
 - время изменения 1 или нескольких семафоров последним процессом
 - время последнего изменения управляющих параметров набора каким-либо процессом
 - указатель на массив семафоров
=> в наборе семафоров семафоры обозначаются индексами начиная с нулевого



- На семафорах определены системные вызовы:
 - `semget()` — создание набора семафоров
 - `semctl()` — изменение управляющих параметров набора семафоров
 - `semop()` — операции на наборе семафоров или на семафоре
- Для определения операций на семафорах определена структура

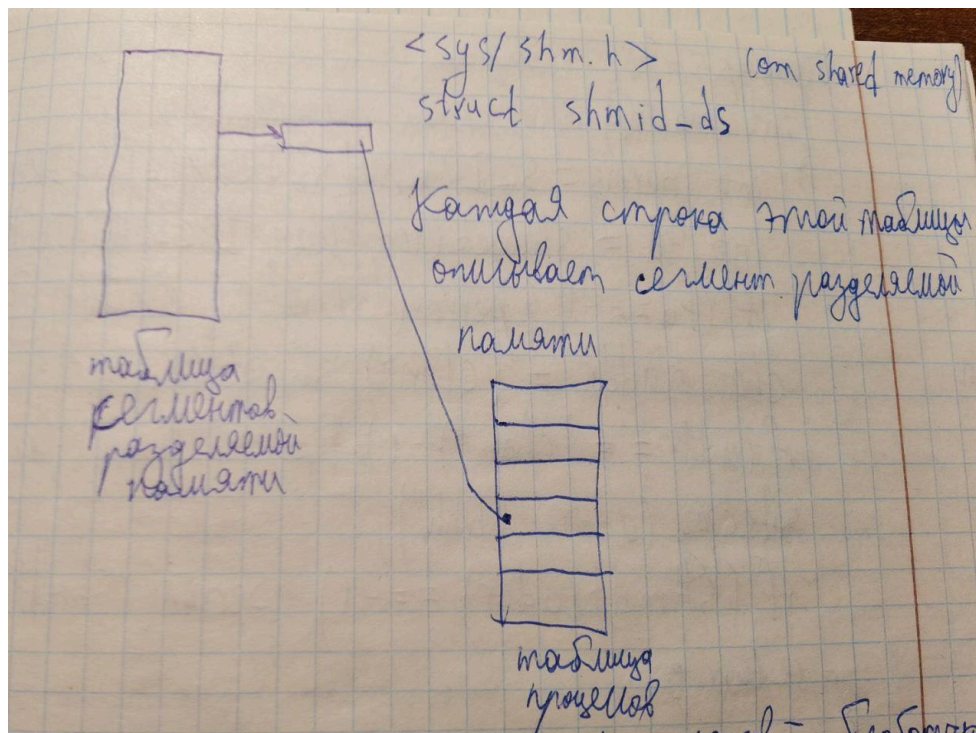
```
struct sembuf
{
    ushort sem_num; //index
    short sem_op; // операция
    short sem_fl; // флаги
}
```

- В отличие от семафоров Дейкстры, на семафорах UNIX определено 3 операции
- `sem_op`:
 - > 0 — инкремент, освобождение семафора
 - < 0 — декремент, захват семафора
 - блокирование процесса, если декремент невозможен
 - $= 0$ — захват не выполняется, процесс проверяет, свободен ресурс или занят
 - будет переведен в состояние ожидания освобождения ресурса. После его освобождения декремент не выполн.

- Флаги sem_fl:
 - IPC_NOWAIT — информирует систему о нежелании процесса переходить в состояние ожидания (освобождения семафора)
 - Наличие данного флага объясняется желанием избежать блокирования всех процессов, находящихся в очереди процессов, в случае, если захвативший ресурс процесс завершится аварийно или получит сигнал kill
 - Так как kill невозможно перехватить, процесс не может освободить семафор
 - Блокированный процесс не получает процессорного времени
 - SEM_UNDO — данный флаг указывает ядру, что необходимо отслеживать изменение значения семафора в результате вызова sem_op.
 - Указывает ядру, что при завершении процесса, вызвавшего sem_op, необходимо ликвидировать сделанные изменения, чтобы процесс не были заблокированы на семафорах навечно
- semget возвращает файловый дескриптор созданного набора семафоров
- Семафоры могут служить как источник информации — процесс освобождает семафор как событие. Но это одно значение — свободен или занят. Но есть и другие средства передачи данных.

Сегменты разделяемой памяти

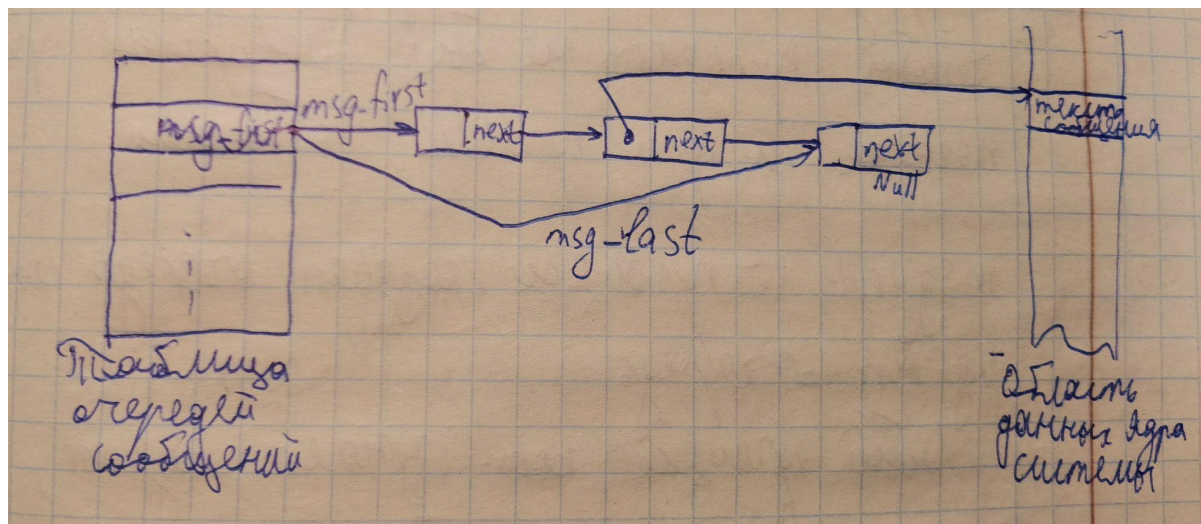
- Одно из средств передачи данных
- Как и программные каналы, сегменты разделяемой памяти создаются в адресном пространстве ядра, потому что адресное пространство процесса защищено и к нему не может обратиться другой процесс => обмениваться данными процессы могут через 3 пространство – адресное пространство ядра.
- В ядре поддерживается таблица сегментов разделяемой памяти



- Каждая строка этой таблицы описывает сегмент разделяемой памяти
- Процесс получает указатель на сегмент разделяемой памяти
 - то есть сегмент разделяемой памяти, созданный в ядре, подключается в пространство процесса, что исключает необходимость копирования данных из адресного пространства процесса в адресное пространство ядра системы.
- На сегментах разделяемой памяти определены следующие системные вызовы:
 - `shmget()` — создать разделяемый сегмент указанными правами доступа
 - `shmctl()`
 - `shmat()` — получить указатель на сегмент разделяемой памяти
 - `shmdt()` – отключение от адресного пространства процесса
- Сегменты разделяемой памяти работают не как трубы, а как обычные буфера (там лежит, пока не удалим). 1 процесс может писать, другой читать.
- Любой процесс, зная идентификатор сегмента, может подключить сегмент, считать данные или записать
- Так как это часть а. п., то по этому а. п. можно перемещаться с помощью обычного указателя.

Очереди сообщений

- Сообщение — наиболее общее средство передачи информации
- Очереди в IPC System V реализованы на отдельно стоящих машинах.
- В системе имеется таблица очередей сообщений. Каждый элемент этой таблицы описывает одну очередь.



```
struct msgid_ds
msgget()
msgctl()
msgsnd()
msgrcv()
```

```
struct msgbuf
{
    long mtype; // тип сообщения
    char mtext[MSGMAX];
}
```

- Данные могут быть любыми, размер ограничен максимальным размером сообщения в системе
- Очереди сообщений построены таким образом, чтобы процессы могли не блокироваться при отправке или приеме сообщений
- Системный вызов `msgget` создает очередь сообщений, идентифицирует эту очередь. После этого любой процесс, который знает идентификатор очереди, может отправить или взять сообщение

- Когда процесс передает сообщение в очередь(указав ее id), ядро создает для него новую запись и перемещает в конец списка записей указанной очереди
- В каждой такой записи указывается
 - тип сообщения
 - число байт данных
 - указатель на фактическую область данных ядра, где будет находиться сообщение
- Ядро копирует сообщение из адресного пространства отправителя в пространство ядра, после чего отправитель может завершиться, а сообщение остается доступным для чтения другим процессом
- Когда процесс выбирает сообщение из очереди, ядро копирует сообщение в адресное пространство процесса, после чего запись удаляется

- 1) Взять самое старое сообщение независимо от типа(принцип fifo)
- 2) Если идентификатор сообщения совпадает с id, указанным процессом
 - a) процесс может взять сообщение по идентификатору
 - b) если существует несколько сообщений с таким id, то выбирается самое старое(то есть то, которое положено раньше других)
- 3) Процесс может взять сообщение, числовое значение типа которого является наименьшим из меньших или равным значению типа, указанного процессом
 - a) взять по типу или самое маленькое
 - b) если несколько, то самое старое

- Итого: очереди сообщений предполагают, что процессы могут не блокироваться при отправке и при получении сообщения. Отправил и забыл.
- Сравним очереди сообщений и сегменты разделяемой памяти
 - очереди:
 - 2 копирования: при отправке (из пространства пользователя в пространство ядра) и при получении (из пространства ядра в пространство пользователя)
 - При использовании очередей сообщений может быть

реализована динамическая связь.

- В System V сообщения можно извлекать не только по принципу FIFO, но и в некотором порядке, который может определяется приоритетами, но это потребует дополнительных настроек и действий.
- В IPC System V имеются совершенные средства взаимодействия процессов.
- UNIX BSD предоставляет одно универсальное средство взаимодействия — сокеты

Демон

- В столбце TTW ? — демон, не имеющий управляющего терминала
- 3 идентификатора демона равны:
 - предка
 - собственный
 - группы (является лидером группы)
- Ssl
 - S — прерываемый сон
 - s — лидер сессии
 - l — многопоточный
- Группа — группа процессов в результате fork
 - они будут получать одни и те же сигналы

Правила программирования демона

- 1) Вызывается функция `umask(0)`
 - a) сбрасывает маску режима создания файла
 - b) `main` – предок, в котором вызовем функцию `daemonise`
 - c) у него предком является `shell`
 - d) потомок наследует сигнальную маску и обработчики сигналов, дескрипторы открытых файлов и маску режимов создания файлов.
 - e) сбрасываем маску режима создания файлов, чтобы демон мог создавать файлы с любыми правами доступа.
Сигнальную маску не сбрасываем

2) Вызывается системный вызов `fork()`

- a) Процесс, вызвавший `fork` создает группу и становится ее лидером => после завершения процесса предка потом лишается родительской группы. (Но не лишается терминальной группы)
- b) создается процесс-потомок и предок завершается
- c) это делается, чтобы процесс-потомок не был лидером группы, лишился группы
- d) это необходимо, чтобы вызвать функцию `setsid()`

3) `setsid()`

- a) делает процесс демоном
- b) создает для потомка сессию, делает его лидером новой группы, лидером сессии (группа будет иметь его же `id`)
- c) потомок теряет терминал (чтобы не влиять на его работу)
Почему у демона не должно быть управляющего терминала?
1 — `init` — терминальный процесс
2 — `cas red daemon`
3 — `cas soft IRC daemon`
... — `worker`

Все эти процессы находятся в состоянии прерываемого сна. Они выполняют важные системные функции. Влиять на выполнение демона не надо => он лишается управляющего терминала.

Группа(статическая) — группа процессов в результате `fork`, они получают один и тот же ...

Сессия(динамическая) — как действие, развернутое во времени

4) `chdir("/")`

- a) текущий каталог меняется на корневой
- b) чтобы если демон запущен с подмонтированной файловой системы, ее можно было отмонтировать (пример: флешка)

5) Закрытие всех открытых файлов

- a) `getrlimit(RLIMIT_NOFILE, &r)`
- b) чтобы закрыть все открытые файлы(в силу того, что потомок наследует файлы, открытые предком)
- c) позволяет получить максимальный номер дескриптора
Используем это значение в цикле для закрытия всех открытых

файлов.

```
d) if (r1.rlim_max == RLIM_INFINITY)
e)     r1.rlim_max = 1024; // максимально возможное количество
    открытых файлов для процесса
f) for (i = 0; i < r1.rlim_max; i++)
g)     close(i);
```

- h) Процесс имеет 2 таблицы по 512 страниц дескрипторов открытых файлов.
- i) У каждого процесса есть 3 дескриптора стандартных потоков in, out, err => не рекомендуется использовать эти названия.
- j) Эти дескрипторы всегда существуют, открываются для любого процесса автоматически. Эти потоки направляются на dev/null.
dev/null — черная дыра. Все что будет вводится, отправится в никуда, чтобы не влиять на работу демона.

```
k) fd0 = open("/dev/null", ORDWR);
l) fd1 = dup(0);
m) fd2 = dup(0);
```

- В функции daemonise игнорируется сигнал SIGHUP (сигнал утраты терминала, но демон и не должен иметь управляющий сигнал) (С помощью функции sigaction)
- Заканчивается демон инициализированием журнала
 - openlog(cmd, LOG_PID|LOG_CONS, LOG_DAEMON)
- Демоны в единственном экземпляре
 - already_running
 - Иначе возникнет неоднозначность: как определить, к какому экземпляру демона обращаться?
 - Как свести результаты выполнения экземпляров демона воедино? Как получить правильный результат? (ведь демоны делают одну и ту же работу)
 - Демоны предоставляют важные системные сервисы
- Чтобы контролировать выполнение демона в единственном экземпляре, существует файл блокировки (устанавливается блокировка на запись)
- Другой демон пробует открыть файл и записать туда свой id — завершится с ошибкой

- В lockfile вызывается fcntl()

```
• struct flock fl;  
•  
•     fl.l_type = F_WRLCK; // тип блокировки  
•     fl.l_start = 0; // относительное смещение  
•     fl.l_whence = SEEK_SET; // начальное смещение  
•     fl.l_len = 0; // размер
```

- В результате блокировка устанавливается на весь файл

Код демона

```
#include <pthread.h>  
#include <syslog.h>  
#include <unistd.h>  
#include <stdlib.h>  
#include <fcntl.h>  
#include <string.h>  
#include <errno.h>  
#include <stdio.h>  
#include <time.h>  
#include <signal.h>  
#include <sys/types.h>  
#include <sys/file.h>  
#include <sys/stat.h>  
#include <sys/resource.h>  
  
#define LOCKFILE "/var/run/daemon.pid"  
#define LOCKMODE (S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH)  
#define SLEEP_TIME 1  
  
sigset_t mask;  
  
int lockfile(int fd)  
{  
    struct flock fl;  
  
    fl.l_type = F_WRLCK;  
    fl.l_start = 0;  
    fl.l_whence = SEEK_SET;  
    fl.l_len = 0;
```



```

        return fcntl(fd, F_SETLK, &fl);
    }

int already_running(void)
{
    int fd;
    char buf[16];
    fd = open(LOCKFILE, O_RDWR|O_CREAT, LOCKMODE);
    if (fd == -1)
    {
        syslog(LOG_ERR, "невозможно открыть %s: %s", LOCKFILE,
strerror(errno));
        return(1);
    }
    if (lockfile(fd) == -1)
    {
        if (errno == EACCES || errno == EAGAIN)
        {
            close(fd);
            return(1);
        }
        syslog(LOG_ERR, "невозможно установить блокировку на %s: %s",
LOCKFILE, strerror(errno));
        return(1);
    }
    ftruncate(fd, 0);
    sprintf(buf, "%ld", (long)getpid());
    write(fd, buf, strlen(buf)+1);
    return(0);
}

void daemonize(const char *cmd)
{
    int i, fd0, fd1, fd2;
    pid_t pid;
    struct rlimit rl;
    struct sigaction sa;

    umask(0);

    if (getrlimit(RLIMIT_NOFILE, &rl) == -1)

```

```
{
    perror("Невозможно получить максимальный номер дескриптора");
    exit(1);
}

if ((pid = fork()) == -1)
{
    perror("Ошибка вызова функции fork");
    exit(1);
}
else if (pid > 0)
{
    exit(0);
}
setsid();

sa.sa_handler = SIG_IGN;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
if (sigaction(SIGHUP, &sa, NULL) == -1)
{
    perror("Невозможно игнорировать сигнал SIGHUP\n");
    exit(1);
}

if (chdir("/") == -1)
{
    perror("Невозможно сделать текущим рабочим каталогом / \n");
    exit(1);
}

if (rl.rlim_max == RLIM_INFINITY)
{
    rl.rlim_max = 1024;
}
for (i = 0; i < rl.rlim_max; i++)
{
    close(i);
}

fd0 = open("/dev/null", O_RDWR);
fd1 = dup(0);
```

```

    fd2 = dup(0);

    openlog(cmd, LOG_CONS, LOG_DAEMON);
    if (fd0 != 0 || fd1 != 1 || fd2 != 2)
    {
        syslog(LOG_ERR, "ошибочные файловые дескрипторы %d %d %d", fd0, fd1,
fd2);
        exit(1);
    }
}

void reread(void)
{
    FILE *fd;
    int pid;

    if ((fd = fopen(LOCKFILE, "r")) == NULL)
    {
        syslog(LOG_ERR, "Error opennig file %s.", LOCKFILE);
        exit(1);
    }

    fscanf(fd, "%d", &pid);
    fclose(fd);

    syslog(LOG_INFO, "==> Pid: %d", pid);
}

void* thr_fn(void *arg)
{
    int err, signo;
    for (;;)
    {
        syslog(LOG_INFO, "aaaaaaaaaaaaaaaaaaaa");
        err = sigwait(&mask, &signo);
        if (err != 0)
        {
            syslog(LOG_ERR, "can`t sigwait");
            exit(1);
        }
        switch (signo)
        {

```

```

        case SIGHUP:
            syslog(LOG_INFO, "Reading config file");
            reread();
            break;
        case SIGTERM:
            syslog(LOG_INFO, "get signal SIGTERM; Exit");
            exit(0);
        default:
            syslog(LOG_INFO, "get unpreictable signal %d\n", signo);
    }
}
pthread_exit(0);
}

int main(int argc, char *argv[])
{
    int err;
    pthread_t tid;
    char* cmd;
    struct sigaction sa;

    if ((cmd = strrchr(argv[0], '/')) == NULL)
        cmd = argv[0];
    else
        cmd++;

    daemonize(cmd);

    if (already_running())
    {
        syslog(LOG_ERR, "daemon already running");
        exit(1);
    }

    sa.sa_handler = SIG_DFL;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGHUP, &sa, NULL) == -1)
    {
        syslog(LOG_ERR, "can`t set SIG_DFL for SIGHUP\n");
        exit(1);
    }
}

```

```

sigfillset(&mask);
if ((err = pthread_sigmask(SIG_BLOCK, &mask, NULL)) != 0)
{
    syslog(LOG_ERR, "error while SIG_BLOCK\n");
    exit(1);
}

err = pthread_create(&tid, NULL, thr_fn, 0);
if (err != 0)
{
    syslog(LOG_ERR, "can't create thread\n");
    exit(1);
}

syslog(LOG_INFO, "Create new thread\n");

time_t timestamp;
struct tm *time_info;
for (;;)
{
    sleep(SLEEP_TIME);
    time(&timestamp);
    time_info = localtime(&timestamp);
    syslog(LOG_DAEMON, "%s", asctime(time_info));
}

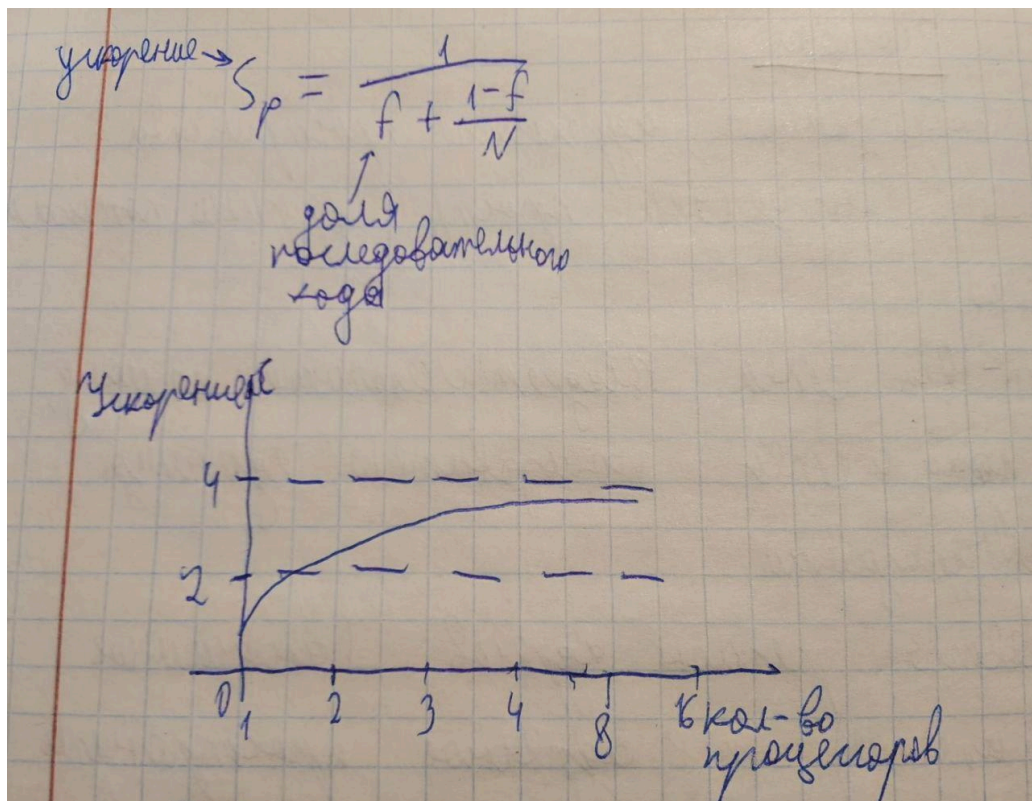
exit(0);
}

```

Потоки

- Потоки собственного адресного пространства не имеют
- Поток разделяет адресное пространство с другими потоками процесса
- Имеет свой стек ядра и стек пользователя
- Владеет счетчиком команд
- Является единицей диспетчеризации

- Выделяется регион адресов (анонимное отображение), то есть адресное пространство процесса расширяется
- Создание нового процесса можно рассматривать как дорогое действие, так как создается новое адресное пространство, таблицы страниц со ссылками и дескрипторами. Но почти сразу вызывается `exec` => напрасная работа `fork`
- Закон Амдала (сформулирован для задачи параллельных процессов)
 - Ускорение зависит не от числа процессоров, но также от величины последовательной части программы
 - $$S_p = \frac{1}{f + \frac{1-f}{N}}$$
 - f — доля последовательного кода



- Пример: 25% кода последовательно, 75% параллельно
 - На 2 ядерном в 1.6 раз
 - На 4 ядерном в 2.28 раза
- 4 физических ядра — реальное распараллеливание
- 4 логических ядра — оптимизация

Функции

- 2 способа создания потоков:
 - библиотека `pthread`

- (стандартный интерфейс для работы с потоками)
- вызов системного вызова clone()
- расширение fork для создания процессов или потоков

```
● int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
void*(*start)(void*), void *arg)
```

- *thread — будет хранить идентификатор потока TID
- attr (приоритет, размер стека)
- указатель на функцию, которую будет выполнять поток
- в случае успеха вернет 0, в случае ошибки — код ошибки или -1
- На потоках определены функции:
 - pthread_join()
 - блокирует вызывающий поток до завершения указанного потока
 - pthread_cancel() — досрочное завершение потока
 - только как запрос
 - pthread_kill()
 - позволяет передавать сообщения от одного потока к другому
- При этом основной поток и дополнительный поток будут выполняться параллельно
- Каждый процесс имеет уникальный идентификатор
- У потоков должны быть идентификаторы
- Но стандарт POSIX требует, чтобы у всех потоков многопоточной программы был один идентификатор
- Потоки многопоточных приложений группируются в группу потоков
- Группе присваивается идентификатор первого потока многопоточного приложения
- Чтобы получить thread_id нужно вызвать gettid()
- Эту функцию нужно определить с помощью __syscall
 - __syscall(pid_t, gettid)

Функции системного таймера

- Windows (обработчик прерывания:)

- По тикку
 - инкремент счетчика реального времени
 - декремент кванта текущего потока
 - декремент счетчиков времени до выполнения отложенных задач
- По главному тикку
 - инициализирует диспетчер настройки баланса путем сбрасывания объекта «событие», на котором он ожидает и выполняет профилирование (анализ времени выполнения потока в режиме ядра и в режиме пользователя)
- По кванту
 - инициализирует диспетчеризацию потоков, то есть ставит соответствующего объекта в очередь DPC
- UNIX
 - По тикку
 - инкремент счетчика времени с момента запуска системы
 - инкремент счетчика реального времени;
 - декремент счетчиков времени и установка флага для обработчиков отложенных вызовов при достижении счетчиками нулевого значения
 - инкремент счетчиков процессорного времени, полученного процессором в режиме пользователя и в режиме ядра
 - По главному тикку
 - инициализация отложенного вызова планировщика
 - инициализация отложенного вызова процедуры wakeup, которая меняет состояние процесса с «спящий» на «готовый к выполнению»;
 - пробуждение системных процессов, таких как pagedaemon
 - декремент счетчика времени, оставшегося до отправки одного из следующих сигналов:
 - – SIGPROF — сигнал, посылаемый процессу по истечении времени, заданного в счетчике тиков профилирования;
 - – SIGALRM — сигнал, посылаемый процессу по

истечении времени, заданного в «виртуальном» таймере;

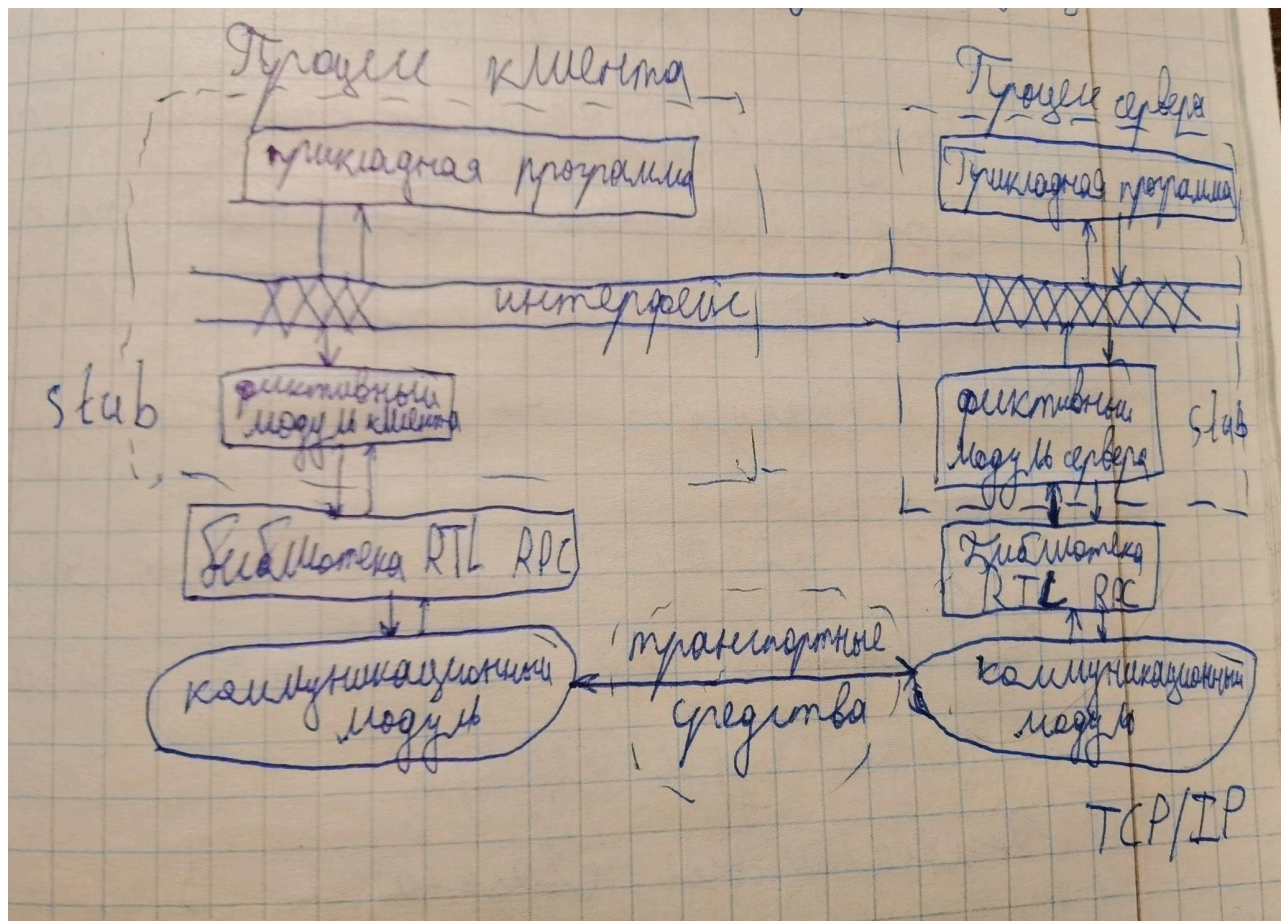
- – SIGVTALRM — сигнал, посылаемый процессу по истечении промежутка реального времени

- По кванту

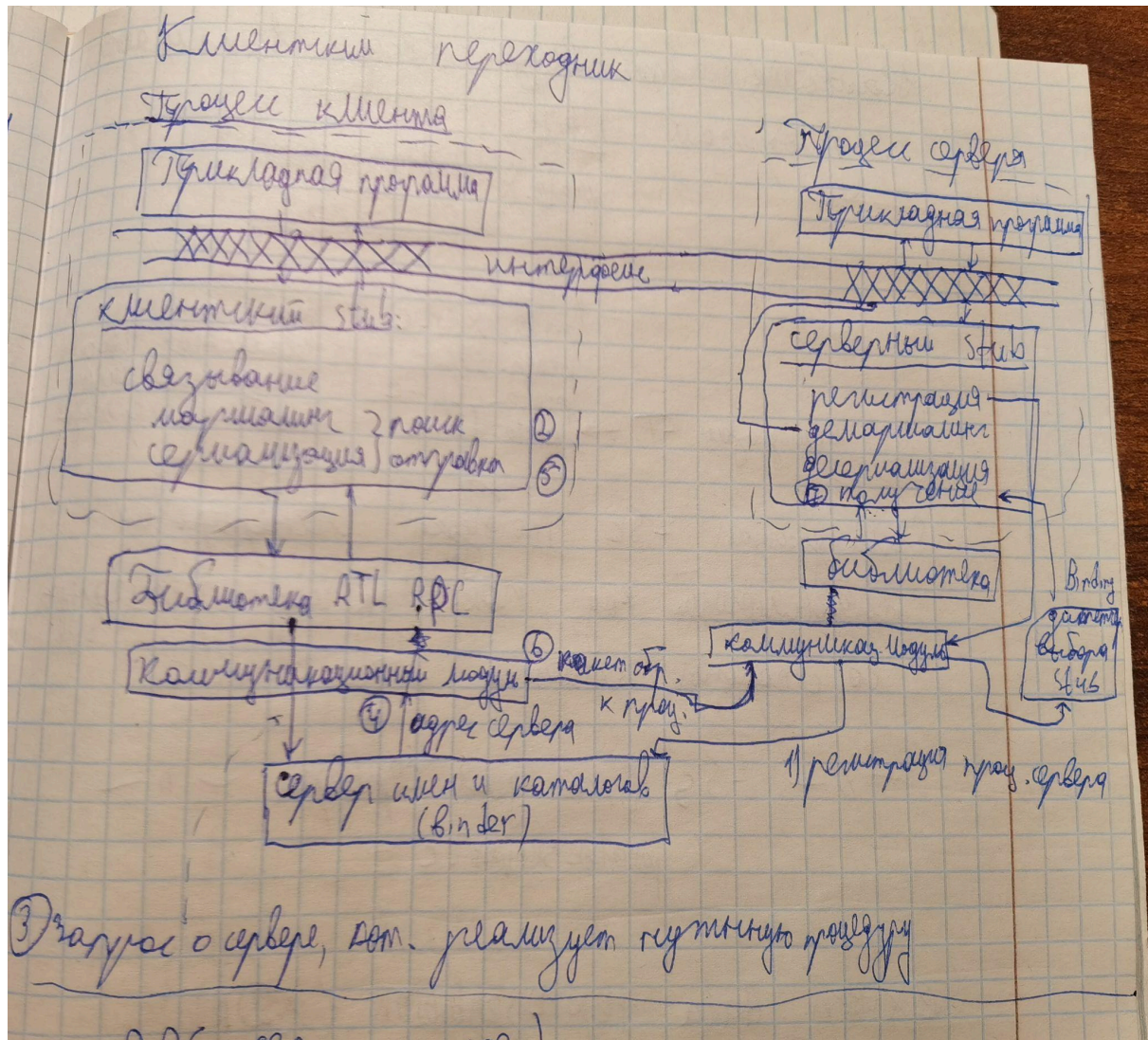
- посылает сигнал SIGXCPU текущему процессу, если он превысил выделенную для него квоту процессорного времени.

RPC

- Remote Procedure Calls
- Механизм, с помощью которого процесс инициализирует другой процесс на этой же или на другой машине от своего имени
- Напоминает вызов локальной функции, а именно процесс вызывает функцию и передает ей параметры, а затем ожидает, когда она вернет результат
- Специфика: вызываемую функцию выполняет другой процесс
- Такое взаимодействие обязательно выполняется по модели клиент-сервер
 - процесс, инициализирующий RPC — клиент
 - процесс, выполняющий вызов RPC-функции — сервер
- Смысл RPC — удаленный вызов выполняется как будто локальный
 - чтобы скрыть от пользователя все механизмы удаленного доступа



- Обращение из приложения через интерфейс транслируется в обращение к клиентскому стабу (заглушке)
- Выполняется через сокеты, но так как все механизмы от пользователя закрываются, сокеты не упоминаются
- Закрытие от пользователя всех особенностей взаимодействия дает возможность обращаться как к обычной программе
- Фигурный модуль клиента:
 - входные параметры от клиента преобразуются в одно или несколько сообщений (пакет)
- Фигурный модуль сервера
 - то же самое наоборот



RPCgen

- Компилятор протокола RPC
- Является инструментом, который генерирует код на языке C на основе протокола RPC
- rpc-language
- proto.x →
 - proto.h
 - proto_xdr.c (файл процедур xdr)
 - proto_svc.c (серверная часть)
 - proto_clnt.c (клиентская часть)
 - proto_server.c (скелетон сервера)
 - proto_client.c (скелетон клиента)
 - proto_tbl.c (если указан -T, таблицы переходов)

- Если указан -Sc — сгенерирован поясняющий код, показывающий процедуры со стороны клиента
- Если указан -Ss — поясняющий код для сервера
- XDR — extern data representation
 - процедуры, выполняющие преобразование данных, к которым выполняется обращение write и read на нескольких типах машин
- -a для генерации скелетонов

Процесс-сервер		Процесс-сервер	
Приложение	Вызов клиентского стаба ↓	Выполнение запрошенной работы ↑	Приложение
Клиентский стаб	Создается буфер	Вызвать приложение-сервер	Серверный стаб
	Упорядочивание параметров в буфере	Поместить параметры в стек	
	Добавление заголовка		
	Системный вызов (переход в режим ядра) ↓	Распаковать параметры ↑	
Ядро	Копирование сообщения в ядро	Переключение в режим пользователя	Ядро
	Определение адреса назначения	Определить, ждет ли стаб	
	Поместить адрес в заголовок сообщения	Определить стаб назначения	
	Установить сетевой интерфейс	Подтвердить правильность	

		пакета	
	Включить таймер	Прервать выполняемую работу (аппаратное прерывание сетевой подсистемы)	

- Нужно отметить в очереди пришедшее сообщение
- Все остальные действия отложенные
- Будут выполняться обработчиком отложенных действий

bakery.x

```

struct BAKERY
{
    int num;

    int id;

    int result;
};

program BAKERY_PROG
{
    version BAKERY_VER
    {
        struct BAKERY GET_NUMBER(struct BAKERY) = 1;

        struct BAKERY WAIT_QUEUE(struct BAKERY) = 2;
    }
}

```

```

    struct BAKERY BAKERY_RESULT(struct BAKERY) = 3;

    } = 1; /* Version number = 1 */

} = 0x20000001;

```

bakery_client.c

```

/*

 * This is sample code generated by rpcgen.

 * These are only templates and you can use them

 * as a guideline for developing your own functions.

 */

#include "bakery.h"

#include <stdio.h>

#include <unistd.h>

#include <time.h>


void

bakery_prog_1(char *host)

{

    CLIENT *clnt;

```

```

    struct BAKERY *result_1;

    struct BAKERY get_number_1_arg;

    struct BAKERY *result_2;

    struct BAKERY wait_queue_1_arg;

    struct BAKERY *result_3;

    struct BAKERY bakery_result_1_arg;

#ifdef DEBUG

    clnt = clnt_create (host, BAKERY_PROG, BAKERY_VER, "udp");

    if (clnt == NULL) {

        clnt_pcreateerror (host);

        exit (1);

    }

#endif /* DEBUG */

    srand(time(NULL));

    sleep(rand() % 4 + 1);

    get_number_1_arg.id = getpid();

    result_1 = get_number_1(&get_number_1_arg, clnt);

    if (result_1 == (struct BAKERY *) NULL) {

        clnt_perror (clnt, "call failed");

```

```

        exit(1);

    }

    printf("Client %d get number %d\n", get_number_1_arg.id, result_1->num);

    sleep(rand() % 4 + 1);

    wait_queue_1_arg.id = result_1->id;

    wait_queue_1_arg.num = result_1->num;

    result_2 = wait_queue_1(&wait_queue_1_arg, clnt);

    if (result_2 == (struct BAKERY *) NULL) {

        clnt_perror (clnt, "call failed");

        exit(1);

    }

    sleep(rand() % 4 + 1);

    bakery_result_1_arg.id = wait_queue_1_arg.id;

    bakery_result_1_arg.num = wait_queue_1_arg.num;

    result_3 = bakery_result_1(&bakery_result_1_arg, clnt);

    if (result_3 == (struct BAKERY *) NULL) {

        clnt_perror (clnt, "call failed");

        exit(1);

    }

    printf("Client %d get %c\n", get_number_1_arg.id, result_3->result);

#ifdef DEBUG

```



```

        clnt_destroy (clnt);

#ifdef  /* DEBUG */

}

int

main (int argc, char *argv[])
{
    char *host;

    if (argc < 2) {

        printf ("usage: %s server_host\n", argv[0]);

        exit (1);

    }

    host = argv[1];

    bakery_prog_1 (host);

exit (0);

}

```

bakery_server.c

```

/*

* This is sample code generated by rpcgen.

```

```
* These are only templates and you can use them

* as a guideline for developing your own functions.

*/

#include "bakery.h"

#include <stdio.h>

#include <unistd.h>

#include <pthread.h>

#include <sys/syscall.h>

#include <stdbool.h>

struct thread_arg_t {

    int id;

    int num;

    int result;

};

bool choosing[40] = { false };

int number[40] = { 0 };

int num = 0;

int symbol = 'a';
```

```
int id_create_thread = 0;

int id_joined_thread = 0;

pthread_t clients[40];

struct thread_arg_t threads_res[40] = { 0 };


void *bakery(void *arg)
{
    struct thread_arg_t *th_arg = arg;

    int i = th_arg->id;

    for (int j = 0; j < 40; ++j)
    {
        while (choosing[j]);

        while ((number[j] > 0) && (number[j] < number[i] || (number[j] ==
number[i] && j < i)));

    }

    th_arg->result = symbol;

    symbol++;

    sleep(5);

    number[i] = 0;

    return 0;
}
```

```
struct BAKERY *get_number_1_svc(struct BAKERY *argp, struct svc_req *rqstp)
{
    static struct BAKERY result;

    int i = num;

    num++;

    result.id = i;

    choosing[i] = true;

    int max = 0;

    for (int j = 0; j < 40; ++j)
    {
        if (max < number[j])
        {
            max = number[j];
        }
    }

    number[i] = max + 1;

    result.num = number[i];

    choosing[i] = false;

    return &result;
}
```

```

struct BAKERY *wait_queue_1_svc(struct BAKERY *argp, struct svc_req *rqstp)
{
    static struct BAKERY result;

    pthread_t thread;

    threads_res[id_create_thread].id = argp->id;

    int err = pthread_create(&thread, NULL, bakery,
&threads_res[id_create_thread]);

    if (err != 0)
    {
        printf("can't create thread\n");

        return NULL;
    }

    clients[id_create_thread] = thread;

    id_create_thread++;

    return &result;
}

```

```

struct BAKERY *bakery_result_1_svc(struct BAKERY *argp, struct svc_req *rqstp)
{

```

```
static struct BAKERY result;

int err = pthread_join(clients[id_joined_thread], NULL);

if (err != 0)
{
    printf("can't join thread\n");

    return NULL;
}

result.id = argp->id;

result.result = threads_res[id_joined_thread].result;

id_joined_thread++;

return &result;
}
```